

CSE 402: Numerical Methods

Complete Practice Booklet

Derivations · Worked Simulations · MCQ Bank · Trap Questions · Mock Quiz

September 21, 2026

Key Point / Exam Tip

Scope. Full syllabus: Errors & Approximations, Root Finding, Systems of Linear Equations, Eigenvalue Decomposition, Optimization (Golden Section, Gradient Descent, Newton’s Method, Lagrange Multipliers), Regression, Interpolation, Simulation & Modeling, and Random Number Generation & Monte Carlo.

How the three parts work together.

- **Part I — Theory & Derivations.** Every proof and worked numeric simulation. Reproduce each boxed derivation from memory.
- **Part II — Problem Bank.** MCQs, multi-step problems and *trap questions* per topic. This is where “conceptually basic but hard to answer” questions live: the ones where the obvious answer is wrong.
- **Part III — Mock Quiz.** A full mixed paper under time pressure, with an answer key. Sit it cold — mixed topics are far harder than topic-by-topic drilling, and that is what the real quiz feels like.

Contents

Part I — Theory, Derivations & Worked Simulations	4
1 Introduction, Approximations & Errors	4
1.1 Error Definitions	4
1.2 Significant Digits	5
1.3 Round-off and Truncation Error	5
2 Root Finding	7
2.1 Bisection Method	7
2.2 False-Position Method	8
2.3 Newton–Raphson Method	9
3 Systems of Linear Equations	11
3.1 Naive Gauss Elimination	11
3.2 Partial Pivoting	12
3.3 Determinant via Elimination	12
3.4 Gauss–Jordan Elimination	12
3.5 LU Decomposition	13
4 Eigenvalue Decomposition (Power Method)	14
4.1 Eigen Decomposition $A = V\Lambda V^{-1}$	14

4.2	The Power Method	15
4.3	Inverse Power Method (Smallest Eigenvalue)	15
4.4	Deflation (Intermediate Eigenvalues)	16
5	Optimization	17
5.1	Golden Section Search	17
5.2	Gradient Descent (GD) and Linear Regression	18
5.3	Newton's Method for Optimization	20
5.4	Foundations: Gradient and Stationary Points	21
5.5	Constrained Optimization (Lagrange Multipliers)	22
6	Regression	22
6.1	Least-Squares Linear Regression	23
6.2	Multiple Linear Regression	24
7	Interpolation	25
7.1	Lagrange Interpolation	25
7.2	Linear Interpolation	26
7.3	Newton's Divided-Difference Interpolation	26
7.4	Spline Interpolation	27
8	Simulation & Modeling	28
8.1	Introduction to Simulation and Modeling	28
8.2	Discrete-Event Simulation & the Single-Server Queue	30
8.3	Steps in a Simulation Study	32
8.4	Model Verification and Validation	33
9	Random Number Generation & Monte Carlo Simulation	35
9.1	Random Number Generation	35
9.1.1	The Linear Congruential Generator (LCG)	35
9.1.2	Statistical Testing of RNGs	36
9.2	Monte Carlo Simulation	38
	Part II — Problem Bank	41
10	Errors & Root Finding — Problem Bank	41
10.1	Errors — MCQs, Problems & Traps	41
10.2	Root Finding — MCQs, Problems & Traps	43
11	Linear Systems & Eigenvalues — Problem Bank	46
11.1	Systems of Equations — MCQs, Problems & Traps	47
11.2	Eigenvalues — MCQs, Problems & Traps	49
12	Optimization — Problem Bank	51
12.1	Optimization — MCQs	51
12.2	Optimization — Problems	53
13	Regression & Interpolation — Problem Bank	55
13.1	Regression — MCQs, Problems & Traps	55
13.2	Interpolation — MCQs, Problems & Traps	57
14	Simulation, RNG & Monte Carlo — Problem Bank	60
14.1	Simulation & Modeling — MCQs	60
14.2	RNG & Monte Carlo — MCQs	61
14.3	Simulation, RNG & Monte Carlo — Problems	62

Part III — Mock Quiz**65****15 Mock Quiz — Full Mixed Paper (attempt under exam conditions)****65**

15.1 Part A — Multiple Choice (35 marks) 65

15.2 Part B — Short Answer (20 marks) 67

15.3 Part C — Problems (40 marks) 67

15.4 Answer Key 69

Part I — Theory, Derivations & Worked Simulations

Every proof and every step-by-step numeric run

1 Introduction, Approximations & Errors

Definition

Engineering problems become mathematical models; many models have no closed-form (pencil-and-paper) solution, so they are solved *approximately* — that approximation process is numerical methods. Pipeline: physical problem → mathematical model → numerical method → approximate solution + error estimate.

1.1 Error Definitions

True error (needs the exact value, rarely available):

$$E_t = \text{True value} - \text{Approximate value}, \quad \epsilon_t = \frac{E_t}{\text{True value}}$$

Approximate error (computed from successive iterates — this is what's actually monitored):

$$E_a = \text{Present approx.} - \text{Previous approx.}, \quad \epsilon_a = \frac{E_a}{\text{Present approx.}}$$

On the very first iteration there is no previous estimate, so ϵ_a is undefined.

Why relative error, not absolute? Rescaling a problem by a constant factor changes the raw error's apparent size without changing real accuracy — only relative error meaningfully reflects accuracy.

Stopping an iteration: either $|\epsilon_a| \leq \epsilon_s$ (a chosen tolerance), or $|\epsilon_a| \leq 0.5 \times 10^{2-m}\%$ (guarantees at least m correct significant digits).

Worked Example

Example — forward-difference derivative. $f(x) = 7e^{0.5x}$, forward difference with $h = 0.3$ at $x = 2$:

$$f'(2) \approx \frac{7e^{0.5(2.3)} - 7e^{0.5(2)}}{0.3} = \frac{22.107 - 19.028}{0.3} = 10.265$$

Exact: $f'(x) = 3.5e^{0.5x} \Rightarrow f'(2) = 3.5e^1 = 9.5140$.

$$E_t = 9.5140 - 10.265 = -0.75061, \quad \epsilon_t = \frac{-0.75061}{9.5140} = -7.8895\%$$

(If $f(x) = 7 \times 10^{-6}e^{0.5x}$ instead, every value shrinks by 10^{-6} so E_t looks $10^6 \times$ smaller though the problem is identical — this is exactly why relative error is preferred.)

With $h = 0.15$: $f'(2) \approx 9.8799 \Rightarrow E_a = 9.8799 - 10.265 = -0.38474$, $\epsilon_a = \frac{-0.38474}{9.8799} = -3.8942\%$.

1.2 Significant Digits

Definition

Significant figures report how much you trust a number (“population = 1 million” has 1 s.f.; the exact count 1,079,587 has 7 s.f.). Decimal places $\neq$ significant figures: 0.00640 has 5 decimal places but only 3 significant figures (6,4,0).

Normalized scientific notation pins one non-zero digit before the decimal: $|x| = m \times 10^n$, $1 \leq m < 10$, $n = \lfloor \log_{10} |x| \rfloor$. The M -th significant digit then sits at place 10^{n-M+1} .

Worked Example

Proof — equivalence of the relative-error and absolute-error criteria for “correct to M sig. digits”.

Claim: $\left| \frac{x - \hat{x}}{x} \right| \leq 0.5 \times 10^{2-M}\% \Rightarrow \hat{x}$ correct to at least M sig. digits. Since “%” means $\times 10^{-2}$:

$$0.5 \times 10^{2-M}\% = 0.5 \times 10^{2-M} \times 10^{-2} = \frac{1}{2} \times 10^{-M}$$

Target (absolute-error) form: $|x - \hat{x}| \leq \frac{1}{2} \times 10^{n-M+1}$.

Starting from $\left| \frac{x - \hat{x}}{x} \right| \leq \frac{1}{2} \times 10^{-M}$ and $|x| = m \cdot 10^n$:

$$|x - \hat{x}| = \left| \frac{x - \hat{x}}{x} \right| \cdot m \cdot 10^n \leq \frac{1}{2} \times 10^{-M} \cdot m \cdot 10^n = \frac{m}{2} \times 10^{n-M}$$

Since $m < 10$:

$$|x - \hat{x}| < \frac{10}{2} \times 10^{n-M} = 5 \times 10^{n-M} = \frac{1}{2} \times 10^{n-M+1}$$

which is exactly the target window — so $\hat{x}$ is correct to at least M significant digits. ■

Worked Example

Worked examples.

$3725.6 = 3.7256 \times 10^3$: leading digit sits at 10^3 ; the 3rd sig. digit (“2”) sits at $10^{3-3+1} = 10^1$.

For $\pi = 3.14159 \dots$ ($m = 3.14159$, $n = 0$), correct to $M = 3$ figures means $|x - \hat{x}| \leq \frac{1}{2} \times 10^{0-3+1} = 0.005$. Rounding $\hat{x} = 3.14$: $|\pi - 3.14| = 0.00159 \leq 0.005 \Rightarrow$ **3.14 is correct to 3 sig. figures.**

Maclaurin series for $e^{0.7}$: with 6 terms $e^{0.7} \approx 2.0136$, with 5 terms ≈ 2.0122 .

$$|\epsilon_a| = \left| \frac{2.0136 - 2.0122}{2.0136} \right| \times 100\% = 0.069527\%$$

For $m = 2$: threshold = $0.5 \times 10^0\% = 0.5\%$. Since $0.069527\% \leq 0.5\%$, **at least 2 sig. digits correct using 6 terms.**

1.3 Round-off and Truncation Error

Round-off error: finite-precision storage. On a machine keeping 6 digits, $\frac{1}{3} \rightarrow 0.333333$:

$$\text{round-off} = \frac{1}{3} - 0.333333 = 3.3333 \dots \times 10^{-7}$$

Truncation error: cutting short an infinite/exact procedure — appears in (a) truncated series, (b) finite-difference derivatives, (c) numerical integration; refining the discretization always shrinks it.

Worked Example

Truncation via a series — full table. Compute $e^{1.2} = \sum \frac{1.2^k}{k!}$, stopping once ≥ 2 sig. digits trusted:

n	S_n	E_a	$ \epsilon_a \%$	#s.d.
1	1.0000	—	—	—
2	2.2000	1.2	54.546	0
3	2.9200	0.72	24.658	0
4	3.2080	0.288	8.9776	0
5	3.2944	0.0864	2.6226	1
6	3.3151	0.020736	0.62550	1
7	3.3193	0.0041472	0.12494	2

At $n = 7$: $|\epsilon_a| = 0.12494\% \leq 0.5\% \Rightarrow$ **stop with ≥ 2 sig. digits.**

Truncation via a derivative. $f(x) = x^2$, exact $f'(3) = 6$. With $\Delta x = 0.2$:

$$f'(3) \approx \frac{3.2^2 - 3^2}{0.2} = \frac{10.24 - 9}{0.2} = 6.2, \quad \text{trunc. error} = 6 - 6.2 = -0.2$$

Truncation via integration. Exact: $\int_3^9 x^2 dx = \left[\frac{x^3}{3}\right]_3^9 = \frac{729-27}{3} = 234$.

- 2 rectangles (width 3, heights 9, 36): $9(3) + 36(3) = 135$; error = $234 - 135 = 99$.
- 4 rectangles (width 1.5, heights 9, 20.25, 36, 56.25): $1.5(9 + 20.25 + 36 + 56.25) = 182.25$; error = $234 - 182.25 = 51.75$.

More rectangles $\Rightarrow$ smaller gap $\Rightarrow$ less error.

Worked Example

Case study — the bascule-bridge trunnion (why models matter, not just arithmetic).

Trunnion OD = 12.363", hub ID = 12.358", required clearance = 0.01" $\Rightarrow$ required contraction = 0.015". Cooling bath at -108°F , room $80^\circ\text{F} \Rightarrow \Delta T = -188^\circ\text{F}$.

Wrong model (constant $\alpha = 6.47 \times 10^{-6}$, room-temp. value):

$$\Delta D = (12.363)(6.47 \times 10^{-6})(-188) = -0.01504''$$

$0.01504'' > 0.015''$ required $\Rightarrow$ looks fine on paper.

Correct model — α actually varies with T ; fit $\alpha(T) = a_0 + a_1T + a_2T^2$ to measured data and integrate:

$$\Delta D = D \int_{T_{\text{room}}}^{T_{\text{fluid}}} \alpha(T) dT = -0.013689''$$

$0.013689'' < 0.015''$ required $\Rightarrow$ the trunnion did *not* shrink enough — it got stuck. **This was a modelling error, not a calculation error.**

Q1: What is the difference between true error and approximate error?

A: E_t = true – approx. needs the exact answer (rarely known). E_a = present – previous approximation uses only successive iterates — hence it is what's actually monitored in practice.

Q2: Why is relative error preferred over absolute error?

A: Absolute error magnitude is misleading — rescaling the same problem changes the raw error's apparent size without changing real accuracy. Relative error normalizes by the answer's size.

Q3: State the two stopping rules for an iterative computation.

A: (i) $|\epsilon_a| \leq \epsilon_s$, a chosen tolerance; (ii) $|\epsilon_a| \leq 0.5 \times 10^{2-m}\%$, guaranteeing $\geq m$ correct significant digits.

Q4: What does “correct to M significant digits” mean precisely?

A: $|x - \hat{x}| \leq \frac{1}{2} \times 10^{n-M+1}$ where $n = \lfloor \log_{10} |x| \rfloor$ — the error is at most half a unit in the place of

the M -th significant digit; equivalent to the relative-error bound $0.5 \times 10^{2-M}\%$.

Q5: What two error types are inherent to numerical methods even with a perfect model and program?

A: Round-off error (finite-precision storage) and truncation error (cutting short an infinite/exact mathematical procedure).

Q6: Why did the trunnion example fail even though the arithmetic was correct?

A: The thermal-expansion coefficient α actually varies with temperature; using a single room-temperature value overestimated the contraction. Integrating the true $\alpha(T)$ gives a smaller contraction than required — a modelling error, not a computational one.

Q7: Why does truncation error in rectangle-based integration shrink with more rectangles?

A: More, narrower rectangles trace the curve more closely, shrinking the gap between the curve and the rectangle tops (e.g. $\int_3^9 x^2 dx = 234$: error 99 with 2 rectangles vs. 51.75 with 4).

Q8: What three steps does the course prescribe for handling unavoidable error?

A: (1) Identify the source; (2) quantify it with computable error measures; (3) minimise it to the accuracy actually needed.

2 Root Finding

Definition

Find x such that $f(x) = 0$. Transcendental equations ($e^x = 3x$, $\sin x = x/2$) have no closed form, so roots are approximated iteratively until the error is small enough. Running example (all three methods): a rubber ball, specific gravity 0.6, radius $R = 5.5$ cm floats in water with submerged depth x satisfying

$$f(x) = x^3 - 0.165x^2 + 3.993 \times 10^{-4} = 0, \quad 0 \leq x \leq 0.11.$$

2.1 Bisection Method

Intermediate Value Theorem. If f is real and continuous and $f(x_\ell)f(x_u) < 0$, then $f(x) = 0$ has *at least one* root in (x_ℓ, x_u) — never guaranteed exactly one; if $f(x_\ell)f(x_u) > 0$ there may or may not be a root.

Algorithm.

1. Choose x_ℓ, x_u with $f(x_\ell)f(x_u) < 0$.
2. $x_m = \frac{x_\ell + x_u}{2}$.
3. $f(x_\ell)f(x_m) < 0 \Rightarrow x_u = x_m$; $> 0 \Rightarrow x_\ell = x_m$; $= 0 \Rightarrow$ done.
4. $|\epsilon_a| = \left| \frac{x_m^{\text{new}} - x_m^{\text{old}}}{x_m^{\text{new}}} \right| \times 100\%$.
5. Repeat until $|\epsilon_a| \leq \epsilon_s$.

Worked Example

Worked simulation — floating ball. $f(0) = +3.993 \times 10^{-4}$, $f(0.11) = -2.662 \times 10^{-4}$: product < 0 , root guaranteed in $[0, 0.11]$.

Iter	x_ℓ	x_u	x_m	$ \epsilon_a \%$
1	0.00000	0.11000	0.05500	—
2	0.05500	0.11000	0.08250	33.33
3	0.05500	0.08250	0.06875	20.00
4	0.05500	0.06875	0.06188	11.11
5	0.06188	0.06875	0.06531	5.263
6	0.06188	0.06531	0.06359	2.702
7	0.06188	0.06359	0.06273	1.370
8	0.06188	0.06273	0.06230	0.6897
9	0.06230	0.06273	0.06252	0.3436
10	0.06230	0.06252	0.06241	0.1721

Digit check: $0.1721 \leq 0.5 \times 10^{2-m} \Rightarrow m \leq 2.463 \Rightarrow m = 2$. **Result:** $x_m = 0.06241$, ≥ 2 **correct digits, after 10 iterations.**

Advantages. Always convergent (bracket always halves $\Rightarrow$ guaranteed); predictable error (iteration count for a target accuracy can be computed in advance).

Drawbacks. Slow (linear convergence); wastes proximity information — always jumps to the exact midpoint even if one endpoint is already very near the root.

Failure cases. (1) Roots that only *touch* the axis without a sign change, e.g. $f(x) = x^2$ at $x = 0$ — no valid bracket exists. (2) Singularities that flip sign, e.g. $f(x) = 1/x$ with $x_\ell = -2, x_u = 3$: $f(x_\ell)f(x_u) < 0$ but there is *no root* — bisection converges to the singularity instead.

2.2 False-Position Method

Worked Example

Derivation. The secant line through $(x_L, f(x_L))$ and $(x_U, f(x_U))$ crosses the axis at x_r ; by similar triangles:

$$\frac{0 - f(x_L)}{x_r - x_L} = \frac{0 - f(x_U)}{x_r - x_U}$$

Cross-multiplying: $x_U f(x_L) - x_L f(x_U) = x_r \{f(x_L) - f(x_U)\}$, so

$$x_r = \frac{x_U f(x_L) - x_L f(x_U)}{f(x_L) - f(x_U)} = x_U - \frac{f(x_U)(x_U - x_L)}{f(x_U) - f(x_L)}. \quad \blacksquare$$

Algorithm. Identical to bisection except step 2 uses the x_r formula above instead of the midpoint; steps 1, 3–5 are unchanged (“the only difference from bisection is the formula in steps 2 and 4”).

Worked Example

Worked simulation — same floating ball.

Iter	x_L	x_U	x_r ($ \epsilon_a \%$)
1	0.0000	0.1100	0.0660 (—)
2	0.0000	0.0660	0.0611 (8.00)
3	0.0611	0.0660	0.0624 (2.05)
4	0.0611	0.0624	0.06238 (0.020)
5	0.0611	0.06238	0.06238 (0.0001)

Note: x_L freezes at 0.0611 for iterations 3–5 while $f(x_r)$ shrinks $10^{-5} \rightarrow 10^{-13}$ — one endpoint gets “stuck.” **Result:** **5 iterations reach ~ 5 correct digits** — faster here than bisection’s 10

iterations for 2 digits.

Worked Example

Worked simulation 2. $f(x) = (x - 4)^2(x + 2)$, $x_L = -2.5$, $x_U = -1.0$, $\epsilon_s = 0.1\%$.

Iter	x_L	x_U	x_r	$ \epsilon_a \%$
1	-2.5	-1.000	-1.813	—
2	-2.5	-1.813	-1.971	8.024
3	-2.5	-1.971	-1.996	1.229
4	-2.5	-1.996	-1.999	0.1828
5	-2.5	-1.999	-2.000	0.02706

Result: converges to $x = -2$ in 5 iterations, ≥ 3 correct digits ($0.02706 \leq 0.5 \times 10^{2-m} \Rightarrow m \leq 3.27$).

Key Point / Exam Tip

Can false-position be slower than bisection? Yes. For $f(x) = \ln x$, bracket $[10^{-4}, 10^4]$, stopping at $|\epsilon_a| < 0.0001\%$: bisection took **34 iterations (5418 ns)**; false-position took **126 iterations (14753 ns)** — bisection was $2.7\times$ faster. Reason: an endpoint can stay frozen for many rounds while its function value shrinks only slowly, dragging out convergence.

2.3 Newton–Raphson Method

Worked Example

Derivation. At $(x_i, f(x_i))$ the tangent's slope equals $f'(x_i) = \tan \theta = \frac{f(x_i) - 0}{x_i - x_{i+1}}$, where x_{i+1} is where the tangent meets the axis. Solving:

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}. \quad \blacksquare$$

Algorithm. (1) Evaluate $f'(x)$. (2) $x_{i+1} = x_i - f(x_i)/f'(x_i)$. (3) $|\epsilon_a| = \left| \frac{x_{i+1} - x_i}{x_{i+1}} \right| \times 100\%$. (4) Repeat until $|\epsilon_a| \leq \epsilon_s$.

Worked Example

Worked simulation — same floating ball. $f'(x) = 3x^2 - 0.33x$. Start $x_0 = 0.05$ ($x_0 = 0$ or 0.11 would give $f' = 0$).

$$x_1 = 0.05 - \frac{1.118 \times 10^{-4}}{-9 \times 10^{-3}} = 0.06242, \quad |\epsilon_a| = 19.90\% \text{ (0 digits)}$$

$$x_2 = 0.06242 - \frac{3.97781 \times 10^{-7}}{8.90973 \times 10^{-3}} = 0.06238, \quad |\epsilon_a| = 0.0716\% \text{ (2 digits)}$$

$$x_3 = 0.06238 - \frac{4.44 \times 10^{-11}}{8.91171 \times 10^{-3}} = 0.06238, \quad |\epsilon_a| \approx 0\% \text{ (4 digits)}$$

Result: 3 iterations, $19.9\% \rightarrow 0$; **correct digits roughly double each step (quadratic convergence)** — far faster than bisection (10 iter.) or false-position (5 iter.) on the same problem.

Advantages over bracketing methods. Faster (fewer iterations); quadratic convergence (correct digits double each step). **Disadvantage.** Not guaranteed to converge (needs only 1 guess, no bracket).

Key Point / Exam Tip**Four Newton–Raphson failure modes (memorize all four):**

1. **Division by zero / near-zero derivative.** E.g. ball’s guesses $x_0 = 0, 0.11$ give $f' = 0$ exactly. For $f(x) = x^3 - 0.03x^2 + 2.4 \times 10^{-6}$ (f' vanishes at $x = 0, 0.02$), starting near $x_0 = 0.01999$ avoids an exact crash but the tiny denominator still causes wild, unstable steps — fails to converge after 9 iterations.
2. **Divergence near an inflection point.** $f(x) = (x - 1)^3 + 0.512$, $x_0 = 5.0$: the iterate nears the inflection $x = 1$ (where $f' = 0$), the step explodes to $x = -30.119$, then crawls back, finally reaching $x \approx 0.2$ after 18 iterations — a bracketing method would have boxed this in immediately.
3. **Oscillation near a local max/min (no real root).** $f(x) = x^2 + 2 = 0$ has no real roots; starting $x_0 = -1.0$, iterates bounce around the minimum forever and never settle (eventually diverge).
4. **Root jumping.** $f(x) = \sin x = 0$ has infinitely many roots; starting at $x_0 = 2.4\pi \approx 7.54$ (intending $2\pi \approx 6.283$), the iterates leap across several roots and converge to $x = 0$ instead.

Q1: What does the Intermediate Value Theorem guarantee for bisection, and what does it *not* guarantee?

A: If f is continuous and $f(x_\ell)f(x_u) < 0$, at least one root exists in (x_ℓ, x_u) — never exactly one; multiple roots are possible, and $f(x_\ell)f(x_u) > 0$ says nothing either way.

Q2: Why is bisection guaranteed to converge?

A: It always keeps a bracket around the root and halves it every iteration, so the interval width shrinks to zero — this also lets you compute the number of iterations needed in advance.

Q3: What is bisection’s main weakness compared to false-position?

A: It always takes the exact midpoint regardless of how close an endpoint’s function value already is to zero, wasting that proximity information — converges only linearly.

Q4: Give two situations where the sign-change bracketing test misleads bisection.

A: (1) A root that only touches the axis without changing sign, e.g. $f(x) = x^2$ at $x = 0$ — no valid bracket exists. (2) A singularity that flips sign, e.g. $f(x) = 1/x$ on $[-2, 3]$ — satisfies $f(x_\ell)f(x_u) < 0$ but there is no real root; bisection converges to the singularity.

Q5: Can false-position be slower than bisection — what evidence shows this?

A: Yes: for $f(x) = \ln x$ on $[10^{-4}, 10^4]$ with $|\epsilon_a| < 0.0001\%$, bisection took 34 iterations vs. false-position’s 126 — bisection was $2.7\times$ faster, because false-position can leave one endpoint “stuck” for many rounds.

Q6: What is the fundamental difference between Newton–Raphson and the bracketing methods?

A: Newton–Raphson is an open method (needs only 1 guess, no bracket); not guaranteed to converge, but converges quadratically (fast) when it does — versus bracketing methods, which always converge but only linearly (slow).

Q7: Why is $x_0 = 0$ or $x_0 = 0.11$ a bad initial guess in the floating-ball Newton–Raphson example?

A: $f'(x) = 3x(x - 0.11)$ vanishes at both, giving a horizontal tangent and division by zero.

Q8: What is an inflection point, and does $f''(c) = 0$ always mean one is present?

A: A point where concavity flips sign; requires $f'(c)$ to exist and f'' to *change sign* at c . $f''(c) = 0$ alone is not sufficient — e.g. $f(x) = x^4 - 16$ has $f''(0) = 0$ but no concavity flip there.

Q9: What is “root jumping” and which example illustrates it?

A: When a function has many roots (e.g. $\sin x = 0$), a Newton step can overshoot past the intended nearby root and converge to a different one entirely; starting at $x_0 = 2.4\pi$ aiming for 2π , the iteration lands on $x = 0$ instead.

Q10: Compare convergence speed of all three methods on the floating-ball problem.

A: Bisection: 10 iterations for 2 correct digits. False-position: 5 iterations for ~ 5 digits. Newton–Raphson: 3 iterations, digits roughly doubling each step ($0 \rightarrow 2 \rightarrow 4$) — demonstrating quadratic vs.

linear convergence.

3 Systems of Linear Equations

Definition

Solve $[A]\{X\} = \{B\}$: n equations, n unknowns. **Gauss elimination** has two steps: (1) *forward elimination* — knock out unknowns until the system is upper triangular; (2) *back substitution* — solve the last (single-unknown) equation, then substitute upward.

3.1 Naive Gauss Elimination

Multiplier: $m_{i1} = a_{i1}/a_{11}$, used via $R_i \leftarrow R_i - m_{i1}R_1$. General step k :

$$m_{ik} = \frac{a_{ik}^{(k-1)}}{a_{kk}^{(k-1)}}, \quad a_{ij}^{(k)} = a_{ij}^{(k-1)} - m_{ik}a_{kj}^{(k-1)}, \quad i, j > k$$

Back substitution: $x_n = b_n^{(n-1)}/a_{nn}^{(n-1)}$, then $x_i = \left(b_i^{(i-1)} - \sum_{j>i} a_{ij}^{(i-1)}x_j\right)/a_{ii}^{(i-1)}$ for $i = n-1, \dots, 1$.

Worked Example

Worked example — rocket velocity fit. Fit $v(t) = a_1t^2 + a_2t + a_3$ to $(5, 106.8), (8, 177.2), (12, 279.2)$:

$$\begin{bmatrix} 25 & 5 & 1 \\ 64 & 8 & 1 \\ 144 & 12 & 1 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 106.8 \\ 177.2 \\ 279.2 \end{bmatrix}$$

Step 1 ($m_{21} = 2.56$, $m_{31} = 5.76$):

$$\begin{bmatrix} 25 & 5 & 1 & 106.8 \\ 0 & -4.8 & -1.56 & -96.208 \\ 0 & -16.8 & -4.76 & -335.968 \end{bmatrix}$$

Step 2 ($m_{32} = 3.5$):

$$\begin{bmatrix} 25 & 5 & 1 & 106.8 \\ 0 & -4.8 & -1.56 & -96.208 \\ 0 & 0 & 0.7 & 0.76 \end{bmatrix}$$

Back substitution: $a_3 = 1.08571$, $a_2 = 19.6905$, $a_1 = 0.290472$.

$$v(t) = 0.290472t^2 + 19.6905t + 1.08571$$

Key Point / Exam Tip

Two failure modes of naive Gauss elimination.

1. **Division by zero.** E.g. $a_{11} = 0$ from the start — nothing to pivot on, even though the system is perfectly solvable. Sneakier version: a pivot that is nonzero initially can become *exactly zero* after an earlier elimination step (e.g. $a'_{22} = 0$ appears only after eliminating column 1).
2. **Round-off error.** Dividing by a very small pivot amplifies tiny rounding differences catastrophically. Worked case: system with exact answer $(1, 1, 1)$; a pivot of 0.001 appears at step 2, giving multiplier -2750 . Rounding the next entry to **6** vs. **5** significant digits gives final

answers (0.9625, 1.05, 0.999995) (max error $\approx 5\%$) vs. (0.625, 1.5, 0.99995) (max error $\approx 50\%$)
 — *one fewer digit of precision, an entirely wrong answer.*

3.2 Partial Pivoting

Definition

At the start of each elimination step k , scan column k for the largest $|a_{ik}|$ ($i \geq k$) and swap that row into the pivot position before eliminating. Fixes *both* failure modes at once: no zero pivots, and much smaller multipliers $\Rightarrow$ far less round-off.

Worked Example

Same round-off example, with pivoting. At step 2, candidate pivots are $|0.001|$ vs. $|-2.75|$ — swap rows: pivot becomes -2.75 , multiplier $m_{32} = 0.001/-2.75 \approx -0.00036$ (*seven million times smaller* than the naive -2750). Result with only 5 significant digits: $x_1 = x_2 = x_3 = 1$ — **exact**. “Same five digits, same system. Only difference: we swapped one row.”

3.3 Determinant via Elimination

Three theorems make $\det(A)$ almost free after elimination:

1. Adding/subtracting a multiple of one row to another leaves $\det$ unchanged.
2. Triangular/diagonal $[A]$: $\det(A) = \prod_i a_{ii}$.
3. Swapping two rows flips the sign of $\det$.

Worked Example

Worked example. Rocket matrix $\rightarrow$ triangular form $\begin{bmatrix} 25 & 5 & 1 \\ 0 & -4.8 & -1.56 \\ 0 & 0 & 0.7 \end{bmatrix}$: $\det(A) = 25 \times (-4.8) \times 0.7 = -84.00$ (Theorems 1, 2). With one row swap during pivoting elsewhere: $\det(B) = 10 \times 2.5 \times 6.002 = 150.05 \Rightarrow \det(A) = -150.05$ (Theorem 3).

3.4 Gauss–Jordan Elimination

Definition

One step further than Gauss elimination: (i) eliminate the pivot variable from *every* other row (above and below, not just below); (ii) normalize each pivot row so every pivot becomes 1. Result: the coefficient matrix becomes the identity — the RHS *is* the solution, no back substitution needed. Trade-off: strictly more arithmetic than plain Gauss elimination.

Worked Example

Worked example. $3x_1 - 0.1x_2 - 0.2x_3 = 7.85$, $0.1x_1 + 7x_2 - 0.3x_3 = -19.3$, $0.3x_1 - 0.2x_2 + 10x_3 = 71.4$. After normalizing/eliminating pivot columns 1, 2, 3 in turn:

$$\begin{bmatrix} 1 & 0 & 0 & 3.0000 \\ 0 & 1 & 0 & -2.5000 \\ 0 & 0 & 1 & 7.0000 \end{bmatrix}$$

Answer read directly off the RHS: $x_1 = 3$, $x_2 = -2.5$, $x_3 = 7$.

3.5 LU Decomposition

Definition

$A = LU$: U is exactly the upper-triangular result of naive Gauss forward elimination; L is unit-lower-triangular, built “for free” by storing each multiplier m_{ij} at position (i, j) instead of discarding it. Solving $AX = C$ becomes $LZ = C$ (forward substitution, no division — L has 1’s on the diagonal) then $UX = Z$ (back substitution, one division per row).

Worked Example

Worked example (rocket matrix again). Step 1 ($m_{21}=2.56, m_{31}=5.76$), step 2 ($m_{32}=3.5$):

$$U = \begin{bmatrix} 25 & 5 & 1 \\ 0 & -4.8 & -1.56 \\ 0 & 0 & 0.7 \end{bmatrix}, \quad L = \begin{bmatrix} 1 & 0 & 0 \\ 2.56 & 1 & 0 \\ 5.76 & 3.5 & 1 \end{bmatrix}$$

Forward sub. with $C = [106.8, 177.2, 279.2]^\top$: $z_1 = 106.8$, $z_2 = -96.208$, $z_3 = 0.76$ (identical to Gauss elimination’s RHS column — not a coincidence: $Z = L^{-1}C$). Back sub. on $UX = Z$ recovers the same a_1, a_2, a_3 as before.

Finding A^{-1} via LU. Solve $A\{b_j\} = \{e_j\}$ for each unit column e_j (same L, U , only RHS changes):

$$A^{-1} = \begin{bmatrix} 0.04762 & -0.08333 & 0.03571 \\ -0.9524 & 1.417 & -0.4643 \\ 4.571 & -5.000 & 1.429 \end{bmatrix}$$

Key Point / Exam Tip

Cost comparison (clock-cycle model: $+, -, \times = 4T, \div = 16T$). For a *single* solve, $C_T|_{LU} = C_T|_{GE}$ exactly — LU buys nothing over Gauss elimination. But for computing A^{-1} (equivalently, n different right-hand sides): LU decomposes once ($\sim n^3$) and reuses it for n cheap ($\sim n^2$) substitutions — total $\sim n^3$; Gauss must redo the full $\sim n^3$ elimination for every RHS — total $\sim n^4$. Ratio $\approx n/4$ (e.g. $2501\times$ at $n = 10,000$). **“Decompose once, reuse often.”**

Q1: What are the two big steps of Gauss elimination?

A: Forward elimination (reduce to upper triangular) followed by back substitution (solve from the last equation upward).

Q2: Why can naive Gauss elimination fail even if no original coefficient is zero?

A: A pivot can become exactly zero *after* earlier elimination steps modify the matrix, causing division by zero later even though the original matrix had no zero pivot candidates.

Q3: What is partial pivoting, and what two problems does it fix simultaneously?

A: Swap in the row with the largest $|\text{coefficient}|$ in the current column before eliminating; this avoids zero pivots and greatly reduces round-off error (much smaller multipliers).

Q4: How do you get $\det(A)$ “for free” from Gauss elimination?

A: Row addition/subtraction doesn’t change $\det$; once triangular, $\det(A) = \prod a_{ii}$; each row swap during pivoting flips the sign.

Q5: What two upgrades does Gauss–Jordan add over Gauss elimination, and at what cost?

A: Eliminates the pivot variable from every row (not just below) and normalizes every pivot to 1, so the RHS becomes the solution directly (no back substitution) — at the cost of strictly more arithmetic.

Q6: What is the LU decomposition of A , and how is it obtained?

A: $A = LU$ where U is the upper-triangular result of naive Gauss forward elimination and L is unit-lower-triangular holding the multipliers m_{ij} at position (i, j) — obtained for free during

elimination.

Q7: Is LU decomposition cheaper than Gauss elimination for a single solve?

A: No — $C_{T|LU} = C_{T|GE}$ exactly for one right-hand side.

Q8: When does LU actually pay off?

A: With many right-hand sides sharing the same A (e.g. computing A^{-1}): decompose once ($\sim n^3$) and reuse cheap ($\sim n^2$) substitutions each time, vs. Gauss's $\sim n^4$ total — ratio $\approx n/4$ for large n .

Q9: Why is division especially expensive in the cost model, and where does this matter?

A: Division costs 16 cycles vs. 4 for $+$, $-$, $\times$ in the model used; this is why forward substitution (unit-diagonal L , no division) is cheaper than back substitution (one division per row).

4 Eigenvalue Decomposition (Power Method)

Definition

A matrix is a machine that eats a vector and returns another vector (a linear transformation). An **eigenvector** $\mathbf{v}$ of A is a nonzero vector whose *direction* is unchanged by A — only stretched/shrunk/flipped: $A\mathbf{v} = \lambda\mathbf{v}$. $\lambda > 1$: stretches; $0 < \lambda < 1$: shrinks; $\lambda < 0$: flips; $\lambda = 0$: crushes to zero.

Characteristic equation. $A\mathbf{v} = \lambda\mathbf{v} \Rightarrow (A - \lambda I)\mathbf{v} = 0$; for a nonzero $\mathbf{v}$, $\det(A - \lambda I) = 0$.

Worked Example

Worked example. $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$: $\det \begin{bmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{bmatrix} = (2-\lambda)^2 - 1 = 0 \Rightarrow \lambda^2 - 4\lambda + 3 = 0 \Rightarrow (\lambda-3)(\lambda-1) = 0$. So $\lambda_1 = 3, \lambda_2 = 1$. Eigenvectors: $\lambda_1 = 3 \Rightarrow \mathbf{v}_1 = [1, 1]^\top$; $\lambda_2 = 1 \Rightarrow \mathbf{v}_2 = [1, -1]^\top$. Check: $A\mathbf{v}_1 = [3, 3]^\top = 3\mathbf{v}_1 \checkmark$; $A\mathbf{v}_2 = [1, -1]^\top = 1\mathbf{v}_2 \checkmark$.

4.1 Eigen Decomposition $A = V\Lambda V^{-1}$

Worked Example

Derivation. Writing $A\mathbf{v}_i = \lambda_i\mathbf{v}_i$ for every eigenvector as one matrix equation: $AV = V\Lambda$ where $V = [\mathbf{v}_1 \ \mathbf{v}_2 \ \cdots]$, $\Lambda = \text{diag}(\lambda_i)$. Right-multiplying by V^{-1} :

$$A = V\Lambda V^{-1}. \quad \blacksquare$$

Λ is diagonal because in eigen-coordinates each direction is scaled independently, with no mixing — exactly what a diagonal matrix does. Reading $Ax = V(\Lambda(V^{-1}x))$ right-to-left: $V^{-1}x$ rewrites x in eigenvector language; $\Lambda(\cdot)$ scales each eigen-direction; $V(\cdot)$ translates back to standard coordinates.

Why this matters for powers: $A^k = V\Lambda^k V^{-1}$ — the middle $V^{-1}V$ pairs telescope away, so k matrix multiplications collapse into raising each diagonal entry of Λ to the k -th power.

Worked Example

Numeric walk-through. $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$, $\mathbf{x} = [3, 1]^\top = 2\mathbf{v}_1 + \mathbf{v}_2$, $V = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$, $V^{-1} = \begin{bmatrix} 1/2 & 1/2 \\ 1/2 & -1/2 \end{bmatrix}$, $\Lambda = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$.

$$V^{-1}\mathbf{x} = [2, 1]^\top \xrightarrow{\Lambda} [6, 1]^\top \xrightarrow{V} 6[1, 1]^\top + 1[1, -1]^\top = [7, 5]^\top$$

Direct check: $A[3, 1]^\top = [7, 5]^\top \checkmark$. Also $\Lambda^{100} = \text{diag}(3^{100}, 1)$ — trivial via the diagonal, versus 100

explicit matrix multiplications.

4.2 The Power Method

Definition

Any vector is a mix of eigenvectors: $\mathbf{x} = c_1\mathbf{v}_1 + \cdots + c_n\mathbf{v}_n$. Repeated multiplication by A makes the term with the largest-magnitude eigenvalue grow fastest and eventually dominate — this finds the dominant eigenvalue *without expanding determinants*.

Worked Example

Derivation.

$$A^k\mathbf{x} = c_1\lambda_1^k\mathbf{v}_1 + \cdots + c_n\lambda_n^k\mathbf{v}_n = \lambda_1^k \left[c_1\mathbf{v}_1 + \sum_{i \geq 2} c_i \left(\frac{\lambda_i}{\lambda_1} \right)^k \mathbf{v}_i \right]$$

If $|\lambda_1| > |\lambda_i|$ for all $i \geq 2$, then $(\lambda_i/\lambda_1)^k \rightarrow 0$ as $k \rightarrow \infty$, so $A^k\mathbf{x} \rightarrow \lambda_1^k c_1 \mathbf{v}_1$. ■

Algorithm. (1) Start with any nonzero $\mathbf{x}_0$. (2) $\mathbf{y}_{k+1} = A\mathbf{x}_k$. (3) Normalize: divide by the largest-magnitude entry to get $\mathbf{x}_{k+1}$; that entry is the eigenvalue estimate (once $\mathbf{x}_k \approx \mathbf{v}_1$ with largest entry 1, $A\mathbf{x}_k \approx \lambda_1\mathbf{x}_k$, so its largest entry $\approx \lambda_1$). (4) Repeat until convergence.

Requires a strict dominance gap $|\lambda_1| > |\lambda_2| \geq \cdots$; convergence rate is governed by $|\lambda_2/\lambda_1|$ (closer to 1 $\Rightarrow$ slower).

Worked Example

Worked example (2×2). $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ ($\lambda_1 = 3, \mathbf{v}_1 = [1, 1]^\top$), $\mathbf{x}_0 = [1, 0]^\top$.

Iter	$\mathbf{x}_k$	λ_k ($ \epsilon_a $ %)
1	$[1, 0.500]^\top$	2.000 (–)
2	$[1, 0.800]^\top$	2.500 (20.0)
3	$[1, 0.929]^\top$	2.800 (10.7)
4	$[1, 0.976]^\top$	2.929 (4.40)
∞	$[1, 1]^\top$	3.000 (exact)

The $\mathbf{v}_2$ -component shrinks each iteration by $\lambda_2/\lambda_1 = 1/3$.

Key Point / Exam Tip

Sign-flip pitfall. In a 3×3 example, the normalizing factor changes sign between iterations 2–4 (e.g. $\lambda_3 \approx -7.112$), inflating the reported $|\epsilon_a|$ to 150%+ even though the sequence is converging normally toward the true $\lambda_{\max} \approx 6.070$ — a sign flip just distorts the simple relative-error formula; it does *not* mean the method is failing.

4.3 Inverse Power Method (Smallest Eigenvalue)

Worked Example

Derivation. $A\mathbf{v} = \lambda\mathbf{v} \Rightarrow \mathbf{v} = \lambda A^{-1}\mathbf{v} \Rightarrow A^{-1}\mathbf{v} = \frac{1}{\lambda}\mathbf{v}$ — so $1/\lambda$ is an eigenvalue of A^{-1} . This *flips the ranking*: the smallest $|\lambda|$ of A becomes the largest (dominant) eigenvalue of A^{-1} . Run the ordinary power method on A^{-1} (practically: solve $A\mathbf{y}_{k+1} = \mathbf{x}_k$ via one LU factorization, reused every iteration) and take the reciprocal of the normalizing factor for $\lambda_{\min}$.

Worked Example

Worked example. $A = \begin{bmatrix} 8 & 2 & 0 & 0 \\ 2 & 8 & 0 & 0 \\ 0 & 0 & 3 & 1 \\ 0 & 0 & 1 & 3 \end{bmatrix}$, eigenvalues 10, 6, 4, 2; $\mathbf{x}_0 = [1, 0, 1, 0]^\top$. Iterating $A^{-1}\mathbf{x}_k$

and normalizing:

Iter	Normalizing factor	$\lambda_{\min} \approx 1/\text{factor}$
1	0.37500	2.66667
2	0.41667	2.40000
4	0.47222	2.11765
6	0.49242	2.03077

Converges to $\lambda_{\min} = 2$ (the dominant eigenvalue of A^{-1} is $1/2 = 0.5$).

4.4 Deflation (Intermediate Eigenvalues)**Definition**

The power method always finds the “loudest” eigenvalue. To find λ_2 , mute λ_1 ’s contribution: form $A_2 = A - \lambda_1 \hat{\mathbf{v}}_1 \hat{\mathbf{v}}_1^\top$ (Hotelling’s deflation), where $\hat{\mathbf{v}}_1 = \mathbf{v}_1 / \sqrt{\mathbf{v}_1^\top \mathbf{v}_1}$; then run the power method on A_2 .

Worked Example

Worked example. $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$, $\lambda_1 = 3$, $\hat{\mathbf{v}}_1 = \frac{1}{\sqrt{2}}[1, 1]^\top$.

$$\lambda_1 \hat{\mathbf{v}}_1 \hat{\mathbf{v}}_1^\top = 3 \times \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 1.5 & 1.5 \\ 1.5 & 1.5 \end{bmatrix} \Rightarrow A_2 = \begin{bmatrix} 0.5 & -0.5 \\ -0.5 & 0.5 \end{bmatrix}$$

Check: $A_2 \mathbf{v}_1 = [0, 0]^\top$ (λ_1 muted to 0); $A_2 \mathbf{v}_2 = [1, -1]^\top = 1 \times \mathbf{v}_2$ (unchanged). A_2 has eigenvalues $\{0, 1\}$ — power method on A_2 finds $\lambda_2 = 1$. Orthogonality $\hat{\mathbf{v}}_1^\top \hat{\mathbf{v}}_2 = 0$ is exactly why $\mathbf{v}_2$ is left untouched (requires A symmetric).

Q1: What is an eigenvector and eigenvalue, in one sentence?

A: An eigenvector’s direction is unchanged by A ; A only scales it by the eigenvalue λ : $A\mathbf{v} = \lambda\mathbf{v}$.

Q2: Derive $A = V\Lambda V^{-1}$.

A: Stack $A\mathbf{v}_i = \lambda_i \mathbf{v}_i$ for every eigenvector into $AV = V\Lambda$; right-multiply by V^{-1} to get $A = V\Lambda V^{-1}$.

Q3: Why is eigen decomposition useful for computing A^k ?

A: $A^k = V\Lambda^k V^{-1}$ — raising a diagonal matrix to a power is trivial (per-entry), avoiding k explicit matrix multiplications.

Q4: What is the key insight behind the power method?

A: Any vector decomposes as $\mathbf{x} = \sum c_i \mathbf{v}_i$; under repeated multiplication by A , the dominant-eigenvalue term grows fastest and eventually swamps the rest, so $A^k \mathbf{x}$ aligns with $\mathbf{v}_1$.

Q5: Why does the normalizing factor approximate λ_1 ?

A: Once $\mathbf{x}_k \approx \mathbf{v}_1$ (normalized to largest entry 1), $A\mathbf{x}_k \approx \lambda_1 \mathbf{x}_k$, so the largest entry of $A\mathbf{x}_k$ is $\approx \lambda_1$.

Q6: What condition is required for the power method to converge?

A: A strict dominance gap $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|$; convergence rate depends on $|\lambda_2/\lambda_1|$.

Q7: How do you find the smallest eigenvalue using the power method?

A: Apply the power method to A^{-1} (inverse power method) — eigenvalues of A^{-1} are $1/\lambda$, flipping the ranking so $\lambda_{\min}$ of A becomes dominant in A^{-1} ; solve $A\mathbf{y} = \mathbf{x}$ via LU rather than forming A^{-1} explicitly.

Q8: How do you find an intermediate eigenvalue λ_2 ?

A: Deflation: form $A_2 = A - \lambda_1 \hat{\mathbf{v}}_1 \hat{\mathbf{v}}_1^\top$ to mute λ_1 to zero while leaving other eigenpairs unchanged, then run the power method on A_2 .

Q9: Why does deflation leave $\mathbf{v}_2$ untouched?

A: Because $\hat{\mathbf{v}}_1 \perp \hat{\mathbf{v}}_2$ (true for symmetric matrices), so the projection of $\mathbf{v}_2$ onto $\hat{\mathbf{v}}_1$ is zero — nothing is subtracted from the $\mathbf{v}_2$ direction.

Q10: What is a limitation of deflation?

A: Each stage uses a numerically approximate eigenvector, so errors compound through successive deflated matrices — reliable for the first few eigenvalues, degrading for later ones.

Q11: Why can the reported percentage error spike during power-method iterations even while it converges normally?

A: If the normalizing factor changes sign between iterations, the relative-error formula $|\varepsilon_a| = |(\lambda_k - \lambda_{k-1})/\lambda_k| \times 100\%$ spikes artificially — it does not mean the method is diverging.

5 Optimization

5.1 Golden Section Search

Definition

Golden Section Search (GSS) is an optimization technique: an efficient way to locate the extremum (maximum or minimum) of a single-variable, *unimodal* function, using the Golden Ratio to place search points so function evaluations are reused across iterations.

Setup and assumption. Choose x_L, x_u bracketing a single extremum. **Key assumption:** exactly one extremum lies in $[x_L, x_u]$ (unimodal on the interval). The method *fails* if there are multiple extrema, because the “keep the better side” logic can discard the region containing the true optimum.

Algorithm (maximization).

1. Evaluate endpoints $f_L = f(x_L)$, $f_u = f(x_u)$.
2. Golden distance $d = R(x_u - x_L)$ with $R = \frac{\sqrt{5}-1}{2} \approx 0.618$.
3. Interior points: $x_1 = x_u - d$, $f_1 = f(x_1)$; $x_2 = x_L + d$, $f_2 = f(x_2)$.
4. Locate region: $f_1 > f_2 \Rightarrow \text{max is left } (x_u \leftarrow x_2, \text{ reuse } x_2 \rightarrow x_1, f_2 \rightarrow f_1, \text{ recompute new } x_2)$;
 $f_1 < f_2 \Rightarrow \text{max is right } (x_L \leftarrow x_1, \text{ reuse } x_1 \rightarrow x_2, f_1 \rightarrow f_2, \text{ recompute new } x_1)$.
5. Repeat until $\left| \frac{2(x_u - x_L)}{x_u + x_L} \right| < \varepsilon_s$; report $x_{opt} = \frac{x_u + x_L}{2}$, $f_{opt} = f(x_{opt})$.

Efficiency: first iteration costs 2 interior evaluations (4 with endpoints); every later iteration needs only *one* new function call, because golden placement always lands a new point exactly on a previously evaluated one.

Worked Example

Derivation of the Golden Ratio. Let $\ell_0 = x_u - x_L$, $\ell_1 = x_2 - x_L$, $\ell_2 = x_u - x_2$ be interval lengths in one iteration. To make next-iteration points reuse a previous point:

$$(i) \ell_0 = \ell_1 + \ell_2, \quad (ii) \frac{\ell_1}{\ell_0} = \frac{\ell_2}{\ell_1}.$$

Substitute (i) into (ii):

$$\frac{\ell_1}{\ell_1 + \ell_2} = \frac{\ell_2}{\ell_1} \implies \ell_1^2 = \ell_2(\ell_1 + \ell_2) \implies 1 + \frac{\ell_2}{\ell_1} = \frac{\ell_1}{\ell_2}.$$

Let $R = \ell_2/\ell_1$. Then $1 + R = 1/R \Rightarrow R^2 + R - 1 = 0$, so

$$R = \frac{-1 \pm \sqrt{5}}{2}.$$

Taking the positive root: $R = \frac{\sqrt{5} - 1}{2} \approx 0.618$ — the Golden Ratio. ■

Worked Example

Worked simulation. Maximize $f(x) = 2 \sin x - \frac{x^2}{10}$ on $[0, 4]$, $R = 0.61803$.

Iteration 1: $x_L = 0, x_u = 4, d = R(4) = 2.47214$. $x_1 = 1.52786, x_2 = 2.47214$.

$$f(x_1) = 2 \sin(1.52786) - \frac{1.52786^2}{10} = 1.99924 - 0.23344 = 1.76580$$

$$f(x_2) = 2 \sin(2.47214) - \frac{2.47214^2}{10} = 1.25701 - 0.61115 = 0.64586$$

Since $f_1 > f_2$: max is left $\Rightarrow x_u = x_2 = 2.47214$ (reuse $x_1 \rightarrow x_2, f_2 = 1.76580$).

Iteration 2: $d = R(2.47214 - 0) = 1.52786$, new $x_1 = 0.94427$:

$$f(x_1) = 2 \sin(0.94427) - \frac{0.94427^2}{10} = 1.61804 - 0.08916 = 1.52888$$

$f_1 = 1.52888 < f_2 = 1.76580 \Rightarrow$ max is right $\Rightarrow x_L = x_1 = 0.94427$.

Convergence check: $\left| \frac{2(2.47214 - 0.94427)}{2.47214 + 0.94427} \right| = 0.8947$ (still large; continue). Iterating further converges to $x^* \approx 1.4276, f(x^*) \approx 1.7757$.

Q1: What assumption must hold for GSS to work, and when does it fail?

A: Unimodality — exactly one extremum in $[x_L, x_u]$. It fails with multiple extrema in the bracket, since the discard rule can drop the region holding the true optimum.

Q2: Why is the method called “golden”, i.e. what is the efficiency advantage?

A: Interior points are placed at ratio $R \approx 0.618$ so one new point of the next iteration coincides with an already-evaluated point — only 1 new function evaluation per iteration after the first, minimizing (possibly expensive) function calls.

Q3: How do you decide which side contains the maximum, and which old point is reused?

A: Compare f_1, f_2 : $f_1 > f_2 \Rightarrow$ max left, $x_u = x_2$; $f_1 < f_2 \Rightarrow$ max right, $x_L = x_1$. The surviving interior point is reused (with its stored f value) as one of the new interior points.

Q4: How is the final optimum reported and when do you stop?

A: $x_{opt} = \frac{x_u + x_L}{2}, f_{opt} = f(x_{opt})$; stop when $\left| \frac{2(x_u - x_L)}{x_u + x_L} \right| < \varepsilon_s$.

5.2 Gradient Descent (GD) and Linear Regression

Definition

Gradient Descent is a general iterative optimization tool: it can tune the slope/intercept of a regression line, the parameters of a logistic “squiggle”, or a cluster arrangement — any differentiable loss surface.

Why GD instead of setting the derivative to zero? Analytically solving slope = 0 is fine for small problems, but often infeasible in closed form with many parameters or huge data. GD instead

takes steps from an initial guess — cheap coarse steps far from the optimum, finer steps near it.

Single-variable update:

$$\text{step} = \frac{d(SSR)}{d(b)} \times \alpha, \quad b_{\text{new}} = b_{\text{old}} - \text{step}.$$

Schedule: the rule for shrinking the learning rate from large (early) to small (later). **Stopping:** step size ≈ 0 , or a maximum number of steps reached.

Multiple parameters (slope m and intercept b together):

1. Differentiate the loss w.r.t. each parameter.
2. Initialize each parameter (e.g. $b=0$, $m=1$).
3. Plug current values into the derivatives.
4. $\text{step} = \text{derivative} \times \alpha$ for each parameter.
5. $\text{new} = \text{old} - \text{step}$, for each parameter.
6. Repeat until convergence.

Stochastic / mini-batch GD. Problem: with millions of parameters/samples, a full-data gradient every step is too expensive (data is often redundant). Fix: Stochastic GD (SGD) uses one random sample per step (strict definition) or a small random mini-batch (common practice) — cheaper per step, robust to redundancy, and lets you keep updating as new data streams in without restarting.

Worked Example

Derivation of the SSR gradients. Residual $r_i = y_i - (b + mx_i)$, $SSR = \sum_{i=1}^n (y_i - b - mx_i)^2$.

$$\frac{\partial SSR}{\partial b} = \sum_{i=1}^n 2(y_i - b - mx_i)(-1) = -2 \sum_{i=1}^n (y_i - b - mx_i)$$

$$\frac{\partial SSR}{\partial m} = \sum_{i=1}^n 2(y_i - b - mx_i)(-x_i) = -2 \sum_{i=1}^n x_i(y_i - b - mx_i)$$

These two derivatives are exactly what GD plugs (b, m) into at every step.

Worked Example

Worked simulation — 1-variable GD. Data $(x, y) = (1, 2), (2, 3), (3, 5)$; fix slope $m = 1$, tune intercept b . Learning rate $\alpha = 0.1$. $SSR(b) = \sum (y_i - b - x_i)^2$, gradient $= -2 \sum (y_i - b - x_i)$.

$b_0 = 0$: residuals 1, 1, 2 ($y_i - x_i$), sum = 4; grad = $-2(4) = -8$; step = $0.1 \times (-8) = -0.8$; $b_1 = 0 - (-0.8) = 0.8$.

$b_1 = 0.8$: residuals $2 - 1.8 = 0.2$, $3 - 2.8 = 0.2$, $5 - 3.8 = 1.2$, sum = 1.6; grad = -3.2 ; step = -0.32 ; $b_2 = 0.8 + 0.32 = 1.12$.

$b_2 = 1.12$: residuals -0.12 , -0.12 , 0.88 , sum = 0.64; grad = -1.28 ; step = -0.128 ; $b_3 = 1.248$.

Steps ($0.8 \rightarrow 0.32 \rightarrow 0.128 \rightarrow \dots$) shrink each round. Analytic optimum: set gradient = 0 $\Rightarrow b^* = \frac{1}{3} \sum (y_i - x_i) = \frac{1}{3}(1 + 1 + 2) = 1.3333$. The iterates march toward 1.3333.

Q1: Why square the residuals in the loss function?

A: So positive and negative errors don't cancel; SSR vs. a parameter is then a convex bowl whose bottom is the optimum.

Q2: What is a “schedule” in GD?

A: The rule for shrinking the learning rate from relatively large early on to relatively small later.

Q3: What problem does plain (batch) GD face on very large datasets, and what is the fix?

A: Every step scans the whole dataset — too expensive with millions of samples/parameters (also redundant). Fix: Stochastic/mini-batch GD, using one sample or a small random subset per step.

Q4: State two advantages of SGD.

A: (i) Best of both worlds — cheaper steps, still statistically representative; (ii) more stable estimates in fewer effective passes; also lets you update online as new data arrives without restarting.

5.3 Newton's Method for Optimization

Definition

Root finding uses first-order information (f'); optimization uses second-order information (gradient *and* Hessian) to jump directly to the vertex of a local quadratic approximation of f .

$$\text{Zero-finding: } x_{t+1} = x_t - \frac{f(x_t)}{f'(x_t)} \quad \text{Optimization: } x_{t+1} = x_t - \frac{f'(x_t)}{f''(x_t)}$$

Vector form: $x_{t+1} = x_t - H^{-1}g$, with $g = \nabla f(x_t)$, $H = \nabla^2 f(x_t)$; damped variant $x_{t+1} = x_t - tH^{-1}g$.

Limitations.

1. Finds *any* stationary point (min, max, or saddle) — not guaranteed the global optimum; a saddle/max has zero slope but is not a minimum.
2. Can move toward a maximum if started in the wrong place, since the raw update does not check the sign of f'' . Remedy: mix in GD steps / check curvature direction.

Advantages over GD.

1. Faster (fewer iterations).
2. Not sensitive to a learning rate (none to tune).
3. Quadratic convergence — correct digits roughly double each step.

When to use which: GD for large problems (Hessian huge/expensive); Newton for small problems (Hessian cheap) to enjoy quadratic convergence.

Worked Example

Derivation via 2nd-order Taylor expansion. Near a , for smooth f :

$$f(x) \approx f(a) + g^\top(x - a) + \frac{1}{2}(x - a)^\top H(x - a), \quad g = \nabla f(a), \quad H = \nabla^2 f(a).$$

Expand the quadratic term (using $H = H^\top$, $x^\top a = a^\top x$):

$$(x - a)^\top H(x - a) = x^\top Hx - 2a^\top Hx + a^\top Ha.$$

Standard quadratic form: $q(x) = \frac{1}{2}x^\top Hx + b^\top x + c$, with $b = g - Ha$, $c = f(a) - g^\top a + \frac{1}{2}a^\top Ha$.

Minimize: $\nabla q = Hx + b = 0 \Rightarrow x = -H^{-1}b = -H^{-1}(g - Ha) = a - H^{-1}g$.

Hence the Newton step $x_{t+1} = x_t - H^{-1}g$. ■

Second-order (minimum) condition. $\nabla^2 q = H$; the stationary point is a minimum iff H is positive semi-definite (all eigenvalues real, ≥ 0); positive definite if all > 0 . Convex $f \Rightarrow H$ PSD.

Worked Example

Worked simulation. $f(x) = x^2 - 4 \cos x$ (radians), $x_0 = 1$.

$$f'(x) = 2x + 4 \sin x, \quad f''(x) = 2 + 4 \cos x.$$

$$x_0 = 1: f' = 2 + 4 \sin 1 = 2 + 3.3659 = 5.3659; f'' = 2 + 4 \cos 1 = 2 + 2.1612 = 4.1612.$$

$$x_1 = 1 - \frac{5.3659}{4.1612} = 1 - 1.2895 = -0.2895.$$

$$x_1 = -0.2895: f' = 2(-0.2895) + 4 \sin(-0.2895) = -0.5790 - 1.1420 = -1.7210; f'' = 2 +$$

$$4 \cos(-0.2895) = 2 + 3.8336 = 5.8336.$$

$$x_2 = -0.2895 - \frac{-1.7210}{5.8336} = -0.2895 + 0.2950 = 0.00554.$$

$$x_2 = 0.00554: f' \approx 2(0.00554) + 4 \sin(0.00554) \approx 0.01108 + 0.02216 = 0.03324; f'' \approx 2 + 4 \cos(0.00554) \approx 5.99994.$$

$$x_3 = 0.00554 - \frac{0.03324}{5.99994} = 0.00554 - 0.00554 = 0.00000.$$

True minimum is at $x = 0$. Correct digits roughly double each step (x_1 : 0 dp, x_2 : 2 dp, x_3 : 5 dp) — quadratic convergence. $f'' > 0$ throughout, so we converge to a minimum.

Q1: How does the order of information differ between root finding and optimization?

A: Root finding uses first-order information (f and f'); optimization uses second-order information (gradient and Hessian).

Q2: State the two limitations of Newton's method for optimization.

A: (i) Converges to *any* stationary point, not necessarily the global optimum; (ii) can converge to a maximum if started poorly, since the update ignores the sign of the curvature.

Q3: What smoothness condition does f need?

A: $f : \mathbb{R}^n \rightarrow \mathbb{R}$ must be sufficiently smooth (continuous derivatives exist); otherwise the method fails.

Q4: What is done in practice instead of explicitly inverting H ?

A: Solve the linear system $Hy = g$ for y , then update $x_{t+1} = x_t - y$ — cheaper and more numerically stable. If H fails to be positive definite, blend toward GD (Levenberg–Marquardt).

5.4 Foundations: Gradient and Stationary Points

Definition

The gradient returns a vector field giving the direction and magnitude of steepest ascent at every point. All optimization reduces to: (1) set up an objective function; (2) solve/search for locations of zero gradient.

Worked Example

1-D classification examples.

$$J = x^2 : J' = 2x = 0 \Rightarrow x = 0, J'' = 2 > 0 \Rightarrow \text{minimum.}$$

$$J = -x^2 : J' = -2x = 0 \Rightarrow x = 0, J'' = -2 < 0 \Rightarrow \text{maximum.}$$

$$J = x^3 : J' = 3x^2 = 0 \Rightarrow x = 0, J'' = 6x \text{ changes sign} \Rightarrow \text{saddle point.}$$

2-D example. $J(\mathbf{x}) = x_1^2 + x_2^2 \Rightarrow \nabla J = (2x_1, 2x_2)^\top = 0 \Rightarrow (x_1, x_2) = (0, 0)$ (minimum).

Q1: For $J = x^3$, the derivative is zero at $x = 0$ — is that a minimum?

A: No. $J' = 3x^2 = 0$ at $x = 0$, but $J'' = 6x$ changes sign across $x = 0$, so it is a saddle/inflection point — a zero derivative alone never guarantees an extremum; you must check the second derivative (or Hessian) too.

5.5 Constrained Optimization (Lagrange Multipliers)

Definition

Constrained optimization finds the min/max of $J(\mathbf{x})$ subject to $C(\mathbf{x}) = 0$ — harder than the unconstrained case because the search is confined to the constraint surface.

Geometric idea. On a contour plot, the constrained optimum is where the constraint curve is tangent to a level curve of J — equivalently, ∇J and ∇C are parallel.

Worked Example

Why the gradients are parallel (derivation). At a constrained optimum, J cannot decrease by moving along the constraint, so ∇J has no component along the constraint curve — i.e. ∇J is parallel to ∇C :

$$\nabla J(\mathbf{x}) = k \nabla C(\mathbf{x}) \implies \nabla J(\mathbf{x}) - k \nabla C(\mathbf{x}) = 0 \implies \nabla J(\mathbf{x}) + \lambda \nabla C(\mathbf{x}) = 0, \quad \lambda = -k.$$

This is exactly the stationarity of $\Lambda = J(\mathbf{x}) + \lambda C(\mathbf{x})$. ■

Bordered Hessian. Used to classify the stationary point (min/max/saddle):

$$H(\Lambda) = \begin{pmatrix} 0 & \partial C / \partial x_1 & \cdots & \partial C / \partial x_n \\ \partial C / \partial x_1 & \partial^2 \Lambda / \partial x_1^2 & \cdots & \partial^2 \Lambda / \partial x_1 \partial x_n \\ \vdots & \vdots & \ddots & \vdots \\ \partial C / \partial x_n & \partial^2 \Lambda / \partial x_1 \partial x_n & \cdots & \partial^2 \Lambda / \partial x_n^2 \end{pmatrix}$$

The $(1,1)$ entry is 0 because $\partial \Lambda / \partial \lambda = C(\mathbf{x}) = 0$; the border is the constraint gradient.

Worked Example

Worked example (full solve). Minimize $J = x_1^2 + x_2^2$ subject to $C = x_1 + x_2 + 3 = 0$.

$$\Lambda = x_1^2 + x_2^2 + \lambda(x_1 + x_2 + 3).$$

Stationarity: $\partial \Lambda / \partial x_1 = 2x_1 + \lambda = 0$, $\partial \Lambda / \partial x_2 = 2x_2 + \lambda = 0$, $\partial \Lambda / \partial \lambda = x_1 + x_2 + 3 = 0$.

From the first two: $x_1 = x_2 = -\lambda/2$. Substitute into the third: $-\lambda/2 - \lambda/2 + 3 = 0 \Rightarrow \lambda = 3$.

$$x_1 = -1.5, \quad x_2 = -1.5, \quad \lambda = 3.$$

Check: $\nabla J = (2x_1, 2x_2) = (-3, -3)$ and $\nabla C = (1, 1)$; indeed $\nabla J = -3\nabla C$, parallel. ■

Q1: Why do we need the bordered Hessian if we already have the parallel-gradient condition?

A: The parallel-gradient (stationarity) condition only *locates* a stationary point; it cannot tell whether it is a min, max, or saddle. The bordered Hessian (second-derivative information) is needed to classify it.

Q2: If the constraint is given as $C(\mathbf{x}) = g$ rather than $= 0$, what do you do first?

A: Rewrite it as $C(\mathbf{x}) - g = 0$, then form the Lagrangian and proceed as usual.

Q3: State three advantages of the Lagrange-multiplier method.

A: (i) Converts a constrained problem into an unconstrained one; (ii) λ carries useful scale information; (iii) it shows how the objective changes as the constraint changes ($\lambda = \text{sensitivity / shadow price}$).

6 Regression

6.1 Least-Squares Linear Regression

Definition

Fit $y = mx + b$ so that all data points lie as close as possible to the line (small residuals). Error = real – predicted = $y_i - (mx_i + b)$. Squaring the errors prevents $+/-$ cancellation:

$$SSE = \sum_{i=1}^n (y_i - mx_i - b)^2.$$

Worked Example

Proof of the least-squares formulas.

Intercept. $\frac{\partial SSE}{\partial b} = -2 \sum (y_i - mx_i - b) = 0 \Rightarrow \sum y_i - m \sum x_i - nb = 0$, so

$$b = \frac{\sum y - m \sum x}{n}. \quad (\text{A})$$

Slope. $\frac{\partial SSE}{\partial m} = -2 \sum x_i (y_i - mx_i - b) = 0 \Rightarrow \sum x_i y_i - m \sum x_i^2 - b \sum x_i = 0$. Substitute b from (A):

$$\sum xy - m \sum x^2 - \frac{\sum y - m \sum x}{n} \sum x = 0.$$

Multiply by n and collect terms in m :

$$n \sum xy - mn \sum x^2 - \sum y \sum x + m \left(\sum x \right)^2 = 0$$

$$n \sum xy - \sum x \sum y = m \left[n \sum x^2 - \left(\sum x \right)^2 \right]$$

$$m = \frac{n \sum xy - \sum x \sum y}{n \sum x^2 - (\sum x)^2} \quad \blacksquare$$

Worked Example

Worked simulation — fit a line and find r . Data $(1, 2), (2, 4), (3, 5), (4, 4), (5, 5)$, $n = 5$.

	x	y	x^2	y^2	xy
	1	2	1	4	2
	2	4	4	16	8
	3	5	9	25	15
	4	4	16	16	16
	5	5	25	25	25
Σ	15	20	55	86	66

$$m = \frac{5(66) - 15(20)}{5(55) - 15^2} = \frac{330 - 300}{275 - 225} = \frac{30}{50} = 0.6$$

$$b = \frac{20 - 0.6(15)}{5} = \frac{20 - 9}{5} = \frac{11}{5} = 2.2 \quad \Rightarrow \quad \hat{y} = 0.6x + 2.2$$

$$r = \frac{5(66) - 15(20)}{\sqrt{[5(55) - 15^2][5(86) - 20^2]}} = \frac{30}{\sqrt{50 \times 30}} = \frac{30}{\sqrt{1500}} = \frac{30}{38.7298} = 0.7746$$

$r^2 = 0.60$ — the line explains 60% of the variance in y .

Q1: What does the correlation coefficient r measure, and how do you read its value?

A: The strength and direction of the linear relationship between x and y . $r = \pm 1$: perfect positive/negative line; $|r|$ near 0.6–0.8: fairly strong but scattered; $r = 0$: no linear relation. Closer to ± 1 = stronger.

Q2: What does r^2 mean?

A: The fraction of variance in the data explained by the model; $r^2 = 1$ means 100% explained, $r^2 = 0.6$ means 60%.

Q3: Does a strong correlation imply causation?

A: No. Two variables can move together by coincidence, or because a hidden third factor drives both.

Q4: Which running sums must be tabulated to compute m, b, r ?

A: $\sum x$, $\sum y$, $\sum x^2$, $\sum y^2$, $\sum xy$.

6.2 Multiple Linear Regression

Definition

Model with n features — a hyperplane in n dimensions:

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n,$$

where w_0 is the bias/intercept and $w_1, \dots, w_n$ are weights. (1D $\rightarrow$ point, 2D $\rightarrow$ line, 3D $\rightarrow$ plane, higher $\rightarrow$ hyperplane. Without w_0 the fit is forced through the origin.)

Worked Example

Derivation of the MSE gradient. With $\hat{y}_i = \sum_k w_k x_{ik}$ (and $x_{i0} = 1$) and cost

$$J(\mathbf{w}) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2,$$

$$\frac{\partial J}{\partial w_j} = \frac{1}{2m} \sum_{i=1}^m 2(\hat{y}_i - y_i) \frac{\partial \hat{y}_i}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i) x_{ij}.$$

Hence the concrete update:

$$w_j \leftarrow w_j - \frac{\alpha}{m} \sum_{i=1}^m (\hat{y}_i - y_i) x_{ij}. \quad \blacksquare$$

Q1: List the five assumptions of multiple linear regression.

A: Linearity; Independence (samples independent); Homoscedasticity (constant error variance); Normality (errors $\sim \mathcal{N}$); No multicollinearity (features not linearly dependent on each other).

Q2: Scenario: the errors follow a Poisson distribution. Can you still apply MLR?

A: No — Poisson errors are not normally distributed, so the Normality assumption is violated; MLR is not appropriate here.

Q3: What does a violated homoscedasticity assumption look like?

A: The error variance is not constant — the spread of residuals grows, or some points sit extremely far from the fit (fanning out), instead of scattering evenly.

Q4: What does “no multicollinearity” require, and why does it matter?

A: The input features must not be linearly dependent on one another (not redundant); violating it makes individual weight estimates unstable/uninterpretable.

Q5: Why does the cost function use $\frac{1}{2m}$ rather than just $\sum (\hat{y}_i - y_i)^2$?

A: Squaring stops $+/ -$ residuals from cancelling; the $\frac{1}{2m}$ factor removes scale effects (the 2

cancels the derivative's factor of 2, and dividing by m averages over samples) so GD updates stay well-scaled.

7 Interpolation

Key Point / Exam Tip

Regression finds a best fit / explanation and need not pass through every point (tolerates noise/outliers). Interpolation instead builds a curve that passes through *all* given points exactly. Degree rule: $(n+1)$ points $\Rightarrow$ *at most* an n -th order polynomial (“at most” because collinear points collapse to a lower-degree curve). Forcing too high an order through noisy data over-fits the noise.

7.1 Lagrange Interpolation

Worked Example

Proof. Assume an n -th degree polynomial written as

$$f(x) = A_0(x-x_1)(x-x_2)\cdots(x-x_n) + A_1(x-x_0)(x-x_2)\cdots(x-x_n) + \cdots + A_n(x-x_0)\cdots(x-x_{n-1}). \quad (1)$$

Put $x = x_0$: every term except the first has a factor $(x - x_0)$ and vanishes,

$$f(x_0) = A_0(x_0 - x_1)(x_0 - x_2)\cdots(x_0 - x_n),$$

and since $f(x_0) = y_0$,

$$A_0 = \frac{y_0}{(x_0 - x_1)(x_0 - x_2)\cdots(x_0 - x_n)}.$$

Putting $x = x_1, \dots, x_n$ similarly yields $A_1, \dots, A_n$. Substituting all A_k back into (1) gives

$$f(x) = \sum_{j=0}^n y_j \prod_{\substack{k=0 \\ k \neq j}}^n \frac{x - x_k}{x_j - x_k}. \quad \blacksquare$$

Worked Example

Worked simulation. Points $(1, 1), (3, 9), (4, 16)$ (sampled from $y = x^2$). Estimate $f(2)$.

$$L_0(2) = \frac{(2-3)(2-4)}{(1-3)(1-4)} = \frac{(-1)(-2)}{(-2)(-3)} = \frac{2}{6} = 0.3333$$

$$L_1(2) = \frac{(2-1)(2-4)}{(3-1)(3-4)} = \frac{(1)(-2)}{(2)(-1)} = \frac{-2}{-2} = 1$$

$$L_2(2) = \frac{(2-1)(2-3)}{(4-1)(4-3)} = \frac{(1)(-1)}{(3)(1)} = -0.3333$$

$$f(2) = 1(0.3333) + 9(1) + 16(-0.3333) = 0.3333 + 9 - 5.3333 = 4$$

Exact value $2^2 = 4$ — matches (three points exactly determine the underlying quadratic).

Q1: Given $(n+1)$ points, what is the interpolating polynomial's degree, and why “at most”?

A: Degree = $(\# \text{points} - 1)$, i.e. at most n -th order; “at most” because if the points happen to be collinear the interpolant reduces to a lower-degree curve (e.g. a straight line).

Q2: Why not just use a very high-order polynomial to fit data?

A: It over-fits — chases noise and outliers, producing a wildly oscillating curve. Interpolation is for passing exactly through known points, not for modelling noisy data.

7.2 Linear Interpolation

Worked Example

Derivation. Line through bracketing points $(x_2, y_2), (x_3, y_3)$: $y_2 = mx_2 + b$, $y_3 = mx_3 + b \Rightarrow m = \frac{y_3 - y_2}{x_3 - x_2}$.

For query point (x_m, y_m) on the same line, $y_m - y_2 = m(x_m - x_2) \Rightarrow m = \frac{y_m - y_2}{x_m - x_2}$.

Equating the two slope expressions:

$$\frac{y_3 - y_2}{x_3 - x_2} = \frac{y_m - y_2}{x_m - x_2}. \quad \blacksquare$$

Rule: always use the two tabulated points that bracket the query x .

Worked Example

Worked simulation. Table $(x_2, y_2) = (10, 100)$, $(x_3, y_3) = (20, 400)$. Find y at $x_m = 13$.

$$\frac{400 - 100}{20 - 10} = \frac{y_m - 100}{13 - 10} \Rightarrow 30 = \frac{y_m - 100}{3} \Rightarrow y_m - 100 = 90 \Rightarrow y_m = 190.$$

Q1: How do you choose which two points to use for a query x ?

A: Always use the two tabulated points that surround (bracket) the query — e.g. for x_m between x_2, x_3 , use $(x_2, y_2), (x_3, y_3)$.

7.3 Newton's Divided-Difference Interpolation

Worked Example

Derivation of the first two coefficients. $P_0(x) = a_0$; at $x = x_0$: $P_0(x_0) = a_0 = y_0$.

$P_1(x) = a_0 + a_1(x - x_0)$; at $x = x_1$: $y_1 = y_0 + a_1(x_1 - x_0) \Rightarrow a_1 = \frac{y_1 - y_0}{x_1 - x_0} = f[x_1, x_0]$.

$P_2(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1)$; at $x = x_2$:

$$y_2 = y_0 + f[x_1, x_0](x_2 - x_0) + a_2(x_2 - x_0)(x_2 - x_1).$$

Rearranging, $\frac{y_2 - y_0}{x_2 - x_0} = f[x_1, x_0] + a_2(x_2 - x_1)$, hence

$$a_2 = \frac{f[x_2, x_0] - f[x_1, x_0]}{x_2 - x_1} = \frac{f[x_2, x_1] - f[x_0, x_1]}{x_2 - x_0} = f[x_2, x_1, x_0]. \quad \blacksquare$$

Worked Example

Proof of the recurrence relation (sketch). Let $P_n(x)$ interpolate $x_0 \dots x_n$ and $L_n(x)$ interpolate $x_1 \dots x_{n+1}$ (both order n). Define

$$P_{n+1}(x) = \frac{(x - x_0)L_n(x) + (x_{n+1} - x)P_n(x)}{x_{n+1} - x_0}.$$

At $x = x_0$: first term vanishes, $P_{n+1}(x_0) = \frac{(x_{n+1} - x_0)P_n(x_0)}{x_{n+1} - x_0} = P_n(x_0) = y_0$.

At $x = x_{n+1}$: second term vanishes, $P_{n+1}(x_{n+1}) = L_n(x_{n+1}) = y_{n+1}$.

At an interior $x = x_k$: both $L_n(x_k) = P_n(x_k) = y_k$, so $P_{n+1}(x_k) = \frac{(x_k - x_0)y_k + (x_{n+1} - x_k)y_k}{x_{n+1} - x_0} = y_k$.

Thus P_{n+1} interpolates all $n + 2$ points. Writing both P_n, L_n in Newton form and equating the coefficient of the top power x^{n+1} on each side gives

$$f[x_0, \dots, x_{n+1}] = \frac{f[x_1, \dots, x_{n+1}] - f[x_0, \dots, x_n]}{x_{n+1} - x_0}. \quad \blacksquare$$

Worked Example

Worked simulation. Points $(1, 1), (2, 8), (3, 27), (4, 64)$ (from $y = x^3$), $\Delta x = 1$.

1st divided differences: $f[x_0, x_1] = \frac{8-1}{2-1} = 7$, $f[x_1, x_2] = \frac{27-8}{3-2} = 19$, $f[x_2, x_3] = \frac{64-27}{4-3} = 37$.

2nd: $f[x_0, x_1, x_2] = \frac{19-7}{3-1} = 6$, $f[x_1, x_2, x_3] = \frac{37-19}{4-2} = 9$.

3rd: $f[x_0, x_1, x_2, x_3] = \frac{9-6}{4-1} = 1$.

Top diagonal: $a_0 = 1$, $a_1 = 7$, $a_2 = 6$, $a_3 = 1$:

$$P_3(x) = 1 + 7(x-1) + 6(x-1)(x-2) + 1(x-1)(x-2)(x-3).$$

Check at $x = 2.5$: $1 + 7(1.5) + 6(1.5)(0.5) + 1(1.5)(0.5)(-0.5) = 1 + 10.5 + 4.5 - 0.375 = 15.625 = 2.5^3$.
✓

Adding a new point $(5, 125)$ only appends one bottom row and one new coefficient a_4 ; $a_0..a_3$ untouched.

Q1: Why prefer Newton's divided-difference form over Lagrange?

A: The recalculation-free property: adding a new point only appends one new term/row; the existing coefficients $a_0, a_1, \dots$ don't change. Lagrange forces you to rebuild every basis polynomial from scratch.

Q2: What happens when the nodes are equally spaced?

A: The divided differences simplify, and you can switch to the simpler forward/backward difference formulas.

7.4 Spline Interpolation

Definition

Global vs. local. Newton and Lagrange are *global*: all points feed one big polynomial, so $(n + 1)$ points force an n -th order polynomial. Splines are *local*: fit short, low-order pieces between consecutive points.

Runge's Phenomenon. As more data points are added, a single global polynomial's order rises, and high-order polynomials oscillate wildly — worst near the edges of equally-spaced nodes (classic example: interpolating $\frac{1}{1+x^2}$, the fit wiggles and can even go negative). Fix: use many small local pieces (splines) instead of one big curve.

Linear spline. Connect consecutive points with straight lines. Weakness: no curvature — sharp corners at joints (not smooth).

Worked Example

Quadratic spline — equation-count bookkeeping and worked example. Each piece

$P_i = a_i x^2 + b_i x + c_i$: 3 unknowns/piece $\Rightarrow 3n$ unknowns for n pieces.

$$3n = \underbrace{2n}_{\text{through points}} + \underbrace{(n-1)}_{\text{smoothness: } P'_i \text{ match}} + \underbrace{1}_{\text{assumption } P''_1(x_1)=0}$$

Worked example: points $(0,0), (1,1), (2,4)$, $n = 2$ splines: $P_1 = a_1 x^2 + b_1 x + c_1$ on $[0,1]$, $P_2 = a_2 x^2 + b_2 x + c_2$ on $[1,2]$ ($3n = 6$ unknowns).

(1) Through points: $P_1(0) = 0 \Rightarrow c_1 = 0$; $P_1(1) = 1 \Rightarrow a_1 + b_1 + c_1 = 1$; $P_2(1) = 1 \Rightarrow a_2 + b_2 + c_2 = 1$; $P_2(2) = 4 \Rightarrow 4a_2 + 2b_2 + c_2 = 4$.

(2) Smoothness at $x = 1$: $P'_1(1) = P'_2(1) \Rightarrow 2a_1 + b_1 = 2a_2 + b_2$.

(3) Assumption $P''_1(0) = 0 \Rightarrow 2a_1 = 0 \Rightarrow a_1 = 0$.

Solve: $a_1 = 0 \Rightarrow b_1 = 1$ (from $a_1 + b_1 = 1$), $c_1 = 0$, so $P_1(x) = x$.

Smoothness: $2(0) + 1 = 2a_2 + b_2 \Rightarrow 2a_2 + b_2 = 1$. Subtract $(a_2 + b_2 + c_2 = 1)$ from $(4a_2 + 2b_2 + c_2 = 4)$: $3a_2 + b_2 = 3$. With $2a_2 + b_2 = 1$: $a_2 = 2$, $b_2 = -3$, $c_2 = 1 - a_2 - b_2 = 1 - 2 + 3 = 2$.

$$P_1(x) = x \text{ on } [0,1], \quad P_2(x) = 2x^2 - 3x + 2 \text{ on } [1,2].$$

Check: $P_2(2) = 8 - 6 + 2 = 4$. ✓

Cubic spline (the important one). Each piece $P_i = a_i x^3 + b_i x^2 + c_i x + d_i$: 4 unknowns/piece $\Rightarrow 4n$ unknowns.

$$4n = \underbrace{2n}_{\text{through pts}} + \underbrace{(n-1)}_{P' \text{ match}} + \underbrace{(n-1)}_{P'' \text{ match}} + \underbrace{2}_{\text{boundary}}$$

Ensures continuity *and* smoothness (matching curvature). Benefits: good function fit, reduces oscillation, smooth and continuous. **Natural cubic:** $P''_1(x_{first}) = 0$, $P''_n(x_{last}) = 0$ (zero curvature at ends). **Clamped cubic:** given end slopes dy/dx instead.

Q1: What is Runge's Phenomenon and why does it motivate splines?

A: High-order global polynomial interpolants oscillate badly, especially near the edges of equally-spaced nodes, as more points are added. Splines avoid this by fitting many small, low-order local pieces instead of one large polynomial.

Q2: What does "smoothness" mean at a spline joint?

A: Both curves meeting at the joint share the same tangent (first derivative match); if the tangents differ, there is a visible kink.

Q3: Contrast natural vs. clamped cubic-spline boundary conditions.

A: Natural: second derivative (curvature) is zero at both end nodes. Clamped: the first-derivative (slope) values at the two end nodes are given instead. Either supplies the 2 extra equations needed to pin down all $4n$ unknowns.

Q4: What is the trade-off in choosing spline order?

A: Higher order gives more smoothness but costs more computation and carries more chance of error; linear is cheapest but has corner kinks, cubic is smooth/continuous but costlier.

8 Simulation & Modeling

8.1 Introduction to Simulation and Modeling

Definition

Simulation: the imitation of the operation of a real-world process or system over time. We generate an artificial history of a system, observe it, and draw conclusions about the real system.

System: a collection of entities that act and interact together toward some logical end. **System environment:** everything outside the system boundary — it can affect the system, but the system cannot control it.

System boundary depends on the study's purpose — e.g. studying bank waiting times $\Rightarrow$ system = tellers + customers in line; studying the whole bank operation $\Rightarrow$ system also includes loan officers, ATMs, etc.

Components of a system: Entity (object of interest), Attribute (property of an entity), Activity (a time period of specified length), Event (an instantaneous occurrence that may change system state), State variable (describes the system at any time). *State variables change only when events occur* — nothing changes between events.

Discrete system: state changes at separated points in time (e.g. # customers in a bank). **Continuous system:** state changes continuously (e.g. water level behind a dam). Most real systems mix both; classified by the predominant type.

Definition

Model: a representation of a system for the purpose of studying it — always a simplification (analogy: a map of Dhaka shows roads and landmarks, not every brick).

Three classification dimensions of simulation models:

1. **Static vs. Dynamic** — a snapshot (e.g. Monte Carlo) vs. evolution over time (e.g. a bank from 9am–4pm).
2. **Deterministic vs. Stochastic** — no randomness (same input $\rightarrow$ same output) vs. random variables (outputs are estimates).
3. **Discrete vs. Continuous** — state changes at separated points vs. continuously.

This course focuses on **Discrete-Event Simulation (DES)**: Dynamic + Stochastic + Discrete.

Key Point / Exam Tip

Because a stochastic model's inputs are random, its *outputs are random too* — running the same model twice with different random numbers gives different results. Simulation outputs are **estimates**, not exact answers, and require statistical interpretation.

How to study a system (decision tree): System $\rightarrow$ experiment with the actual system, or with a model. Model $\rightarrow$ physical model, or mathematical model. Mathematical model $\rightarrow$ analytical solution, or **simulation** (used when the mathematical model is too complex to solve analytically).

When simulation IS the right tool (any of): system too complex mathematically; want to experiment without disrupting the real system; system doesn't exist yet; want extreme-condition tests; want to compress/expand time; want to train people safely; want to test designs before committing resources.

When simulation is NOT appropriate: (1) common sense suffices (e.g. minimum servers = arrival rate / service rate per server, no simulation needed); (2) the problem has a clean analytical/closed-form solution — “math gives you answers, simulation gives you estimates”; (3) direct real-world experiments are cheaper; also: cost exceeds savings, no time/budget, no data available, or system behavior is too complex to even define.

Q1: Give the formal definition of simulation.

A: The imitation of the operation of a real-world process or system over time.

Q2: What determines a system's boundary?

A: The purpose of the study — the same real-world setup can have different boundaries depending on what's being investigated.

Q3: Define entity, attribute, activity, event, and state variable.

A: Entity = object of interest; attribute = a property of an entity; activity = a time period of specified length; event = an instantaneous occurrence that may change system state; state variable = a variable describing the system at any time.

Q4: List the three dimensions used to classify simulation models, and which combination defines DES.

A: Static/Dynamic, Deterministic/Stochastic, Discrete/Continuous; DES = Dynamic + Stochastic + Discrete.

Q5: Why must realistic simulation models be stochastic rather than deterministic?

A: The real world is random (arrivals, service times, breakdowns vary); ignoring randomness produces a dangerously wrong model.

Q6: State the three explicit “don’t simulate” rules.

A: (1) Don’t simulate if common sense suffices; (2) don’t simulate if the problem has a clean analytical solution; (3) don’t simulate if direct experiments are cheaper.

Q7: Give two of the seven reasons simulation is the right tool.

A: Any two of: system too complex to solve mathematically; want to experiment without disrupting the real system; system doesn’t exist yet; want extreme-condition tests; want to compress/expand time; want to train people safely; want to test designs before committing resources.

8.2 Discrete-Event Simulation & the Single-Server Queue

Definition

Discrete-event simulation (DES): the modeling of a system in which state variables change only at a discrete set of points in time, called events; nothing changes between events. **Simulation clock:** a variable recording current simulated time (simulated time $\neq$ real computer time).

Next-Event Time Advance (used throughout this course): jump the clock directly from one event to the next, skipping idle gaps — used by virtually all modern simulation software. **Fixed-Increment Time Advance:** advance by fixed Δt and check for events each step — wasteful.

Ten components of a programmed DES model: system state, simulation clock, event list (a.k.a. future event list — scheduled events with their times), statistical counters, initialization routine, timing routine (finds next event, advances clock), event routines (update state on an event; may call library routines for random variates), library routines, report generator, main program.

Five components of a queueing system: calling population, arrival process, service mechanism, queue discipline (e.g. FIFO), system capacity.

Worked Example

Flow of Control in a DES. Start: main program invokes initialization (clock = 0; init. state & counters; init. event list). **Repeat:** (1) timing routine determines next event type i and advances the clock; (2) event routine i updates state, updates counters, schedules future events. **Check:** simulation over? No $\rightarrow$ repeat; Yes $\rightarrow$ report generator (compute estimates, write report) $\rightarrow$ stop. **Order of updates at each event (critical, exam-relevant):** (1) update area accumulators using the *old* state values; (2) update state variables; (3) update time-of-last-event to the current clock; (4) update the event list. Getting this order wrong (e.g. updating state before computing areas) is one of the most common simulation bugs.

Notation. A_i = interarrival time between customers $i-1, i$; S_i = service time of customer i ; D_i = delay (waiting time before service; = 0 if server idle on arrival). Arrival time of customer $i = \sum_{j \leq i} A_j$; total time in system = $D_i + S_i$. $Q(t)$ = # in queue at time t (step function); $B(t) = 1$ if server busy at t , else 0.

$$\hat{d}(n) = \frac{1}{n} \sum_{i=1}^n D_i \quad (\text{discrete-time statistic — simple average})$$

$$\hat{q}(n) = \frac{\int_0^{T(n)} Q(t) dt}{T(n)}, \quad \hat{u}(n) = \frac{\int_0^{T(n)} B(t) dt}{T(n)} \quad (\text{continuous-time — time-weighted})$$

Why time-weighted? A queue of length 5 lasting 10 minutes matters more than one lasting 0.1 seconds; naively averaging values sampled only at event times is biased. (E.g. queue empty 9 hrs, length 10 for 1 hr: naive average $(0+10)/2 = 5$ is wrong; correct time-weighted answer $\approx (0 \cdot 9 + 10 \cdot 1)/10 = 1.0$.)

Area accumulation rule: new area = (old value of $Q(t)$ or $B(t)$) $\times$ (clock – time of last event)
 — always use the value that held *during* the just-elapsed interval.

Worked Example

Full worked example — single-server queue. $A_1..A_9 = 0.4, 1.2, 0.5, 1.7, 0.2, 1.6, 0.2, 1.4, 1.9$;
 $S_1..S_6 = 2.0, 0.7, 0.2, 1.1, 3.7, 0.6$. Simulate until $n = 6$ delays observed.

Event trace (clock jumps 0.4, 1.6, 2.1, 2.4, 3.1, 3.3, 3.8, 4.0, 4.9, 5.6, 5.8, 7.2, 8.6 — 13 events):

t	Event	Q	Delay	#Del
0.4	Arr. C1 (server idle)	0	$D_1=0$	1
1.6	Arr. C2 (server busy)	$0 \rightarrow 1$	–	1
2.1	Arr. C3	$1 \rightarrow 2$	–	1
2.4	Dep. C1; C2 starts	$2 \rightarrow 1$	$D_2=0.8$	2
3.1	Dep. C2; C3 starts	$1 \rightarrow 0$	$D_3=1.0$	3
3.3	Dep. C3 (queue empty, dep. $\rightarrow \infty$)	0	–	3
3.8	Arr. C4 (server idle)	0	$D_4=0$	4
4.0	Arr. C5	$0 \rightarrow 1$	–	4
4.9	Dep. C4; C5 starts	$1 \rightarrow 0$	$D_5=0.9$	5
5.6	Arr. C6	$0 \rightarrow 1$	–	5
5.8	Arr. C7	$1 \rightarrow 2$	–	5
7.2	Arr. C8	$2 \rightarrow 3$	–	5
8.6	Dep. C5; C6 starts	$3 \rightarrow 2$	$D_6=3.0$	6

Stop: 6 delays observed, $T(6) = 8.6$. **Key rule:** when the queue empties, set the next departure time to ∞ so the timing routine can't process a "phantom" departure — this forces the next event to be an arrival.

Results: $D_1..D_6 = 0, 0.8, 1.0, 0, 0.9, 3.0$:

$$\hat{d}(6) = \frac{5.7}{6} = 0.95 \text{ (avg. delay)}, \quad \hat{q}(6) = \frac{9.9}{8.6} = 1.15 \text{ (avg. \# in queue)}$$

$$\hat{u}(6) = \frac{7.7}{8.6} = 0.90 \text{ (server busy 90\% of the time)}$$

Caveat: these are estimates from a single run with only $n = 6$ — real studies use $n = 1000+$ with multiple independent replications.

Q1: What is the simulation clock, and how does it relate to real computer time?

A: A variable recording current simulated time; it is unrelated to actual wall-clock computer time (24 simulated hours might run in 0.01s, or 1ns of network activity might take 10 minutes to simulate).

Q2: Compare next-event and fixed-increment time advance.

A: Next-event jumps directly to the next scheduled event, skipping idle periods (used by virtually all modern software); fixed-increment checks every Δt step, wasting computation on steps where nothing happens.

Q3: Why is delay D_i the focus rather than total time in system?

A: D_i measures system inefficiency (pure waste); service time S_i is inherent to the task itself.

Q4: Why is $\hat{q}(n)$ a time-weighted average rather than a simple average of observed queue lengths?

A: Because a queue length that persists longer should count more — sampling only at event times and averaging naively is biased; you must weight by the duration each value held.

Q5: State the area-accumulation rule and explain why it uses the *old* state value.

A: new area = (old $Q(t)$ or $B(t)$) $\times$ (clock – time of last event); the old value is what actually

held throughout the just-elapsed interval, so using the post-update (new) value would give the wrong rectangle height.

Q6: What is the correct order of updates at each event, and what bug results from getting it wrong?

A: (1) Area accumulators (old values); (2) state variables; (3) time-of-last-event; (4) event list. Reversing (1) and (2) gives wrong rectangle heights/widths — one of the most common simulation bugs.

Q7: Why set the next departure time to ∞ when the queue becomes empty?

A: To force the next event to be an arrival rather than a nonexistent (“phantom”) departure.

8.3 Steps in a Simulation Study

Key Point / Exam Tip

A simulation study is **not a simple sequential process** — validation failures send you back to earlier steps, most often all the way back to Step 2 (data & model), since invalid output usually means the conceptual model or data is wrong, not just the code.

The 10 steps (condensed):

1. **Formulate problem & plan the study** — kickoff meeting(s) with manager, analysts, SMEs fix objectives, questions to answer (these determine required model detail), performance measures, scope, and resources.
2. **Collect data & define a model** — gather structure/procedure info and parameter data; write an **assumptions document**; start simple and embellish only as needed (no forced one-to-one correspondence between model and real-system elements).
3. **Assumptions document valid?** — structured walk-through before managers/analysts/SMEs; **No** → back to Step 2.
4. **Construct & verify the program** — **Verification** = does the program match the model? (code review, tracing, output-reasonableness checks, simplifying-assumption runs with known answers).
5. **Pilot runs** — preliminary, non-final runs for validation (“drive around the block before a 1000 km road trip”).
6. **Model valid?** — compare to existing-system data if available; sensitivity analysis on influential factors; **No** → back to Step 2 (not just to programming).
7. **Design experiments** — specify run length, warmup period (if the “empty and idle” start is unrealistic), and number of independent replications.
8. **Production runs** — the real, decision-driving runs using the Step 7 design.
9. **Analyze output data** — (a) absolute performance of one configuration, (b) relative comparison of alternatives; requires confidence intervals / statistical tests, since output is random — treating a single run’s number as deterministic is a serious error.
10. **Document, present, use results** — document assumptions, code, and results; present with animation for credibility; use only if results are both **valid** (technical) and **credible** (social/trusted).

Worked Example

The Two Validation Checkpoints. Step 2 → “Assumptions valid?” → No: loop back to Step 2; Yes: Steps 4–5 → “Model valid?” → No: loop back to Step 2 (*not* to Step 4); Yes: Steps 7–10. Both failure branches return all the way to Step 2 by design.

Q1: What determines the required level of model detail?

A: The specific questions the study must answer (fixed at the Step 1 kickoff meeting) — not decided arbitrarily up front.

Q2: If the Step 6 validity check fails, where does the process return, and why?

A: All the way back to Step 2 — invalid output usually means the conceptual model or data is wrong, not merely the code.

Q3: What must Step 7 specify for each configuration?

A: Run length, warmup-period length (if needed), and the number of independent replications.

Q4: How do production runs differ from pilot runs?

A: Production runs use the full experimental design (proper lengths, warmup, replications) and drive real decisions; pilot runs are preliminary, used only to validate the model.

Q5: Why is treating simulation output as deterministic a serious error?

A: Outputs are random because inputs are random; a single run's number is unreliable without confidence intervals and significance tests.

Q6: What two conditions must results satisfy to be used in decision-making?

A: They must be **valid** (technically accurate) and **credible** (decision-makers trust them enough to act).

8.4 Model Verification and Validation

Definition

Verification: “Did we build the model right?” — does the computer program correctly implement the conceptual model? **Validation:** “Did we build the right model?” — does the model's behavior match reality? **Calibration:** the iterative compare→revise→compare loop that drives validation.

The V&V triangle (Real System, Conceptual Model, Operational Model at the corners): Real $\leftrightarrow$ Conceptual = *conceptual validation*; Real $\leftrightarrow$ Operational = *calibration & validation*; Conceptual $\leftrightarrow$ Operational = *model verification*. Pursued iteratively, not once — a model can be verified yet still invalid (correct code, wrong assumptions), or vice versa.

Key Point / Exam Tip

Apparent realism does not equal validity: “a wrong model, convincingly animated, gives you false confidence — like a GPS confidently driving you off a cliff.” Validation is not binary either: no model is ever 100% valid; the real question is whether it's accurate enough for the decisions at hand.

Naylor–Finger (1967) three-step validation approach:

1. **Face validity** — does the model look reasonable to knowledgeable people? User involvement improves assumptions, data reliability, bug-catching, and credibility.
2. **Validate model assumptions** — *structural* (system organization: queues, servers, discipline — validated by observation/discussion) and *data* assumptions (input distributions — validated via: identify distribution $\rightarrow$ estimate parameters $\rightarrow$ goodness-of-fit test, e.g. chi-square or K-S).
3. **Validate input–output transformations** — view the model as $f(X, D) = Y$ (X =uncontrollable inputs, D =decision variables, Y =outputs); do model outputs match real outputs for the same inputs? Requires an existing (or comparable) system — a purely planned system can never be *completely* validated this way.

Minor vs. major changes (validating a model of a modified system): minor (single parameter, or distribution form) $\Rightarrow$ carry over confidence after careful verification; major (logical/structural change, or wholly new design) $\Rightarrow$ at best partial validation.

Eight verification techniques (always do the first two + documentation): (1) independent code review, (2) flow diagrams, (3) examine output for reasonableness (forecast expected ranges *before*

running, to avoid rationalizing a bug away), (4) check input parameters weren't altered during execution, (5) self-documentation, (6) animation, (7) Interactive Run Controller/debugger, (8) graphical interfaces.

Worked Example

Verification by trace — catching a bug. A single-server queue run over 16 time units gave $\hat{L}_Q = 0.4375$ — looked perfectly reasonable. Trace revealed:

CLOCK	Event	NCUST	STATUS
0	Start	0	Idle
3	Arrival	1	Idle ← Bug!
5	Depart	0	Idle
11	Arrival	1	Idle
12	Arrival	2	Busy
16	Depart	1	Busy

At CLOCK= 3 there is 1 customer but STATUS shows idle — impossible (service should start immediately). Yet the statistic was *computed correctly* from this wrong data:

$$\hat{L}_Q = \frac{(1-0)(2) + (0-0)(6) + (1-0)(1) + (2-1)(4)}{16} = \frac{7}{16} = 0.4375$$

Moral: a reasonable-looking, correctly-computed summary statistic can still be wrong if the underlying state data is wrong — the trace caught an error the summary statistic completely hid.

Q1: State the one-line definitions of verification and validation.

A: Verification: does the program correctly implement the conceptual model? Validation: does the model's behavior match reality?

Q2: What is calibration, and how does it relate to validation?

A: The iterative compare→revise→compare loop between model and real system; it is the mechanism that usually *achieves* validation.

Q3: Describe the three edges of the V&V triangle.

A: Real↔Conceptual model = conceptual validation; Real↔Operational model = calibration & validation; Conceptual↔Operational model = model verification.

Q4: Why can a model appear convincing yet still be invalid?

A: Visual/behavioral detail (e.g. animation) can look realistic even when the underlying logic doesn't match reality — apparent realism is not validity.

Q5: What are the three steps of the Naylor–Finger approach?

A: (1) Face validity; (2) validate structural and data assumptions; (3) validate input-output transformations.

Q6: Why can a model of a brand-new (nonexistent) system never be completely validated?

A: Input-output validation requires comparing model outputs to real-system outputs for the same inputs, which requires an actual (or closely comparable) system to exist and produce data — a purely planned system has none.

Q7: In the trace-bug example, what did the trace reveal that the summary statistic did not?

A: The statistic $\hat{L}_Q = 0.4375$ was computed correctly but from wrong underlying data (STATUS incorrectly showed the server idle while a customer was present) — the trace exposed a state-update bug the aggregate number completely concealed.

Q8: Why split validation data into calibration and validation sets?

A: To show the model generalizes to unseen data rather than being merely fitted/tuned to the one dataset used to build it.

9 Random Number Generation & Monte Carlo Simulation

9.1 Random Number Generation

Definition

A random number R is drawn from $[0, 1]$ with every value equally likely: $R \sim U(0, 1)$. A sequence $R_1, R_2, \dots$ must satisfy **two properties simultaneously**: **Uniformity** (no subinterval of $[0, 1]$ is preferred) and **Independence** (knowing R_1 tells you nothing about R_2). Since computers are deterministic, generated numbers are only *pseudo-random* — computed by a formula but statistically indistinguishable from true randomness. Repeatability from a known seed is actually useful: debugging, fair comparison of designs, and verification by others.

A good generator is: fast, portable, has a long period, is repeatable (same seed $\Rightarrow$ same sequence), and is statistically good (passes uniformity/independence tests).

Worked Example

Mean and variance of $U(0, 1)$. pdf $f(x) = 1$ on $[0, 1]$.

$$E(R) = \int_0^1 x \, dx = \left[\frac{x^2}{2} \right]_0^1 = \frac{1}{2}, \quad E(R^2) = \int_0^1 x^2 \, dx = \frac{1}{3}$$

$$\text{Var}(R) = E(R^2) - [E(R)]^2 = \frac{1}{3} - \frac{1}{4} = \frac{1}{12} \approx 0.0833$$

9.1.1 The Linear Congruential Generator (LCG)

$$X_{i+1} = (aX_i + c) \bmod m, \quad R_i = \frac{X_i}{m}$$

X_0 = seed, a = multiplier, c = increment, m = modulus. $c \neq 0$: **mixed** congruential; $c = 0$: **multiplicative** congruential. **Key weakness:** each X_i completely determines X_{i+1} — no actual randomness, only apparent randomness from the choice of (a, c, m) .

Period. Since $X_i \in \{0, \dots, m-1\}$, the sequence must eventually repeat; the period P is the length before it repeats. Ideal: $P = m$.

Maximum-period conditions:

- $m = 2^b$, $c \neq 0$ (mixed): $P = m$ if $\gcd(c, m) = 1$ and $a = 1 + 4k$.
- $m = 2^b$, $c = 0$ (multiplicative): $P = m/4 = 2^{b-2}$ only, if X_0 is odd and $a = 3 + 8k$ or $a = 5 + 8k$.
- m prime, $c = 0$: $P = m - 1$, if the smallest k with $m \mid (a^k - 1)$ is $k = m - 1$.

Density. Gap between representable R_i values = $1/m$; a large m (e.g. $2^{31} - 1 \approx 2.1 \times 10^9$) gives gap $\approx 4.7 \times 10^{-10}$ — “practically continuous.” Powers of 2 are attractive because $X \bmod 2^b$ is just “keep the last b bits” — essentially free on a binary computer (directly analogous to $\bmod 100$ = keep the last 2 decimal digits).

Worked Example

Worked example — basic LCG. $X_0 = 27$, $a = 17$, $c = 43$, $m = 100$.

$$X_1 = (17 \times 27 + 43) \bmod 100 = 502 \bmod 100 = 2 \Rightarrow R_1 = 0.02$$

$$X_2 = (17 \times 2 + 43) \bmod 100 = 77 \Rightarrow R_2 = 0.77, \quad X_3 = (17 \times 77 + 43) \bmod 100 = 52 \Rightarrow R_3 = 0.52$$

Worked Example

Worked example — period behavior. $X_{i+1} = 13X_i \bmod 64$ ($c = 0$), four seeds $X_0 = 1, 2, 3, 4$. Generating the full sequence for each seed until it returns to X_0 gives observed periods: $X_0 = 4 \rightarrow$ period 4; $X_0 = 2 \rightarrow$ period 8; $X_0 = 1 \rightarrow$ period 16; $X_0 = 3 \rightarrow$ period 16. **Odd seeds reach the**

max period; even seeds fall short.

Verification: $m = 2^6 = 64$, $c = 0 \Rightarrow$ multiplicative case, max period $= m/4 = 16$. Multiplier check: $a = 13 = 5 + 8(1) \checkmark$ (form $5 + 8k$). Seed check: $X_0 = 1, 3$ (odd) $\rightarrow$ period 16 $\checkmark$; $X_0 = 2, 4$ (even) $\rightarrow$ shorter periods (8, 4) as predicted.

Worked Example

A real-world generator. $a = 7^5 = 16,807$, $c = 0$, $m = 2^{31} - 1 = 2,147,483,647$ (prime $\Rightarrow$ max period $P = m - 1 > 2$ billion). Seed $X_0 = 123,457$: $X_1 = 2,074,941,799 \Rightarrow R_1 = X_1/2^{31} = 0.9662$; $X_2 = 559,872,160 \Rightarrow R_2 = 0.2607$; $X_3 = 1,645,535,613 \Rightarrow R_3 = 0.7662$.

9.1.2 Statistical Testing of RNGs

Definition

Two categories, matching the two sacred properties: **uniformity tests** (K-S, Chi-square) and **independence tests** (Autocorrelation, Runs, Serial). Passing all tests never *guarantees* randomness (a pattern might go undetected), but failing one is a red flag. Hypotheses: $H_0 : R_i \sim U(0, 1)$ (or independent) vs. H_1 : not so, at significance level α (typically 0.05).

Multiple-testing problem. For k independent tests at level α , all H_0 true:

$$P(\text{at least one false rejection}) = 1 - (1 - \alpha)^k$$

E.g. $\alpha = 0.05$: $k = 10 \rightarrow 0.40$, $k = 20 \rightarrow 0.64$. **Lesson:** an isolated test failure does not prove an RNG is bad — repeated failures of the *same* test, or failures across several different tests, are the real red flag.

Kolmogorov–Smirnov (K-S) test. Sort sample: $R_{(1)} \leq \dots \leq R_{(N)}$.

$$D^+ = \max_i \left(\frac{i}{N} - R_{(i)} \right), \quad D^- = \max_i \left(R_{(i)} - \frac{i-1}{N} \right), \quad D = \max(D^+, D^-)$$

Reject H_0 if $D > D_\alpha$ (table value for given N, α).

Worked Example

Worked K-S test. Data 0.44, 0.81, 0.14, 0.05, 0.93, $N = 5, \alpha = 0.05$. Sorted: 0.05, 0.14, 0.44, 0.81, 0.93.

i	$R_{(i)}$	i/N	D_i^+	$(i-1)/N$	D_i^-
1	0.05	0.20	0.15	0.00	0.05
2	0.14	0.40	0.26	0.20	0.06
3	0.44	0.60	0.16	0.40	0.04
4	0.81	0.80	-0.01	0.60	0.21
5	0.93	1.00	0.07	0.80	0.13

$D^+ = 0.26$, $D^- = 0.21 \Rightarrow D = 0.26$. $D_{0.05,5} = 0.565$. Since $0.26 < 0.565$: **do not reject H_0** .

Chi-square test. Bins $1, \dots, n$; $E_i = N/n$; O_i observed count:

$$\chi_0^2 = \sum_{i=1}^n \frac{(O_i - E_i)^2}{E_i}$$

Reject if $\chi_0^2 > \chi_{\alpha, n-1}^2$. Valid for large samples ($N \gtrsim 50$); K-S is more powerful and works for small samples too.

Worked Example

Worked Chi-square test. $N = 100$, $n = 10$ bins, $E_i = 10$ each.

O_i (10 bins, [0,1) split into tenths):	8,8,10,9,12,8,10,14,10,11	Total O_i	100
---	---------------------------	-------------	-----

$\chi_0^2 = \sum (O_i - E_i)^2 / E_i = 0.4 + 0.4 + 0 + 0.1 + 0.4 + 0.4 + 0 + 1.6 + 0 + 0.1 = 3.4$. Critical value $\chi_{0.05,9}^2 = 16.9$. Since $3.4 \ll 16.9$: **do not reject** H_0 — “looks fine.”

Autocorrelation test. Tests independence at lag ℓ starting at index i ; $H_0 : \rho_{i\ell} = 0$ (two-tailed — both > 0 and < 0 indicate dependence). M = largest integer with $i + (M+1)\ell \leq N$.

$$\hat{\rho}_{i\ell} = \frac{1}{M+1} \sum_{k=0}^M R_{i+k\ell} R_{i+(k+1)\ell} - 0.25$$

(0.25 subtracted since independent $U(0,1)$'s give $E[RR'] = \frac{1}{2} \cdot \frac{1}{2} = 0.25$.)

Worked Example

Derivation of $\sigma_{\hat{\rho}_{i\ell}}$. Let $Y_k = R_{i+k\ell} R_{i+(k+1)\ell} = AB$, A, B independent $U(0,1)$.

$$E[Y_k] = E[A]E[B] = \frac{1}{4}, \quad E[Y_k^2] = E[A^2]E[B^2] = \frac{1}{9}, \quad \text{Var}(Y_k) = \frac{1}{9} - \frac{1}{16} = \frac{7}{144}$$

Adjacent products $Y_k = AB$, $Y_{k+1} = BC$ share B :

$$E[Y_k Y_{k+1}] = E[A]E[B^2]E[C] = \frac{1}{2} \cdot \frac{1}{3} \cdot \frac{1}{2} = \frac{1}{12}, \quad \text{Cov}(Y_k, Y_{k+1}) = \frac{1}{12} - \frac{1}{16} = \frac{1}{48}$$

(non-adjacent products have zero covariance). With $S = \sum_{k=0}^M Y_k$ and $\hat{\rho}_{i\ell} = \frac{S}{M+1} - 0.25$:

$$\text{Var}(S) = (M+1) \frac{7}{144} + 2M \frac{1}{48}, \quad \text{Var}(\hat{\rho}_{i\ell}) = \frac{\text{Var}(S)}{(M+1)^2} = \frac{13M+7}{144(M+1)^2}$$

$$\sigma_{\hat{\rho}_{i\ell}} = \sqrt{\text{Var}(\hat{\rho}_{i\ell})} = \frac{\sqrt{13M+7}}{12(M+1)} \quad \blacksquare$$

Test statistic $Z_0 = \hat{\rho}_{i\ell} / \sigma_{\hat{\rho}_{i\ell}}$; reject H_0 if $|Z_0| > 1.96$ ($\alpha = 0.05$).

Worked Example

Worked autocorrelation test. $i = 3$, $\ell = 5$, $N = 30 \Rightarrow M = 4$. Subsequence $R_3, R_8, \dots, R_{28} = 0.23, 0.28, 0.33, 0.27, 0.05, 0.36$.

$$\begin{aligned} \hat{\rho}_{35} &= \frac{1}{5} [(0.23)(0.28) + (0.28)(0.33) + (0.33)(0.27) + (0.27)(0.05) + (0.05)(0.36)] - 0.25 \\ &= 0.0555 - 0.25 = -0.1945 \end{aligned}$$

$$\sigma = \frac{\sqrt{13(4)+7}}{12(5)} = \frac{\sqrt{59}}{60} = 0.1280, \quad Z_0 = \frac{-0.1945}{0.1280} = -1.52$$

$|Z_0| = 1.52 < 1.96$: **do not reject** H_0 — no evidence of autocorrelation.

Q1: What two properties must a random-number sequence satisfy, and must they hold together?

A: Uniformity and Independence, simultaneously — one without the other is not enough (e.g. $0.01, 0.02, \dots, 0.09, 0.11, \dots$ is perfectly uniform but completely predictable).

Q2: Why are computer-generated random numbers called “pseudo-random”?

A: Computers are deterministic — the same seed always produces the same sequence via a formula, not true randomness; but this repeatability enables debugging, fair comparison, and verification.

Q3: What is the key structural weakness of an LCG?

A: Each X_i completely determines X_{i+1} — there is no actual randomness, only the appearance of it, entirely governed by the choice of (a, c, m, X_0) .

Q4: Why do many real generators choose m as a power of 2?

A: Because $X \bmod 2^b$ is just “keep the last b bits” on a binary computer — essentially free to compute — though (per the multiplicative case) this only achieves period $m/4$, a speed/period trade-off versus a prime modulus.

Q5: Why does testing an RNG with many statistical tests risk false alarms?

A: By the multiple-testing effect, $P(\text{at least one false rejection}) = 1 - (1 - \alpha)^k$ grows quickly with k (e.g. $\approx 40\%$ for 10 tests at $\alpha = 0.05$) even for a perfectly correct generator — so isolated failures don’t prove the RNG is bad.

Q6: Why are uniformity tests (K-S, Chi-square) alone insufficient to certify a generator?

A: They ignore ordering — a sequence can be perfectly uniform yet fully predictable in sequence; independence must be separately tested (e.g. via autocorrelation).

Q7: In the autocorrelation test, why is 0.25 subtracted in $\hat{\rho}_{i\ell}$?

A: For independent $U(0, 1)$ variables, $E[RR'] = E[R]E[R'] = \frac{1}{2} \cdot \frac{1}{2} = 0.25$; subtracting it centers the estimator at 0 under the null (independence).

Q8: Why is the autocorrelation test two-tailed?

A: Both $\rho_{i\ell} > 0$ (products larger than expected) and $\rho_{i\ell} < 0$ (smaller than expected) signal dependence; only $\rho_{i\ell} = 0$ indicates independence.

Q9: Compare K-S and Chi-square tests.

A: Both test uniformity; K-S is more powerful and valid even for small samples, while Chi-square requires large samples ($N \gtrsim 50$).

9.2 Monte Carlo Simulation

Definition

A scheme employing random numbers to solve stochastic *or deterministic* problems (named after the Monte Carlo Casino; developed during WWII on the Manhattan Project by Stanislaw Ulam and John von Neumann). The surprising part: randomness can compute an exact, non-random answer (e.g. estimating π).

Motivation. Evaluating $I = \int_a^b g(x) dx$ when g has no closed form; classical quadrature (Simpson’s, trapezoidal) works well in 1-D but struggles in high dimensions — Monte Carlo does not suffer the same curse of dimensionality.

Worked Example

Derivation — integration as expectation. Let $X \sim U(a, b)$ (pdf $f_X(x) = \frac{1}{b-a}$) and $Y = (b-a)g(X)$.

$$E(Y) = (b-a) \int_a^b g(x) f_X(x) dx = (b-a) \cdot \frac{1}{b-a} \int_a^b g(x) dx = \int_a^b g(x) dx = I \quad \blacksquare$$

So the deterministic integral I equals the expected value of the random variable Y — an estimation problem, solved (by the Law of Large Numbers) with a sample mean:

$$\bar{Y}(n) = (b-a) \cdot \frac{1}{n} \sum_{i=1}^n g(X_i), \quad X_1, \dots, X_n \stackrel{iid}{\sim} U(a, b)$$

Geometrically, $\bar{Y}(n)$ is the area of a rectangle of base $(b-a)$ and height equal to the average sampled height of g .

Worked Example

Worked example — $\int_0^\pi \sin x \, dx$, $n = 10$. Exact: $[-\cos x]_0^\pi = 2$. Sampled $X_i \sim U(0, \pi)$: 0.52, 1.19, 2.83, 0.21, 1.77, 2.50, 0.89, 1.53, 2.97, 2.11; $\sin(X_i)$: 0.497, 0.929, 0.301, 0.208, 0.980, 0.599, 0.777, 0.999, 0.172, 0.870.

$$\bar{g} = \frac{1}{10} \sum \sin(X_i) = 0.633, \quad \bar{Y}(10) = \pi \times 0.633 = 1.989$$

True value 2; error $\approx 0.5\%$ with just 10 points.

Convergence table (a separate run).

n	$\bar{Y}(n)$	Error
10	2.213	10.7%
20	1.951	2.5%
40	1.948	2.6%
80	1.989	0.6%
160	1.993	0.4%

Note $\bar{Y}(40) = 1.948$ is *worse* than $\bar{Y}(20) = 1.951$ — not a bug: $\bar{Y}(n)$ is itself a random variable, so error need not shrink monotonically; it converges only “on average” (Law of Large Numbers).

Worked Example

Estimating π . Quarter circle radius 1 inscribed in the unit square: $P(\text{inside circle}) = \frac{\pi/4}{1} = \frac{\pi}{4}$. Generate n pairs $(X_i, Y_i) \sim U(0, 1)$, count hits h where $X_i^2 + Y_i^2 \leq 1$:

$$\hat{\pi} = \frac{4h}{n}$$

n	$\hat{\pi}$
100	3.24
1,000	3.16
10,000	3.1436
100,000	3.1412
1,000,000	3.14178

Converges toward $\pi = 3.14159\dots$ as n grows (not strictly monotonically).

Q1: Why can Monte Carlo simulation solve deterministic problems (like computing π)?

A: Any deterministic quantity that can be expressed as an expected value or a probability (e.g. area ratio) can be estimated by sampling — the randomness is a computational tool, not a property of the answer itself.

Q2: What is the key trick that converts $\int_a^b g(x) \, dx$ into a Monte Carlo estimation problem?

A: If $X \sim U(a, b)$ and $Y = (b-a)g(X)$, then $E(Y) = \int_a^b g(x) \, dx$; the integral becomes the expected value of a random variable, estimated by a sample mean.

Q3: Why does the Monte Carlo estimate improve as n increases, and does it do so monotonically?

A: By the Law of Large Numbers, the sample mean converges to the true expectation as $n \rightarrow \infty$ — but $\bar{Y}(n)$ is itself a random variable, so an individual run’s error need not shrink at every step (e.g. $n = 40$ can be worse than $n = 20$).

Q4: Describe the Monte Carlo method for estimating π and justify the formula $\hat{\pi} = 4h/n$.

A: Inscribe a quarter unit-circle in a unit square; generate random $(X, Y) \in [0, 1]^2$ and count hits

with $X^2 + Y^2 \leq 1$. Since $P(\text{hit}) = \pi/4$, the hit fraction $h/n \approx \pi/4$, so $\pi \approx 4h/n$.

Q5: Why is Monte Carlo integration preferred over Simpson's/trapezoidal rule in some cases?

A: Classical quadrature works well in 1-D but degrades badly in high-dimensional (multiple) integrals; Monte Carlo does not suffer the same curse of dimensionality.

Part II — Problem Bank

MCQs, multi-step problems and trap questions, topic by topic

10 Errors & Root Finding — Problem Bank

10.1 Errors — MCQs, Problems & Traps

MCQ / Rapid-Fire Bank

Q1. An iteration reports $|\epsilon_a| = 0.5\%$. The number of significant digits guaranteed correct is
(a) 1 (b) 2 (c) 3 (d) 4

Ans: (b) — $m = \lfloor 2 - \log_{10}(2 \times 0.5) \rfloor = \lfloor 2 - 0 \rfloor = 2$. Memorize the ladder 5%, 0.5%, 0.05%, 0.005% $\rightarrow$ 1, 2, 3, 4 digits.

Q2. 0.00640 has how many decimal places and significant figures respectively?

(a) 5 and 5 (b) 3 and 3 (c) 5 and 3 (d) 3 and 5

Ans: (c) — decimal places count everything after the point; significant figures start at the first non-zero digit (6, 4, 0).

Q3. The central-difference derivative approximation has truncation error of order

(a) $O(h)$ (b) $O(h^2)$ (c) $O(h^3)$ (d) $O(1)$

Ans: (b) — the odd-order Taylor terms cancel. Forward difference is only $O(h)$.

Q4. A forward-difference derivative of $\sin x$ is computed with $h = 10^{-1}, 10^{-2}, \dots, 10^{-16}$. The error is smallest at roughly

(a) $h = 10^{-16}$ (b) $h = 10^{-8}$ (c) $h = 10^{-2}$ (d) it decreases all the way

Ans: (b) — total error is U-shaped with minimum near $h^* \sim \sqrt{\epsilon_{\text{mach}}} \approx 10^{-8}$. Beyond that, round-off in the subtraction takes over and the answer gets *worse*.

Q5. Two numbers each accurate to 6 significant digits are subtracted and are nearly equal. What happens?

(a) absolute error grows (b) relative error grows sharply
(c) both errors shrink (d) nothing changes

Ans: (b) — the absolute error is unchanged but the result is tiny, so the *relative* error explodes. This is subtractive cancellation.

Q6. Which is computable *without* knowing the exact answer?

(a) E_t (b) ϵ_t (c) ϵ_a (d) all three

Ans: (c) — ϵ_a compares the present iterate with the previous one, which is why it is the quantity actually monitored.

Q7. In the trunnion case study the constant- α model predicted a contraction of 0.01504" against 0.015" required, yet the part jammed. This is an example of

(a) round-off error (b) truncation error (c) modelling error (d) a programming bug

Ans: (c) — the arithmetic was correct; the assumption that α is constant was not. No amount of extra precision fixes a wrong model.

Q8. Which statement is **FALSE**?

(a) ϵ_a is undefined at the first iteration (b) E_t can be negative
(c) Round-off error can be eliminated by using more iterations (d) Truncation error appears in numerical integration

Ans: (c) — round-off comes from finite precision, not from stopping early; more iterations can actually make it accumulate.

Practice Problems (with full solutions)

P1. A computation produces successive estimates 2.0122 then 2.0136. (i) Compute $|\epsilon_a|$. (ii) How many significant digits are guaranteed? (iii) What $|\epsilon_a|$ would be needed for 4 digits?

Solution. (i) $|\epsilon_a| = \left| \frac{2.0136 - 2.0122}{2.0136} \right| \times 100\% = 0.069527\%$.

(ii) $m = \lfloor 2 - \log_{10}(2 \times 0.069527) \rfloor = \lfloor 2 - \log_{10}(0.139054) \rfloor = \lfloor 2 + 0.8567 \rfloor = \lfloor 2.8567 \rfloor = 2$. Check against the criterion directly: $0.5 \times 10^{2-2}\% = 0.5\%$ and $0.069527 \leq 0.5\%$ ✓; for $m = 3$ we would need $\leq 0.05\%$, which fails.

(iii) For $m = 4$: $|\epsilon_a| \leq 0.5 \times 10^{2-4}\% = 0.005\%$.

P2. Show that the relative-error criterion $\left| \frac{x-\hat{x}}{x} \right| \leq \frac{1}{2} \times 10^{-M}$ implies the absolute window $|x - \hat{x}| \leq \frac{1}{2} \times 10^{n-M+1}$, where $|x| = m \times 10^n$ with $1 \leq m < 10$.

Solution. Multiply the hypothesis through by $|x| = m 10^n$:

$$|x - \hat{x}| = \left| \frac{x - \hat{x}}{x} \right| \cdot m 10^n \leq \frac{1}{2} \times 10^{-M} \cdot m 10^n = \frac{m}{2} \times 10^{n-M}.$$

Since normalization forces $m < 10$,

$$|x - \hat{x}| < \frac{10}{2} \times 10^{n-M} = 5 \times 10^{n-M} = \frac{1}{2} \times 10^{n-M+1},$$

which is exactly the absolute window — half a unit in the place of the M -th significant digit. ■

P3. $\pi = 3.14159265\dots$ is approximated by $\hat{x} = 3.14$. Verify that this is correct to 3 significant figures, and find the full band of values that would also qualify.

Solution. Here $|x| = 3.14159\dots = m \times 10^n$ with $m = 3.14159$, $n = 0$. For $M = 3$ the window is

$$|x - \hat{x}| \leq \frac{1}{2} \times 10^{0-3+1} = \frac{1}{2} \times 10^{-2} = 0.005.$$

Actual error: $|3.14159265 - 3.14| = 0.00159265 \leq 0.005$ ✓, so 3.14 is correct to 3 significant figures.
Band: $3.14159265 \pm 0.005 = [3.13659, 3.14659]$.

Exam Traps

- **The dash in the first row.** Any iteration table you are asked to complete has $|\epsilon_a|$ undefined on iteration 1. Filling in a number there is an instant mark lost.
- $0.5 \times 10^{2-m}\%$ **already carries the percent sign.** Converting to a fraction *and* keeping the % divides by 100 twice.
- **“Accurate to 5 decimal places” $\neq$ “accurate to 5 significant figures.”** For a number like 0.00640 these differ by two orders of magnitude.
- **Smaller h is not always better.** Below $h^* \sim \sqrt{\epsilon_{\text{mach}}}$ the forward-difference error rises again. An option saying “error $\rightarrow 0$ as $h \rightarrow 0$ ” is a distractor on a real machine.
- **E_t vs E_a .** If a question supplies only two successive iterates, it cannot be asking for true error — there is no true value in play.
- **Sign of the error.** $E_t = \text{true} - \text{approx}$, so a negative E_t means you *overestimated*. Options often differ only in sign.

10.2 Root Finding — MCQs, Problems & Traps

MCQ / Rapid-Fire Bank

Q9. Bisection is applied on $[2, 5]$. The minimum number of iterations guaranteeing an error below 10^{-4} is

- (a) 13 (b) 14 (c) 15 (d) 18

Ans: (c) — $n \geq \log_2(3/10^{-4}) = 14.87 \Rightarrow 15$. (Resulting width $3/2^{15} = 9.16 \times 10^{-5}$.)

Q10. For bisection, the number of iterations needed to reach a given tolerance depends on

- (a) the function only (b) the initial bracket and the tolerance only
(c) the function and the bracket (d) the derivative of the function

Ans: (b) — $n \geq \log_2 \frac{b-a}{\epsilon}$ contains no reference to f whatsoever.

Q11. Starting from $[1, 2]$, the bisection bracket width after 5 iterations is

- (a) 0.0625 (b) 0.03125 (c) 0.1 (d) 0.015625

Ans: (b) — $1/2^5 = 0.03125$.

Q12. $f(x) = 1/x$ on $[-2, 3]$ satisfies $f(x_\ell)f(x_u) < 0$. Bisection will

- (a) find the root $x = 0$ (b) converge to the singularity at $x = 0$, which is not a root
(c) fail to start (d) diverge

Ans: (b) — the sign change is caused by a discontinuity, not a zero. Bisection converges anyway and reports a wrong answer confidently.

Q13. $f(x) = x^2$ has a root at $x = 0$. Bisection

- (a) converges quadratically (b) needs one guess only
(c) cannot be started, since no sign change exists (d) converges in one step

Ans: (c) — the curve touches the axis without crossing, so no bracket with $f(x_\ell)f(x_u) < 0$ exists anywhere.

Q14. “Newton–Raphson converges quadratically” means

- (a) the error halves each step (b) the error is squared each step, so correct digits roughly double
(c) it needs two starting guesses (d) it converges in two iterations

Ans: (b) — $e_{k+1} \approx \frac{f''}{2f'} e_k^2$.

Q15. Newton–Raphson applied to $f(x) = x^2$ from $x_0 = 1$ produces

- (a) 0.5, 0.25, 0.125, ... (linear) (b) 0.5, 0.0625, ... (quadratic)
(c) 0 immediately (d) divergence

Ans: (a) — the step simplifies to $x_{k+1} = x_k - \frac{x_k^2}{2x_k} = \frac{x_k}{2}$. A double root destroys quadratic convergence.

Q16. To restore quadratic convergence at a root of multiplicity k , the iteration is modified to

- (a) $x - \frac{f}{kf'}$ (b) $x - k\frac{f}{f'}$ (c) $x - \frac{f'}{f}$ (d) $x - \frac{f}{f''}$

Ans: (b)

Q17. Newton’s method applied to $f(x) = x^2 - a$ gives the iteration

- (a) $x_{k+1} = \frac{a}{x_k}$ (b) $x_{k+1} = \frac{1}{2} \left(x_k + \frac{a}{x_k} \right)$ (c) $x_{k+1} = x_k(2 - ax_k)$ (d) $x_{k+1} = 2x_k - a$

Ans: (b) — the classical square-root (“Babylonian”) iteration.

Q18. Which statement about false position is **FALSE**?

- (a) It is a bracketing method (b) The bracket width always tends to zero
(c) One endpoint may remain fixed for many iterations (d) It uses a straight line through the two bracket endpoints

Ans: (b) — with a frozen endpoint the bracket width stalls, which is why $|\epsilon_a|$ rather than bracket width must be the stopping test.

Q19. The Intermediate Value Theorem, applied with $f(x_\ell)f(x_u) < 0$, guarantees

- (a) exactly one root (b) at least one root (c) at most one root (d) an even number of roots

Ans: (b)

Q20. In the floating-ball problem $f'(x) = 3x(x - 0.11)$. Which initial guess must be avoided for Newton–Raphson?

- (a) $x_0 = 0.05$ (b) $x_0 = 0.06$ (c) $x_0 = 0.11$ (d) $x_0 = 0.07$

Ans: (c) — $f'(0.11) = 0$, giving a horizontal tangent and division by zero (as does $x_0 = 0$).

Practice Problems (with full solutions)

P4. Apply **bisection** to $f(x) = x^3 - x - 2$ on $[1, 2]$ for 5 iterations. Tabulate $x_\ell, x_u, x_m, f(x_m)$ and $|\epsilon_a|$. State how many significant digits are certified at iteration 5, and confirm the width against the L_n formula.

Solution. $f(1) = -2 < 0$, $f(2) = 4 > 0$, so a root is bracketed.

Iter	x_ℓ	x_u	x_m	$f(x_m)$	$ \epsilon_a \%$
1	1.000000	2.000000	1.500000	-0.125000	—
2	1.500000	2.000000	1.750000	+1.609375	14.2857
3	1.500000	1.750000	1.625000	+0.666016	7.6923
4	1.500000	1.625000	1.562500	+0.252197	4.0000
5	1.500000	1.562500	1.531250	+0.059113	2.0408

Digits at iteration 5: $m = \lfloor 2 - \log_{10}(2 \times 2.0408) \rfloor = \lfloor 2 - 0.6107 \rfloor = \lfloor 1.389 \rfloor = 1$ — only one digit after five iterations, which is exactly bisection's reputation.

Width check: $L_5 = (2 - 1)/2^5 = 0.03125$, and indeed $1.562500 - 1.531250 = 0.03125$ ✓. (True root 1.5213797.)

P5. Apply **false position** to the same $f(x) = x^3 - x - 2$ on $[1, 2]$ for 4 iterations. Comment on which endpoint moves.

Solution.

Iter	x_L	x_U	x_r	$f(x_r)$
1	1.000000	2.000000	1.333333	-9.630×10^{-1}
2	1.333333	2.000000	1.462687	-3.333×10^{-1}
3	1.462687	2.000000	1.504019	-1.018×10^{-1}
4	1.504019	2.000000	1.516331	-2.989×10^{-2}

$|\epsilon_a|$ at iteration 4 = $|(1.516331 - 1.504019)/1.516331| \times 100\% = 0.8119\%$.

Observation: $f(x_r)$ is negative every time, so x_r always replaces x_L ; the right endpoint stays **frozen at** $x_U = 2$ **forever**. The bracket width only falls from 1 to 0.4837 in four iterations even though x_r is converging nicely — the textbook illustration of why bracket width is a useless stopping test here.

P6. Apply **Newton–Raphson** to $f(x) = x^3 - x - 2$ from $x_0 = 1.5$. Do 3 iterations, report $|\epsilon_a|$ and the certified digits each step, and comment on the convergence rate.

Solution. $f'(x) = 3x^2 - 1$.

$$x_1 = 1.5 - \frac{1.5^3 - 1.5 - 2}{3(1.5)^2 - 1} = 1.5 - \frac{-0.125}{5.75} = 1.52173913$$

k	x_k	$ \epsilon_a \%$	digits m
1	1.52173913	1.428571	1
2	1.52137981	0.023618	3
3	1.52137971	0.000007	6

Certified digits go $1 \rightarrow 3 \rightarrow 6$: roughly **doubling each step**, the signature of quadratic convergence. Compare with P4, where bisection bought a single digit in five iterations.

P7. Prove that Newton–Raphson on $f(x) = x^2$ (double root at 0) converges only linearly, and give the exact rate.

Solution.

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} = x_k - \frac{x_k^2}{2x_k} = x_k - \frac{x_k}{2} = \frac{x_k}{2}.$$

Hence $e_{k+1} = \frac{1}{2}e_k$ exactly: convergence is **linear with rate** $1/2$ — identical to bisection, and nowhere near quadratic. From $x_0 = 1$ the iterates are 0.5, 0.25, 0.125, 0.0625, ...

This matches the general rule $e_{k+1} \approx (1 - \frac{1}{k}) e_k$ for multiplicity k : here $k = 2$ gives rate $1 - \frac{1}{2} = \frac{1}{2}$ ✓. The modified step $x - 2\frac{f}{f'} = x_k - x_k = 0$ lands on the root in a single iteration.

P8. Verify Newton's quadratic error law numerically for $f(x) = x^2 - 2$ from $x_0 = 1$.

Solution. The law predicts $e_{k+1} \approx Ce_k^2$ with

$$C = \left| \frac{f''(x^*)}{2f'(x^*)} \right| = \frac{2}{2 \cdot 2\sqrt{2}} = \frac{1}{2\sqrt{2}} = 0.353553.$$

The iteration is the Babylonian one, $x_{k+1} = \frac{1}{2}(x_k + 2/x_k)$:

k	x_k	actual e_k	predicted $C e_{k-1}^2$	match?
1	1.500000000	8.579×10^{-2}	6.066×10^{-2}	rough (far from root)
2	1.416666667	2.453×10^{-3}	2.602×10^{-3}	good
3	1.414215686	2.124×10^{-6}	2.128×10^{-6}	excellent
4	1.414213562	1.595×10^{-12}	1.595×10^{-12}	exact to 4 s.f.

The prediction becomes essentially exact once x_k is close enough for the Taylor argument to hold. Note the error exponents: $10^{-2} \rightarrow 10^{-3} \rightarrow 10^{-6} \rightarrow 10^{-12}$ — **doubling the correct digits every step**.

Exam Traps

- **“Which method is fastest?” has no universal answer.** Newton is fastest *when it converges*; on $\ln x$ over $[10^{-4}, 10^4]$ false position (126 iterations) lost badly to bisection (34).
- **Bisection iteration count ignores f .** Any option that involves the function is wrong by construction.
- **Sign change is sufficient, not necessary.** x^2 has a root with no sign change; $1/x$ has a sign change with no root. Both appear as MCQ stems.
- **Quadratic $\neq$ “two iterations” and $\neq$ “error halves.”** It means the error is *squared*.
- **A double root silently kills Newton's speed.** If the stem says “multiplicity 2” or gives $f = (x - r)^2 g(x)$, the answer is linear, rate $1/2$.
- **$f'(x_0) = 0$ is fatal, and near-zero is almost as bad.** The floating-ball bracket ends 0 and 0.11 both have $f' = 0$.
- **In false position, do not use bracket width to stop.** A frozen endpoint keeps it large forever; use $|\epsilon_a|$ on successive x_r .
- **The reported root must be checked against the interval.** Newton can jump entirely out of your region of interest (root jumping on $\sin x$, converging to 0 instead of 2π).
- **Counting iterations vs counting function evaluations.** Newton needs f and f' each step; a question about “work” may not have the same answer as one about “iterations.”

11 Linear Systems & Eigenvalues — Problem Bank

11.1 Systems of Equations — MCQs, Problems & Traps

MCQ / Rapid-Fire Bank

Q1. In naive Gauss elimination on a 5×5 system, how many multipliers m_{ik} are computed in total?

- (a) 5 (b) 10 (c) 20 (d) 25

Ans: (b) — $n(n-1)/2 = 5 \cdot 4/2 = 10$. (Equivalently: the count of strictly-lower entries of L .)

Q2. After forward elimination with **three** row swaps, the diagonal of U multiplies to 60. Then $\det A$ is

- (a) 60 (b) -60 (c) 180 (d) 20

Ans: (b) — $(-1)^3(60) = -60$. An odd swap count flips the sign.

Q3. For a single right-hand side, the clock-cycle cost of LU decomposition compared with Gauss elimination is

- (a) half (b) exactly equal (c) twice (d) $n/4$ times smaller

Ans: (b) — both equal $(\frac{8}{3}n^3 + 12n^2 + \frac{4}{3}n)T$. LU's advantage appears only with multiple right-hand sides.

Q4. Computing A^{-1} for large n , Gauss elimination costs roughly how many times more than LU?

- (a) $4/n$ (b) $n/4$ (c) n^2 (d) the same

Ans: (b) — $\frac{\frac{8}{3}n^4}{\frac{32}{3}n^3} = \frac{n}{4}$; at $n = 10^4$ the measured ratio is 2501.

Q5. Forward substitution is cheaper than back substitution mainly because

- (a) it has fewer rows (b) L has unit diagonal, so it needs **no divisions**
(c) it runs top to bottom (d) L is sparser

Ans: (b) — and in the course's cost model a division costs 16 cycles against 4 for a multiply.

Q6. Partial pivoting selects the pivot by scanning

- (a) the whole remaining submatrix (b) the current column, rows k to n
(c) the current row (d) the diagonal only

Ans: (b) — scanning the whole submatrix would be *complete* pivoting.

Q7. Naive Gauss elimination on the round-off benchmark produced a 50% error at 5 significant digits. The cure was

- (a) more significant digits (b) a single row swap (c) scaling the RHS (d) switching to Gauss-Jordan

Ans: (b) — pivoting replaced the multiplier -2750 with -0.00036 and returned the exact answer at the *same* 5 digits.

Q8. Which statement is **FALSE**?

- (a) $\det L = 1$ for the unit-lower-triangular L of an LU factorization
(b) Gauss-Jordan requires no back substitution
(c) Gauss-Jordan performs less arithmetic than Gauss elimination
(d) A zero pivot can appear after an elimination step even if none existed initially

Ans: (c) — Gauss-Jordan does *more* work ($\sim n^3$ vs $\sim \frac{2}{3}n^3$); eliminating above the pivot as well is not free.

Q9. Increasing the working precision from 5 to 6 significant digits will fix

- (a) a structurally zero pivot (b) round-off amplification only (c) both (d) neither

Ans: (b) — only a row swap fixes a zero pivot; division by zero is not a precision issue.

Practice Problems (with full solutions)

P1. Solve by Gauss elimination **with partial pivoting**, showing every swap and multiplier, and

report $\det A$ from the elimination:

$$\begin{bmatrix} 1 & 3 & 2 \\ 4 & 1 & -1 \\ 2 & -2 & 5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 13 \\ 3 \\ 13 \end{bmatrix}$$

Solution. *Column 1:* candidates $|1|, |4|, |2|$ — pivot is 4, so **swap** $R_1 \leftrightarrow R_2$:

$$\begin{bmatrix} 4 & 1 & -1 & 3 \\ 1 & 3 & 2 & 13 \\ 2 & -2 & 5 & 13 \end{bmatrix}$$

Multipliers $m_{21} = 1/4 = 0.25$, $m_{31} = 2/4 = 0.5$:

$$\begin{bmatrix} 4 & 1 & -1 & 3 \\ 0 & 2.75 & 2.25 & 12.25 \\ 0 & -2.5 & 5.5 & 11.5 \end{bmatrix}$$

Column 2: candidates $|2.75|, |-2.5|$ — 2.75 already the largest, **no swap**. $m_{32} = -2.5/2.75 = -0.909091$:

$$\begin{bmatrix} 4 & 1 & -1 & 3 \\ 0 & 2.75 & 2.25 & 12.25 \\ 0 & 0 & 7.545455 & 22.636364 \end{bmatrix}$$

Back substitution:

$$x_3 = \frac{22.636364}{7.545455} = 3, \quad x_2 = \frac{12.25 - 2.25(3)}{2.75} = \frac{5.5}{2.75} = 2, \quad x_1 = \frac{3 - 1(2) + 1(3)}{4} = \frac{4}{4} = 1$$

$$(x_1, x_2, x_3) = (\mathbf{1}, \mathbf{2}, \mathbf{3})$$

Determinant: one swap (odd), so

$$\det A = (-1)^1 (4)(2.75)(7.545455) = -83.$$

P2. For $A = \begin{bmatrix} 4 & 3 \\ 6 & 3 \end{bmatrix}$, find L and U , then solve for the *two* right-hand sides $C_1 = (10, 12)^\top$ and $C_2 = (7, 9)^\top$. Comment on what was reused.

Solution. $m_{21} = 6/4 = 1.5$, and $3 - 1.5(3) = -1.5$, so

$$L = \begin{bmatrix} 1 & 0 \\ 1.5 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 4 & 3 \\ 0 & -1.5 \end{bmatrix} \quad (LU = A \checkmark)$$

RHS 1: forward $z_1 = 10$, $z_2 = 12 - 1.5(10) = -3$. Back: $x_2 = -3 / -1.5 = 2$, $x_1 = (10 - 3(2))/4 = 1$. So $X_1 = (1, 2)^\top$.

RHS 2: forward $z_1 = 7$, $z_2 = 9 - 1.5(7) = -1.5$. Back: $x_2 = -1.5 / -1.5 = 1$, $x_1 = (7 - 3(1))/4 = 1$. So $X_2 = (1, 1)^\top$.

What was reused: the entire factorization. L and U never depend on the right-hand side, so the second solve costs only one forward and one back substitution ($O(n^2)$) instead of a fresh $O(n^3)$ elimination. That is the whole argument for LU.

P3. Using the course's clock-cycle model, compare LU and Gauss elimination for $n = 5$ with (i) one right-hand side and (ii) five right-hand sides.

Solution. Building blocks at $n = 5$:

$$C_T|_{DE} = \frac{8}{3}(125) + 4(25) - \frac{20}{3}(5) = 400T, \quad C_T|_{FS} = 4(25) - 4(5) = 80T$$

$$C_T|_{BS} = 4(25) + 12(5) = 160T, \quad C_T|_{FE} = \frac{8}{3}(125) + 8(25) - \frac{32}{3}(5) = 480T$$

(i) *One RHS*:

$$C_T|_{LU} = 400 + 80 + 160 = \mathbf{640T}, \quad C_T|_{GE} = 480 + 160 = \mathbf{640T}$$

Identical, exactly as the theory says.

(ii) *Five RHS*:

$$C_T|_{LU} = 400 + 5(80 + 160) = 400 + 1200 = \mathbf{1600T}$$

$$C_T|_{GE} = 5(480 + 160) = \mathbf{3200T}$$

LU is exactly **twice** as fast here. The asymptotic estimate $n/4 = 1.25$ understates it because $n = 5$ is far from the large- n regime.

11.2 Eigenvalues — MCQs, Problems & Traps

MCQ / Rapid-Fire Bank

Q10. A 2×2 matrix has $\text{tr } A = 9$ and $\det A = 14$. Its eigenvalues are

- (a) 2, 7 (b) 3, 6 (c) 1, 14 (d) 4, 5

Ans: (a) — $\lambda^2 - 9\lambda + 14 = 0 \Rightarrow (\lambda - 7)(\lambda - 2)$.

Q11. If $\lambda = 3$ is an eigenvalue of A , an eigenvalue of $A^2 + 2A - I$ is

- (a) 9 (b) 14 (c) 8 (d) 5

Ans: (b) — for any polynomial, $p(\lambda) = 9 + 6 - 1 = 14$.

Q12. If A has eigenvalue λ with eigenvector $\mathbf{v}$, then $A + 5I$ has

- (a) eigenvalue 5λ , eigenvector $\mathbf{v}$ (b) eigenvalue $\lambda + 5$, eigenvector $\mathbf{v}$
 (c) eigenvalue $\lambda + 5$, eigenvector $\mathbf{v} + 5$ (d) eigenvalue λ , eigenvector $5\mathbf{v}$

Ans: (b) — shifting moves every eigenvalue and leaves every eigenvector alone.

Q13. A matrix has eigenvalues -7 and 5 . The power method converges to

- (a) 5 (b) -7 (c) 7 (d) it fails

Ans: (b) — it finds the largest *magnitude*, sign included.

Q14. The power method is applied to a matrix with eigenvalues 8 and -8 . It will

- (a) converge to 8 (b) converge to -8 (c) fail to settle on one direction (d) converge to 0

Ans: (c) — $|\lambda_2/\lambda_1| = 1$, so nothing decays; the iterate oscillates.

Q15. For eigenvalues 10 and 4 , roughly how many power-method iterations buy one extra decimal digit?

- (a) 1.0 (b) 2.5 (c) 5.0 (d) 10

Ans: (b) — $1/\log_{10}(10/4) = 1/0.39794 = 2.51$.

Q16. The inverse power method applied to A converges to a normalizing factor 0.25 . Then $\lambda_{\min}(A)$ is

- (a) 0.25 (b) 4 (c) -4 (d) 0.5

Ans: (b) — the method returns $1/\lambda_{\min}$, so invert: $\lambda_{\min} = 4$.

Q17. Hotelling deflation $A_2 = A - \lambda_1 \hat{\mathbf{v}}_1 \hat{\mathbf{v}}_1^\top$ leaves $\mathbf{v}_2$ untouched because

- (a) λ_2 is small (b) $\hat{\mathbf{v}}_1$ and $\hat{\mathbf{v}}_2$ are orthogonal (symmetric A)
 (c) A_2 is diagonal (d) $\hat{\mathbf{v}}_1$ is a unit vector

Ans: (b) — the projection $\hat{\mathbf{v}}_1(\hat{\mathbf{v}}_1^\top \mathbf{v}_2)$ vanishes.

Q18. In the 3×3 power-method example the reported $|\epsilon_a|$ jumped to 214% at iteration 4. This indicates

- (a) divergence (b) a coding error (c) a sign change in the normalizing factor (d) $|\lambda_1| = |\lambda_2|$

Ans: (c) — the sequence was converging normally toward 6.07048; the sign flip merely distorts

the relative-error formula.

Q19. Which statement is **FALSE**?

- (a) $\text{tr } A = \sum \lambda_i$ (b) $\det A = \prod \lambda_i$
 (c) $\det(A + cI) = \det A + c$ (d) A and $A^\top$ have the same eigenvalues

Ans: (c) — you must shift each eigenvalue and re-multiply: $\det(A + cI) = \prod(\lambda_i + c)$.

Practice Problems (with full solutions)

P4. For $A = \begin{bmatrix} 6 & 2 \\ 2 & 3 \end{bmatrix}$: (i) find both eigenvalues using only trace and determinant; (ii) run 4 power-method iterations from $\mathbf{x}_0 = [1, 0]^\top$, reporting $|\epsilon_a|$; (iii) state the theoretical convergence ratio and check it against the observed error decay.

Solution. (i) $\text{tr } A = 9$, $\det A = 6(3) - 2(2) = 14$, so

$$\lambda^2 - 9\lambda + 14 = 0 \implies \lambda = 7, 2.$$

No characteristic-polynomial expansion needed.

(ii) Normalizing by the largest-magnitude entry each step:

Iter	$A\mathbf{x}_k$	λ estimate	$\mathbf{x}_{k+1}$	$ \epsilon_a \%$
1	$[6, 2]^\top$	6.000000	$[1, 0.333333]^\top$	—
2	$[6.666667, 3]^\top$	6.666667	$[1, 0.45]^\top$	10.0000
3	$[6.9, 3.35]^\top$	6.900000	$[1, 0.485507]^\top$	3.3816
4	$[6.971014, 3.456522]^\top$	6.971014	$[1, 0.495842]^\top$	1.0187

Converging to $\lambda_1 = 7$, with the eigenvector tending to $[1, 0.5]^\top$.

(iii) Ratio $= |\lambda_2/\lambda_1| = 2/7 = 0.2857$. Check the actual errors: $7 - 6 = 1$, $7 - 6.666667 = 0.333333$, $7 - 6.9 = 0.1$, $7 - 6.971014 = 0.028986$. Successive ratios: 0.3333, 0.30, 0.2899 — converging neatly onto the predicted 0.2857. ✓

P5. Continue P4: deflate A to find the second eigenvalue.

Solution. The dominant eigenvector is $\mathbf{v}_1 = [2, 1]^\top$ (equivalently $[1, 0.5]^\top$). Normalize:

$$\hat{\mathbf{v}}_1 = \frac{1}{\sqrt{5}} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.894427 \\ 0.447214 \end{bmatrix}, \quad \hat{\mathbf{v}}_1 \hat{\mathbf{v}}_1^\top = \frac{1}{5} \begin{bmatrix} 4 & 2 \\ 2 & 1 \end{bmatrix}$$

$$A_2 = A - 7 \cdot \frac{1}{5} \begin{bmatrix} 4 & 2 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 6 & 2 \\ 2 & 3 \end{bmatrix} - \begin{bmatrix} 5.6 & 2.8 \\ 2.8 & 1.4 \end{bmatrix} = \begin{bmatrix} 0.4 & -0.8 \\ -0.8 & 1.6 \end{bmatrix}$$

Check via trace/determinant: $\text{tr } A_2 = 2.0$, $\det A_2 = 0.4(1.6) - (-0.8)^2 = 0.64 - 0.64 = 0$. So the eigenvalues satisfy $\lambda^2 - 2\lambda = 0$, giving $\{0, 2\}$ — $\lambda_1 = 7$ has been muted to zero and $\lambda_2 = 2$ survives untouched, exactly as intended. Running the power method on A_2 now returns 2.

P6. A is a 4×4 matrix with eigenvalues 10, 6, 4, 2. (i) What does the power method return? (ii) What does the inverse power method return, and what normalizing factor does it converge to? (iii) If you wanted the eigenvalue nearest to 5, what would you do?

Solution. (i) The power method returns the dominant eigenvalue $\lambda_1 = 10$.

(ii) A^{-1} has eigenvalues $1/10 = 0.1$, $1/6 = 0.1667$, $1/4 = 0.25$, $1/2 = 0.5$. The dominant one is 0.5, so the normalizing factor converges to **0.5** and $\lambda_{\min} = 1/0.5 = 2$. Note the ranking is fully reversed: what was largest in A is smallest in A^{-1} .

(iii) Use a **shift**. $A - 5I$ has eigenvalues 5, 1, -1, -3. The one nearest 5 in the original problem ($\lambda = 6$) has become the smallest in magnitude (1), so apply the *inverse* power method to $A - 5I$: it converges to $1/1 = 1$, and adding the shift back gives $\lambda = 1 + 5 = 6$. (Applying the plain power method to $A - 5I$ would instead find the eigenvalue *farthest* from 5, namely $\lambda - 5 = 5$, i.e. $\lambda = 10$.)

Exam Traps

- **“Largest eigenvalue” means largest *magnitude*.** With -7 and 5 the power method returns -7 , not 5 .
- $\det(A + cI) \neq \det A + c$ and $\operatorname{tr}(AB) \neq \operatorname{tr} A \operatorname{tr} B$. Only the eigenvalue *map* is simple.
- **Shifting and powering never change eigenvectors.** Any option modifying $\mathbf{v}$ under $A + cI$, A^k , or cA is wrong.
- **Inverse power returns the reciprocal.** Forgetting the final inversion turns $\lambda_{\min} = 4$ into the answer 0.25 — which will be one of the distractors.
- **A spiking $|\epsilon_a|$ may be a sign flip,** not divergence.
- **The normalizing factor is λ ;** the normalized vector’s largest entry is always 1 and carries no information.
- **Deflation assumes symmetry.** Without orthogonal eigenvectors the subtraction contaminates the other eigenpairs.
- **A “bad” start vector ($c_1 = 0$) is not a failure of the matrix.** The method finds λ_2 instead — in exact arithmetic it can never recover.
- **Even $\det L = 1$ has a trap:** it means $\det A = \det U$, so do *not* also multiply by the product of the multipliers.

12 Optimization — Problem Bank**12.1 Optimization — MCQs****MCQ / Rapid-Fire Bank**

Q1. Golden Section Search starts on $[0, 6]$. The bracket width after 3 iterations is

- (a) 0.75 (b) 1.416408 (c) 2.291796 (d) 3.708204

Ans: (b) — $L_3 = 6R^3 = 6(0.236068) = 1.416408$. *This is the past-exam pattern: one multiplication, not three iterations.*

Q2. Starting from $[0, 4]$, the width after 3 iterations is

- (a) 0.944272 (b) 0.5 (c) 1.527864 (d) 2.472136

Ans: (a) — $4R^3 = 0.944272$. Option (c) is $4R^2/\dots$ -style bait and (d) is $4R$.

Q3. R^2 equals

- (a) $1 + R$ (b) $1 - R$ (c) $2R$ (d) $R/2$

Ans: (b) — from $R^2 + R - 1 = 0$. Numerically 0.381966 .

Q4. $1/R$ equals

- (a) $R - 1$ (b) $1 - R$ (c) $1 + R = 1.618034$ (d) $2R$

Ans: (c)

Q5. Beginning with a unit interval, the number of GSS iterations needed to shrink it below 0.01 is

- (a) 7 (b) 8 (c) 10 (d) 12

Ans: (c) — $n \geq \ln(0.01)/\ln(0.618034) = 9.57 \Rightarrow 10$.

Q6. After 6 GSS iterations, the number of *interior* function evaluations performed is

- (a) 6 (b) 7 (c) 12 (d) 13

Ans: (b) — 2 on the first iteration, then 1 each: $2 + 5 = 7 = n + 1$.

Q7. Per iteration, Golden Section Search discards what fraction of the bracket?

- (a) 50% (b) 61.8% (c) 38.2% (d) 23.6%

Ans: (c) — it keeps $R = 61.8\%$ and discards $1 - R = 38.2\%$. Note this is *less* than bisection’s 50%.

Q8. GSS reports the optimum as the bracket midpoint. After n iterations the worst-case error is

- (a) $R^n L_0$ (b) $R^n L_0/2$ (c) $2R^n L_0$ (d) $R^n/2$

Ans: (b)

Q9. Gradient descent on $J(x) = 5(x - 3)^2$ converges for exactly which learning rates?

- (a) $0 < \alpha < 0.1$ (b) $0 < \alpha < 0.2$ (c) $0 < \alpha < 5$ (d) all $\alpha > 0$

Ans: (b) — here $a = 5$, and the condition is $0 < \alpha < 1/a = 0.2$.

Q10. For that same J , which α reaches the minimum in a **single** step?

- (a) 0.05 (b) 0.1 (c) 0.2 (d) no such α exists

Ans: (b) — $\alpha = 1/(2a) = 0.1$ makes the factor $1 - 2a\alpha$ exactly zero.

Q11. Still on $J = 5(x - 3)^2$, taking $\alpha = 0.2$ exactly produces

- (a) convergence (b) divergence to infinity (c) an endless 2-cycle (d) convergence to a saddle

Ans: (c) — the factor is -1 , so from $x_0 = 0$ the iterates cycle $6, 0, 6, 0, \dots$ forever. The stability boundary is not “slow convergence”.

Q12. Newton’s method for optimization applied to any quadratic converges in

- (a) exactly one step, from any start (b) two steps (c) a number of steps depending on x_0 (d) never

Ans: (a) — the second-order Taylor model is exact for a quadratic.

Q13. Newton’s method for optimization is applied to $f(x) = -x^2 + 4x$ from $x_0 = 5$. It converges to $x = 2$, which is

- (a) a minimum (b) a maximum (c) a saddle (d) not stationary

Ans: (b) — $f'' = -2 < 0$. Newton solves $f' = 0$ and never checks curvature.

Q14. At a stationary point, $f_{xx} = 0$ and $\det H = -16$. The point is a

- (a) minimum (b) maximum (c) saddle (d) inconclusive

Ans: (c) — $\det H < 0$ means eigenvalues of opposite sign, whatever f_{xx} is.

Q15. At a stationary point $\det H = 0$. Then the point is

- (a) definitely a minimum (b) definitely a saddle (c) inconclusive by this test (d) definitely a maximum

Ans: (c)

Q16. Which requires **no** derivative information at all?

- (a) Gradient descent (b) Newton’s method (c) Golden Section Search (d) all need derivatives

Ans: (c) — GSS only compares function values, which is exactly why it survives non-differentiable objectives.

Q17. For a problem with $n = 10^6$ parameters, the sensible choice is

- (a) Newton, for quadratic convergence (b) gradient descent, because the Hessian is 10^{12} entries
(c) Golden Section Search (d) Lagrange multipliers

Ans: (b)

Q18. In the bordered Hessian, the $(1, 1)$ entry is 0 because

- (a) the constraint is linear (b) $\partial^2 \Lambda / \partial \lambda^2 = 0$, since λ appears only linearly
(c) the objective is quadratic (d) it is a convention

Ans: (b)

Q19. For a 2-variable, 1-constraint problem, $\det \bar{H} < 0$ indicates a

- (a) maximum (b) minimum (c) saddle (d) inflection

Ans: (b) — verified on $J = x_1^2 + x_2^2$ with $C = x_1 + x_2 + 3$: $\det \bar{H} = -4$, and the point $(-1.5, -1.5)$ is indeed the minimum.

Q20. Which statement is **FALSE**?

- (a) GSS requires unimodality (b) Newton for optimization needs no learning rate
(c) A zero gradient guarantees a minimum (d) λ carries sensitivity information

Ans: (c) — zero gradient is necessary but never sufficient; x^3 at 0 is the standard counterexample.

12.2 Optimization — Problems

Practice Problems (with full solutions)

P1. Maximize $f(x) = 8x - x^2$ on $[0, 6]$ by Golden Section Search. (i) Perform 3 iterations showing x_1, x_2, f_1, f_2 and the surviving bracket. (ii) Verify each width against $L_n = R^n L_0$. (iii) Report the optimum and its uncertainty. (iv) State the width after 8 iterations without iterating.

Solution. $R = 0.618034$, $L_0 = 6$. (True maximum at $x = 4$.)

Iteration 1: $d = R(6) = 3.708204$; $x_1 = 6 - 3.708204 = 2.291796$, $x_2 = 0 + 3.708204 = 3.708204$. $f_1 = 13.082039$, $f_2 = 15.914855$. Since $f_1 < f_2$, the maximum lies right: $x_L \leftarrow x_1$. Bracket $[2.291796, 6]$, width 3.708204; check $6R = 3.708204$ ✓

Iteration 2: $d = R(3.708204) = 2.291796$; $x_1 = 3.708204$ (reused), $x_2 = 4.583592$. $f_1 = 15.914855$, $f_2 = 15.659420$. Now $f_1 > f_2$, so the maximum is left: $x_u \leftarrow x_2$. Bracket $[2.291796, 4.583592]$, width 2.291796; check $6R^2 = 2.291796$ ✓

Iteration 3: $d = R(2.291796) = 1.416408$; $x_1 = 3.167184$, $x_2 = 3.708204$ (reused). $f_1 = 15.306418$, $f_2 = 15.914855$. $f_1 < f_2$, maximum right: $x_L \leftarrow x_1$. Bracket $[3.167184, 4.583592]$, width 1.416408; check $6R^3 = 1.416408$ ✓

(iii) $x_{opt} = \frac{3.167184 + 4.583592}{2} = 3.875388$, uncertainty $\pm L_3/2 = \pm 0.708204$. The true optimum 4 lies comfortably inside.

(iv) $L_8 = 6R^8 = 6(0.021286) = \mathbf{0.127717}$ — no iteration needed.

P2. A GSS run begins on $[1, 9]$. (i) How many iterations guarantee a bracket below 0.05? (ii) What is the actual width then? (iii) How many interior function evaluations does that cost?

Solution. $L_0 = 9 - 1 = 8$.

(i) $n \geq \frac{\ln(0.05/8)}{\ln 0.618034} = \frac{\ln 0.00625}{-0.481212} = 10.55 \Rightarrow n = \mathbf{11}$.

(ii) $L_{11} = 8R^{11} = 8(0.005025) = \mathbf{0.040200} < 0.05$ ✓ (and $L_{10} = 0.065$, which fails — so 11 really is the minimum).

(iii) Interior evaluations = $n + 1 = \mathbf{12}$ (plus the 2 endpoint evaluations if the question counts them, giving 14).

P3. For $J(x) = 5(x - 3)^2$ with $x_0 = 0$: (i) derive the error recursion; (ii) give the full range of α for convergence; (iii) tabulate the first 5 iterates for $\alpha = 0.05, 0.1, 0.15, 0.2, 0.25$ and classify each regime.

Solution. (i) $J'(x) = 10(x - 3)$, so with $e_k = x_k - 3$:

$$x_{k+1} = x_k - 10\alpha(x_k - 3) \implies e_{k+1} = (1 - 10\alpha)e_k.$$

Here $a = 5$, matching the general $e_{k+1} = (1 - 2a\alpha)e_k$.

(ii) $|1 - 10\alpha| < 1 \Rightarrow 0 < \alpha < 0.2$, i.e. $0 < \alpha < 1/a$.

(iii)

α	$q = 1 - 10\alpha$	Iterates $x_1 \dots x_5$	Regime
0.05	+0.50	1.5, 2.25, 2.625, 2.8125, 2.90625	monotone convergence
0.10	0	3, 3, 3, 3, 3	one-step ($\alpha = 1/(2a)$)
0.15	-0.50	4.5, 2.25, 3.375, 2.8125, 3.09375	oscillating convergence
0.20	-1.00	6, 0, 6, 0, 6	2-cycle, never converges
0.25	-1.50	7.5, -3.75, 13.125, -12.1875, 25.78	divergence

Note how $\alpha = 0.2$ — exactly the stability boundary — neither converges nor diverges.

P4. (i) Show Newton's method for optimization reaches the minimum of $f(x) = 3(x - 2)^2$ in one step from $x_0 = 10$. (ii) Apply the same method to $f(x) = -x^2 + 4x$ from $x_0 = 5$ and classify what it finds.

Solution. (i) $f'(x) = 6(x - 2)$, $f''(x) = 6$:

$$x_1 = 10 - \frac{6(10 - 2)}{6} = 10 - 8 = 2 = x^*.$$

One step, and the starting point cancelled out entirely — as it must, since Newton minimizes the quadratic model and here the model *is* f .

(ii) $f'(x) = -2x + 4$, $f''(x) = -2$:

$$x_1 = 5 - \frac{-2(5) + 4}{-2} = 5 - \frac{-6}{-2} = 5 - 3 = 2.$$

Also one step. But $f''(2) = -2 < 0$, so $x = 2$ is a **maximum**, not a minimum. Newton solved $f' = 0$ faithfully and gave the wrong *kind* of point — exactly limitation (2) in the notes. The remedy is to check the sign of f'' (or the definiteness of H) before accepting the answer.

P5. Find and classify all stationary points of $f(x, y) = x^4 + y^4 - 4xy$.

Solution. $f_x = 4x^3 - 4y = 0 \Rightarrow y = x^3$; $f_y = 4y^3 - 4x = 0 \Rightarrow x = y^3$. Substituting: $x = x^9 \Rightarrow x(x^8 - 1) = 0$, giving real solutions $x = 0, 1, -1$ and hence the stationary points

$$(0, 0), \quad (1, 1), \quad (-1, -1).$$

Hessian: $H = \begin{bmatrix} 12x^2 & -4 \\ -4 & 12y^2 \end{bmatrix}.$

Point	H	$\det H$	Classification
$(0, 0)$	$\begin{bmatrix} 0 & -4 \\ -4 & 0 \end{bmatrix}$	$-16 < 0$	saddle
$(1, 1)$	$\begin{bmatrix} 12 & -4 \\ -4 & 12 \end{bmatrix}$	$128 > 0, f_{xx} = 12 > 0$	minimum
$(-1, -1)$	$\begin{bmatrix} 12 & -4 \\ -4 & 12 \end{bmatrix}$	$128 > 0, f_{xx} = 12 > 0$	minimum

Note $(0, 0)$ has $f_{xx} = 0$ yet is decisively a saddle — $\det H < 0$ settles it on its own.

P6. Minimize $J = 2x_1^2 + 3x_2^2$ subject to $x_1 + x_2 = 10$. (i) Solve with Lagrange multipliers. (ii) Classify the point with the bordered Hessian. (iii) Use λ to predict the change in J^* if the constraint becomes $x_1 + x_2 = 10.5$, then verify by re-solving.

Solution. (i) Write the constraint as $C = x_1 + x_2 - 10 = 0$ and form $\Lambda = 2x_1^2 + 3x_2^2 + \lambda(x_1 + x_2 - 10)$:

$$\frac{\partial \Lambda}{\partial x_1} = 4x_1 + \lambda = 0, \quad \frac{\partial \Lambda}{\partial x_2} = 6x_2 + \lambda = 0, \quad x_1 + x_2 = 10.$$

From the first two, $x_1 = -\lambda/4$ and $x_2 = -\lambda/6$. Substituting: $-\lambda\left(\frac{1}{4} + \frac{1}{6}\right) = 10 \Rightarrow -\lambda\frac{5}{12} = 10 \Rightarrow \lambda = -24$.

$$x_1 = 6, \quad x_2 = 4, \quad \lambda = -24, \quad J^* = 2(36) + 3(16) = 120.$$

Gradient check: $\nabla J = (4x_1, 6x_2) = (24, 24)$ and $\nabla C = (1, 1)$ — parallel ✓

(ii) $\Lambda_{x_1 x_1} = 4$, $\Lambda_{x_2 x_2} = 6$, $\Lambda_{x_1 x_2} = 0$, $C_{x_1} = C_{x_2} = 1$:

$$\bar{H} = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 4 & 0 \\ 1 & 0 & 6 \end{bmatrix}, \quad \det \bar{H} = -10 < 0 \Rightarrow \text{minimum } \checkmark$$

(iii) With the constraint written $x_1 + x_2 = b$, the sensitivity rule is $dJ^*/db = -\lambda = 24$. For $\Delta b = 0.5$ the prediction is

$$\Delta J^* \approx 24(0.5) = 12 \implies J^* \approx 132.$$

Verification: solving in general gives $x_1 = 3b/5$, $x_2 = 2b/5$ and $J^* = \frac{6}{5}b^2$. At $b = 10.5$: $J^* = 1.2(110.25) = 132.3$. The prediction 132 is right to within the second-order term — and is exact in the limit of small Δb , since $dJ^*/db = \frac{12}{5}b = 24$ at $b = 10$. ✓

Exam Traps

- $R = 0.618$ **shrinks**; 1.618 **grows**. Both will be offered, and $1/R = 1 + R$ makes them easy to confuse.
- $L_n = R^n L_0$ **needs the original width**. For $[1, 9]$ that is 8, not 9.
- “Iterations” $\neq$ “function evaluations” (n vs $n + 1$ interior, vs $n + 3$ if endpoints count).
- GSS error is $L_n/2$, because the answer is the midpoint.
- GSS removes less per iteration than bisection. “Golden” is about reusing evaluations, not about a faster width reduction.
- A bigger learning rate is not monotonically better. Past $1/(2a)$ you oscillate; at $1/a$ you cycle forever; past that you diverge.
- The stability boundary $\alpha = 1/a$ does not converge slowly — it does not converge at all.
- Newton reaching the answer in one step says nothing about min vs max. Check f'' or H .
- $\det H < 0$ is a saddle even when $f_{xx} = 0$; $\det H = 0$ is inconclusive, not a saddle.
- The sign of λ depends on how you wrote the constraint. With $g(\mathbf{x}) = b$ and $\Lambda = J + \lambda(g - b)$, the sensitivity is $dJ^*/db = -\lambda$. Safest under pressure: re-solve with the perturbed constant and compare.
- The parallel-gradient condition only locates a point. Classification needs $\bar{H}$ — and for 2 variables with 1 constraint, $\det \bar{H} < 0$ means *minimum* (the reverse of the unconstrained intuition).

13 Regression & Interpolation — Problem Bank**13.1 Regression — MCQs, Problems & Traps****MCQ / Rapid-Fire Bank**

Q1. A least-squares line with an intercept is fitted to any data set. It must pass through
 (a) the origin (b) the first and last points (c) $(\bar{x}, \bar{y})$ (d) none of these necessarily

Ans: (c) — immediate from $b = \bar{y} - m\bar{x}$. This single fact answers many “find the missing value” questions.

Q2. For a least-squares fit with an intercept, the sum of the residuals $\sum(y_i - \hat{y}_i)$ equals
 (a) SSE (b) 0 (c) $n\bar{y}$ (d) depends on the data

Ans: (b) — it is the normal equation $\partial SSE / \partial b = 0$.

Q3. If $r = -0.9$, the percentage of variance explained is

(a) -90% (b) 90% (c) 81% (d) -81%

Ans: (c) — $r^2 = 0.81$; r^2 is never negative, and the sign of r only gives direction.

Q4. $SSE = 1.8$ and $SST = 18$. Then r^2 is

(a) 0.1 (b) 0.9 (c) 10 (d) 0.18

Ans: (b) — $r^2 = 1 - SSE/SST = 1 - 0.1 = 0.9$.

Q5. Every x value in a data set is doubled (the y 's unchanged). Then

(a) both m and r halve (b) m halves, r is unchanged

(c) m is unchanged, r halves (d) both are unchanged

Ans: (b) — r is dimensionless and invariant under linear rescaling; the slope carries units and must halve.

Q6. A strong correlation $r = 0.95$ between ice-cream sales and drowning deaths implies

- (a) ice cream causes drowning (b) drowning causes ice-cream sales
 (c) nothing causal; a hidden third factor may drive both (d) the data are wrong

Ans: (c) — correlation is not causation; here the lurking variable is summer temperature.

Q7. In multiple linear regression, the residuals are found to fan out as $\hat{y}$ increases. The violated assumption is

- (a) Linearity (b) Independence (c) Homoscedasticity (d) Normality

Ans: (c) — homoscedasticity demands *constant* error variance.

Q8. Two input features of an MLR model are found to satisfy $x_3 = 2x_1 + 1$. The violated assumption is

- (a) Normality (b) No multicollinearity (c) Homoscedasticity (d) Linearity

Ans: (b) — the features are exactly linearly dependent, so the weights are not identifiable.

Q9. Replacing the MLR cost $J = \frac{1}{2m} \sum (\hat{y}_i - y_i)^2$ by $J = \sum (\hat{y}_i - y_i)^2$ would

- (a) change the optimal weights (b) leave the optimal weights unchanged, but rescale the gradients
 (c) make the problem non-convex (d) make it unsolvable

Ans: (b) — a positive constant factor cannot move the argmin; it only rescales α 's effect.

Practice Problems (with full solutions)

P1. Fit a least-squares line to $(1, 2), (2, 5), (3, 5), (4, 8)$. Report m, b, r, r^2 . Then verify *four* structural properties: the centroid lies on the line, the residuals sum to zero, $r^2 = 1 - SSE/SST$, and $m = r s_y/s_x$.

Solution. Sums with $n = 4$: $\sum x = 10, \sum y = 20, \sum xy = 59, \sum x^2 = 30, \sum y^2 = 118$.

$$m = \frac{4(59) - 10(20)}{4(30) - 10^2} = \frac{236 - 200}{120 - 100} = \frac{36}{20} = 1.8$$

$$b = \frac{20 - 1.8(10)}{4} = \frac{2}{4} = 0.5 \quad \Rightarrow \quad \hat{y} = 1.8x + 0.5$$

$$r = \frac{36}{\sqrt{(20)(4(118) - 20^2)}} = \frac{36}{\sqrt{20 \times 72}} = \frac{36}{37.947} = 0.948683, \quad r^2 = 0.9$$

Check 1 — centroid: $\bar{x} = 2.5, \bar{y} = 5; \hat{y}(2.5) = 1.8(2.5) + 0.5 = 5 = \bar{y} \checkmark$

Check 2 — residuals: $\hat{y} = (2.3, 4.1, 5.9, 7.7)$, so $e = (-0.3, 0.9, -0.9, 0.3)$, summing to 0 $\checkmark$

Check 3 — variance decomposition: $SSE = 0.09 + 0.81 + 0.81 + 0.09 = 1.8$ and $SST = \sum (y_i - \bar{y})^2 = 9 + 0 + 0 + 9 = 18$, so

$$1 - \frac{SSE}{SST} = 1 - \frac{1.8}{18} = 0.9 = r^2 \checkmark$$

Check 4 — slope/correlation link: $s_x = 1.290994, s_y = 2.449490$, so

$$r \frac{s_y}{s_x} = 0.948683 \times \frac{2.449490}{1.290994} = 1.8 = m \checkmark$$

P2. A regression line through 6 points has $\bar{x} = 4$ and $\bar{y} = 11$, and its slope is $m = 2.5$. Find the intercept without touching the raw data. Then find $\sum y_i$.

Solution. The line passes through the centroid, so

$$b = \bar{y} - m\bar{x} = 11 - 2.5(4) = 1 \quad \Rightarrow \quad \hat{y} = 2.5x + 1.$$

And $\sum y_i = n\bar{y} = 6(11) = 66$. Neither step needed a single data point — exactly the shortcut MCQs are built on.

Exam Traps

- r^2 is **never negative** and is not “ r as a percentage.” $r = -0.9$ gives 81%, not -90% .
- r is **scale-invariant**, m is **not**. Rescaling x or y changes the slope but never the correlation.
- **Residuals summing to zero does not mean a good fit** — it holds for *every* least-squares line with an intercept, however bad.
- **A high r^2 does not validate the model**. All five MLR assumptions can be violated at $r^2 = 0.99$.
- **Correlation is not causation**, and the exam likes the ice-cream/drowning style stem.
- **Know which assumption each symptom breaks**: Poisson errors $\rightarrow$ Normality; fanning residuals $\rightarrow$ Homoscedasticity; dependent features $\rightarrow$ No multicollinearity; time-ordered carryover $\rightarrow$ Independence.
- **Regression does not pass through the data points** — that is interpolation. It passes through the *centroid*.
- **The $\frac{1}{2m}$ in the MLR cost is cosmetic**. It cannot change where the minimum is.

13.2 Interpolation — MCQs, Problems & Traps**MCQ / Rapid-Fire Bank**

Q10. Given 7 data points, the interpolating polynomial has degree

- (a) exactly 7 (b) exactly 6 (c) at most 6 (d) at most 7

Ans: (c) — $(n + 1)$ points give *at most* degree n ; collinear or otherwise degenerate data drop the degree.

Q11. Data are sampled from an exact quadratic. The 3rd divided difference of the table is

- (a) the leading coefficient (b) 0 (c) 2 (d) undefined

Ans: (b) — since $f[x_0..x_n] = f^{(n)}(\xi)/n!$ and the 3rd derivative of a quadratic is zero.

Q12. Data are sampled from $y = x^3 - 2x^2 + 3$. The 3rd divided difference equals

- (a) 0 (b) 1 (c) 3 (d) 6

Ans: (b) — it equals the leading coefficient: $f'''/3! = 6/6 = 1$. (True even for unequally spaced nodes.)

Q13. Lagrange and Newton divided-difference interpolation are applied to the same 4 points and both evaluated at $x = 3$. The two results

- (a) differ slightly (b) are identical (c) differ by the error term (d) agree only at the nodes

Ans: (b) — the interpolating polynomial through given points is *unique*; only the algebraic form differs.

Q14. The Lagrange basis polynomials satisfy $\sum_j L_j(x)$ equal to

- (a) 0 (b) 1 for all x (c) x (d) n

Ans: (b) — interpolating the constant function 1 reproduces it exactly.

Q15. Newton’s form is preferred over Lagrange’s mainly because

- (a) it is more accurate (b) adding a new point reuses all existing coefficients
(c) it needs fewer points (d) it avoids division

Ans: (b) — the recalculation-free property. Accuracy is identical (Q13).

Q16. A cubic spline is constructed through 10 data points. The number of unknown coefficients is

- (a) 36 (b) 40 (c) 27 (d) 30

Ans: (a) — 10 points give $n = 9$ intervals, so $4n = 36$.

Q17. For that cubic spline, how many equations come from the second-derivative matching conditions?

- (a) 9 (b) 8 (c) 18 (d) 2

Ans: (b) — $n - 1 = 8$ interior knots.

Q18. A linear spline through a data set is

- (a) continuous and differentiable everywhere (b) continuous but generally not differentiable at the knots
(c) discontinuous (d) twice differentiable

Ans: (b) — the joints are corners.

Q19. Runge's phenomenon says that as more *equally spaced* nodes are added to a global polynomial interpolant, the error

- (a) always decreases (b) grows, worst near the centre (c) grows, worst near the edges (d) stays constant

Ans: (c)

Q20. Which statement is **FALSE**?

- (a) A natural cubic spline sets $P'' = 0$ at both ends (b) A clamped spline specifies end slopes
(c) Interpolation passes through every data point (d) Higher spline order always reduces total error

Ans: (d) — higher order buys smoothness at the cost of computation and greater error risk; that trade-off is explicit in the notes.

Practice Problems (with full solutions)

P3. The function $y = x^3 - 2x^2 + 3$ is sampled at the **unequally spaced** nodes $x = 1, 2, 4, 5$. (i) Build the full divided-difference table. (ii) Write $P_3(x)$ in Newton form. (iii) Evaluate at $x = 3$ and compare with the exact value. (iv) Evaluate the same point by Lagrange interpolation and comment.

Solution. Values: $y(1) = 2$, $y(2) = 3$, $y(4) = 35$, $y(5) = 78$.

(i) First divided differences:

$$\frac{3-2}{2-1} = 1, \quad \frac{35-3}{4-2} = 16, \quad \frac{78-35}{5-4} = 43$$

Second:

$$\frac{16-1}{4-1} = 5, \quad \frac{43-16}{5-2} = 9$$

Third:

$$\frac{9-5}{5-1} = 1$$

x_i	y_i	1st	2nd	3rd
1	2			
		1		
2	3		5	
		16		1
4	35		9	
		43		
5	78			

The 3rd divided difference is **1** — exactly the leading coefficient of $x^3 - 2x^2 + 3$, confirming $f[x_0..x_3] = f'''(\xi)/3! = 6/6 = 1$ **even though the nodes are unequally spaced**. A 4th divided difference would be 0.

(ii) Top diagonal gives the coefficients:

$$P_3(x) = 2 + 1(x-1) + 5(x-1)(x-2) + 1(x-1)(x-2)(x-4)$$

(iii) At $x = 3$:

$$P_3(3) = 2 + 1(2) + 5(2)(1) + 1(2)(1)(-1) = 2 + 2 + 10 - 2 = \mathbf{12}$$

Exact: $3^3 - 2(9) + 3 = 27 - 18 + 3 = 12$ ✓ (as it must be — a cubic interpolated at 4 points is reproduced exactly, so the error term $f^{(4)}(\xi)/4! \prod(x - x_i)$ vanishes identically).

(iv) Lagrange at $x = 3$ gives **12** as well. The two forms are algebraically the same polynomial; Newton simply arranges it so that a fifth data point would append one term and leave 2, 1, 5, 1 untouched, whereas Lagrange would rebuild all four basis polynomials.

P4. A spline is to be built through **6** data points. Give the complete unknown/equation count for the quadratic and the cubic case, and show each balances.

Solution. With $k = 6$ points there are $n = k - 1 = 5$ intervals (this is the step everyone gets wrong).

Quadratic: $P_i = a_i x^2 + b_i x + c_i$, so $3n = 15$ unknowns.

$$\underbrace{2n = 10}_{\text{each piece through its 2 endpoints}} + \underbrace{(n - 1) = 4}_{P' \text{ matched at 4 interior knots}} + \underbrace{1}_{\text{assumption } P_1'' = 0 \text{ at one end}} = 15 \checkmark$$

Cubic: $P_i = a_i x^3 + b_i x^2 + c_i x + d_i$, so $4n = 20$ unknowns.

$$\underbrace{2n = 10}_{\text{through points}} + \underbrace{(n - 1) = 4}_{P' \text{ matched}} + \underbrace{(n - 1) = 4}_{P'' \text{ matched}} + \underbrace{2}_{\text{boundary conditions}} = 20 \checkmark$$

The 2 boundary conditions are $P'' = 0$ at both ends (natural) *or* the two end slopes (clamped).

P5. Use linear interpolation on the table (10, 100), (20, 400) to estimate y at $x = 13$, and explain why you may not use the pair (20, 400), (30, 900) instead.

Solution. Bracketing pair is (10, 100) and (20, 400) since $10 < 13 < 20$:

$$\frac{400 - 100}{20 - 10} = \frac{y_m - 100}{13 - 10} \implies 30 = \frac{y_m - 100}{3} \implies y_m = 190.$$

Using (20, 400), (30, 900) would be *extrapolation*, not interpolation: $x = 13$ lies outside $[20, 30]$, so the straight line there carries no information about the function's behaviour near 13 and the estimate would be far off (it would give $y = 400 + 50(13 - 20) = 50$).

Exam Traps

- **Points vs intervals.** k points give $n = k - 1$ splines. A “cubic spline through 6 points” has $4 \times 5 = 20$ unknowns, not 24.
- **Lagrange and Newton give the same polynomial.** Any option claiming different values is wrong — the interpolant is unique.
- **“At most degree n ”, never “exactly”.** Collinear points collapse the degree.
- **The n -th divided difference of a degree- n polynomial is the leading coefficient** (and the next one is exactly 0) — true for unequal spacing too.
- **More points can make a global interpolant worse** (Runge), and the damage is worst at the *edges*, not the middle.
- **Interpolation is not regression.** If a question mentions noisy data and best fit, a high-order interpolant is the wrong tool.
- **Always use the *bracketing* pair** for linear interpolation — otherwise you are extrapolating.
- **A linear spline is continuous but has corners.** “Smooth” requires at least a quadratic (matching P').
- **Natural vs clamped is about the 2 boundary equations only** — both give $4n$ unknowns in total.

- **Higher spline order is not uniformly better**; the notes flag the smoothness-versus-cost-and-error trade-off explicitly.

14 Simulation, RNG & Monte Carlo — Problem Bank

14.1 Simulation & Modeling — MCQs

MCQ / Rapid-Fire Bank

Q1. A queue holds 0 customers for 45 minutes and 6 customers for 15 minutes. The time-average queue length is

- (a) 3 (b) 1.5 (c) 6 (d) 0.75

Ans: (b) — $(0 \times 45 + 6 \times 15)/60 = 1.5$. Averaging the two *values* to get 3 ignores how long each lasted.

Q2. Customers arrive at a rate of 12/hour and a single server handles 15/hour. The utilization is

(a) 1.25 (b) 0.8 (c) 0.2 (d) 180

Ans: (b) — $\rho = \lambda/\mu = 12/15 = 0.8$, comfortably inside the healthy 70–85% band.

Q3. If $\lambda > \mu$ in a single-server queue, then as time grows

- (a) $\hat{u}$ exceeds 1 (b) the queue length grows without bound (c) the system reaches steady state
(d) the delay approaches zero

Ans: (b) — utilization saturates at 1 (it is a fraction of time); it is $Q(t)$ that explodes.

Q4. In a discrete-event simulation, state variables change

- (a) continuously (b) at fixed time steps (c) only at events (d) at every clock tick

Ans: (c)

Q5. Which of these is a *continuous-time* (time-weighted) statistic?

- (a) $\hat{d}(n)$, the average delay (b) $\hat{q}(n)$, the time-average queue length
(c) the number of customers served (d) the total delay

Ans: (b) — $\hat{q}$ and $\hat{u}$ are areas over time; $\hat{d}$ is a plain mean over customers.

Q6. The area accumulator at an event must use

- (a) the new state value (b) the old state value (c) the average of the two (d) either, it does not matter

Ans: (b) — the old value is what actually held during the interval just elapsed. Getting this wrong is named in the notes as the most common simulation bug.

Q7. When the queue empties and the server goes idle, the next departure time is set to ∞ in order to

- (a) end the simulation (b) prevent a “phantom departure” being scheduled for a non-existent customer
(c) save memory (d) force the clock to advance by one unit

Ans: (b)

Q8. A simulation model passes all verification checks but its predictions disagree with the real system. What failed?

- (a) verification (b) validation (c) calibration only (d) nothing; models always disagree

Ans: (b) — the code correctly implements a model whose assumptions are wrong.

Q9. In the 10-step simulation study, a failed validity check at Step 6 sends you back to

- (a) Step 5 (b) Step 4 (c) Step 2 (d) Step 1

Ans: (c) — invalid output usually means the conceptual model or the data is wrong, not just the code.

Q10. Which statement is **FALSE**?

- (a) Next-event time advance skips idle periods (b) Simulated time equals real computer time
(c) The event list holds future events and their times (d) Stochastic models produce random outputs

Ans: (b) — 24 simulated hours may take 0.01 s, and 1 ns of a network may take 10 minutes.

14.2 RNG & Monte Carlo — MCQs

MCQ / Rapid-Fire Bank

Q11. For $R \sim U(0, 1)$, $\text{Var}(R)$ equals

- (a) $1/2$ (b) $1/3$ (c) $1/12$ (d) $1/4$

Ans: (c) — $E[R^2] - E[R]^2 = \frac{1}{3} - \frac{1}{4} = \frac{1}{12} \approx 0.0833$.

Q12. An LCG has $X_{i+1} = (5X_i + 3) \bmod 16$ with $X_0 = 1$. Then X_2 is

- (a) 8 (b) 11 (c) 10 (d) 5

Ans: (b) — $X_1 = 8$, $X_2 = (40 + 3) \bmod 16 = 43 \bmod 16 = 11$.

Q13. Does that generator achieve the full period $m = 16$?

- (a) no, because m is not prime (b) yes: $\gcd(3, 16) = 1$ and $5 = 1 + 4(1)$
(c) no, because $c \neq 0$ (d) only for even seeds

Ans: (b) — both mixed-generator conditions hold, and the period really is 16.

Q14. For the multiplicative generator $X_{i+1} = 13X_i \bmod 64$, the maximum achievable period is

- (a) 64 (b) 63 (c) 16 (d) 32

Ans: (c) — with $c = 0$ and $m = 2^b$ the ceiling is $m/4 = 16$, attained only for *odd* seeds.

Q15. The values an LCG can produce after scaling are spaced

- (a) $1/m$ apart (b) $1/a$ apart (c) continuously (d) $1/c$ apart

Ans: (a) — $R_i \in \{0, 1/m, \dots, (m-1)/m\}$, which is why a large m matters.

Q16. A sequence gives 0.01, 0.02, 0.03, ..., 0.99. Which test will flag it?

- (a) chi-square (b) Kolmogorov–Smirnov (c) autocorrelation (d) none; it is a good generator

Ans: (c) — it is perfectly uniform (so both uniformity tests pass) but utterly predictable. Only an independence test catches it.

Q17. In the autocorrelation test with $i = 2$, $\ell = 4$, $N = 30$, the value of M is

- (a) 5 (b) 6 (c) 7 (d) 4

Ans: (b) — largest M with $i + (M + 1)\ell \leq N$: $2 + 7(4) = 30 \leq 30 \checkmark$, while $2 + 8(4) = 34 > 30$.

Q18. The 0.25 subtracted in $\hat{\rho}_{i\ell}$ arises because

- (a) it is an arbitrary correction (b) $E[R]E[R'] = \frac{1}{2} \cdot \frac{1}{2}$ for independent uniforms
(c) $\text{Var}(R) = 1/12$ (d) there are 4 quadrants

Ans: (b) — it centres the estimator at zero under independence.

Q19. An RNG is subjected to 20 independent tests at $\alpha = 0.05$ and fails 1. You should

- (a) reject it immediately (b) accept it: about 64% of good generators fail at least one of 20 tests
(c) conclude the tests are broken (d) rerun with $\alpha = 0.5$

Ans: (b) — $1 - 0.95^{20} = 0.6415$, and the expected number of chance failures is $20(0.05) = 1$.

Q20. A Monte Carlo estimate must be made 4 times more accurate. The sample size must grow by a factor of

- (a) 4 (b) 8 (c) 16 (d) 2

Ans: (c) — error $\propto 1/\sqrt{n}$, so n scales as 4^2 .

Q21. In estimating π by hit-or-miss, $\hat{\pi} = 4h/n$ because

- (a) the circle has radius 4 (b) $P(\text{hit}) = (\pi/4)/1$ for a quarter circle in the unit square
(c) there are 4 quadrants to sum (d) 4 normalizes the variance

Ans: (b)

Q22. Which statement is **FALSE**?

- (a) Passing all tests does not prove a generator is random (b) K-S works for small samples; chi-square needs large N
(c) A longer period guarantees independence (d) The seed determines the entire sequence

Ans: (c) — period length says nothing about correlation structure within the cycle.

14.3 Simulation, RNG & Monte Carlo — Problems

Practice Problems (with full solutions)

P1. A single-server queue receives 4 customers with interarrival times 0.3, 0.9, 0.6, 1.2 and service times 1.2, 0.8, 1.5, 0.5. Build the trace and compute $\hat{d}(4)$, $\hat{q}(4)$, $\hat{u}(4)$ over the run ending when customer 4 departs. Then test Little's Law, and comment on stability.

Solution. Arrival times are the cumulative sums: 0.3, 1.2, 1.8, 3.0.

Cust.	Arrives	Starts service	Departs	Delay D_i
1	0.3	0.3	1.5	0.0
2	1.2	1.5	2.3	0.3
3	1.8	2.3	3.8	0.5
4	3.0	3.8	4.3	0.8

(Each customer starts at $\max(\text{own arrival, previous departure})$.) Run length $T = 4.3$.

$$\hat{d}(4) = \frac{0 + 0.3 + 0.5 + 0.8}{4} = \frac{1.6}{4} = \mathbf{0.4}$$

Queue area: customer i waits over $[\text{arrival, start})$, contributing that length to $\int Q dt$:

$$\int_0^{4.3} Q(t) dt = 0 + 0.3 + 0.5 + 0.8 = 1.6 \quad \Rightarrow \quad \hat{q}(4) = \frac{1.6}{4.3} = \mathbf{0.372093}$$

Busy area: the server runs continuously from 0.3 to 4.3:

$$\int_0^{4.3} B(t) dt = 4.0 \quad \Rightarrow \quad \hat{u}(4) = \frac{4.0}{4.3} = \mathbf{0.930233}$$

Little's Law: $\lambda = 4/4.3 = 0.930233$, $W_q = \hat{d}(4) = 0.4$, so

$$\lambda W_q = 0.930233 \times 0.4 = 0.372093 = \hat{q}(4) \quad \checkmark$$

It holds *exactly* here because every customer's queueing time was completed inside the window — nobody was left waiting at T .

Stability: $E[A] = 3.0/4 = 0.75$ so $\lambda = 1.333$; $E[S] = 4.0/4 = 1.0$ so $\mu = 1.0$; $\rho = 1.333 > 1$. The system is **unstable** — work arrives faster than it clears, which is why each successive delay (0, 0.3, 0.5, 0.8) is larger than the last.

P2. For the LCG $X_{i+1} = (5X_i + 3) \bmod 16$ with $X_0 = 1$: generate 6 values and their R_i , and determine whether the full period $m = 16$ is achieved.

Solution.

$$\begin{aligned} X_1 &= (5 + 3) \bmod 16 = 8, & X_2 &= (40 + 3) \bmod 16 = 11, & X_3 &= (55 + 3) \bmod 16 = 10 \\ X_4 &= (50 + 3) \bmod 16 = 5, & X_5 &= (25 + 3) \bmod 16 = 12, & X_6 &= (60 + 3) \bmod 16 = 15 \end{aligned}$$

X_i	8	11	10	5	12	15
$R_i = X_i/16$	0.5000	0.6875	0.6250	0.3125	0.7500	0.9375

Max-period test (mixed generator, $c \neq 0$, $m = 2^4$): need (i) $\gcd(c, m) = 1$ and (ii) $a = 1 + 4k$.

$$\gcd(3, 16) = 1 \quad \checkmark, \quad a = 5 = 1 + 4(1) \quad \checkmark$$

Both hold, so the period is the full **16**: every integer 0..15 appears exactly once before repeating.

Contrast: the generator $X_{i+1} = (3X_i + 7) \bmod 16$ has $\gcd(7, 16) = 1$ but $a = 3$ is *not* of the form $1 + 4k$, and its period is only **8**.

P3. 120 random numbers are binned into 6 equal intervals with observed counts 18, 22, 25, 17, 20, 18. Perform a chi-square uniformity test at $\alpha = 0.05$ (critical value $\chi_{0.05,5}^2 = 11.07$).

Solution. $E_i = N/n = 120/6 = 20$ for every bin.

O_i	18	22	25	17	20	18	120
$(O_i - E_i)^2$	4	4	25	9	0	4	—
$(O_i - E_i)^2/E_i$	0.2	0.2	1.25	0.45	0.0	0.2	2.30

$$\chi_0^2 = 2.30, \quad \text{df} = n - 1 = 5, \quad \chi_{0.05,5}^2 = 11.07$$

Since $2.30 < 11.07$, **do not reject** H_0 — no evidence against uniformity. Note this says nothing about *independence*.

P4. A Monte Carlo estimator has $\sigma = 0.25$. (i) How many samples give a 95% confidence half-width of at most 0.01? (ii) How many for 0.0025? (iii) State the general scaling law.

Solution. (i) Half-width = $1.96\sigma/\sqrt{n} \leq 0.01$:

$$\sqrt{n} \geq \frac{1.96(0.25)}{0.01} = 49 \quad \Rightarrow \quad n \geq \mathbf{2401}$$

(ii) The target is $4\times$ tighter, so $\sqrt{n}$ must grow $4\times$ and n by $4^2 = 16$:

$$n \geq 16(2401) = \mathbf{38,416}$$

(Direct check: $(1.96 \times 0.25/0.0025)^2 = 196^2 = 38416 \checkmark$.)

(iii) Error $\propto \sigma/\sqrt{n}$, so **to improve accuracy by a factor f you need f^2 times as many samples**. Ten times the accuracy costs a hundred times the work — the fundamental limitation of Monte Carlo.

P5. A correct random-number generator is subjected to (i) 10 independent tests, (ii) 100 independent tests, all at $\alpha = 0.05$. In each case give the probability of at least one false rejection and the expected number of rejections. Then state the practical rule.

Solution. Using $P(\geq 1 \text{ false reject}) = 1 - (1 - \alpha)^k$ and expected count = $k\alpha$:

k	$1 - 0.95^k$	expected rejections
1	0.0500	0.05
5	0.2262	0.25
10	0.4013	0.50
20	0.6415	1.00
50	0.9231	2.50
100	0.9941	5.00

(i) 10 tests: about a **40%** chance of at least one failure, expected 0.5 failures. (ii) 100 tests: a **99.4%** chance of at least one failure, expected **5** failures.

Practical rule: a handful of failures among many tests is exactly what a *good* generator produces. Worry only when the failure count is well above $k\alpha$, when the *same* test fails repeatedly, or when several *different* tests fail together.

Exam Traps

- **Time-average $\neq$ average of the values.** Weight every value by how long it lasted (0 for 45 min and 6 for 15 min gives 1.5, not 3).
- **$\hat{u}$ can never exceed 1.** An overloaded system shows $\hat{u} \rightarrow 1$ and a diverging $Q(t)$, not $\hat{u} > 1$.
- **$\rho = \lambda/\mu$ uses *rates*.** If the problem gives mean times, invert first: $\lambda = 1/E[A]$, $\mu = 1/E[S]$.
- **Areas use the old state value** and width (clock – time of last event), in that order.
- **$\hat{q}$ counts only those *waiting***, excluding the customer in service; $\hat{u}$ counts the server, not the

queue.

- **Little's Law is asymptotic.** It matches exactly only when the customers counted and the area measured cover the same population — leftover waiters at time T break it.
- **Verification $\neq$ validation**, and both validation failures return to Step 2.
- **Uniformity does not imply independence.** 0.01, 0.02, 0.03, ... passes K-S and chi-square and is completely deterministic.
- **M in the autocorrelation test is the largest integer with $i + (M + 1)\ell \leq N$** — off-by-one here wrecks both $\hat{\rho}$ and σ .
- **Chi-square df is (bins -1)**, not the sample size and not the number of bins.
- **A single test failure does not condemn a generator** — with 10 tests a good one fails at least once about 40% of the time.
- **Monte Carlo accuracy costs quadratically.** “Twice as accurate” means four times the samples, not twice.
- **The multiplicative case ($c = 0$, $m = 2^b$) caps out at $m/4$** , and only for odd seeds — it can never reach the full m .

Part III — Mock Quiz

A full mixed paper, 60 minutes, answer key at the end

15 Mock Quiz — Full Mixed Paper (attempt under exam conditions)

Key Point / Exam Tip

How to use this. Topic-by-topic drilling makes everything feel easy because you already know which method to apply. A real quiz does not tell you. Sit this paper cold, no notes, **60 minutes**, then mark yourself with the key at the end. Anything you miss, go back to that topic's section in Part I/II.

Structure: Part A — 35 MCQs (1 mark each). Part B — 10 short answers (2 marks each). Part C — 8 problems (5 marks each). Total 95.

15.1 Part A — Multiple Choice (35 marks)

MCQ / Rapid-Fire Bank

Q1. If the relative approximate error of an iterative scheme is $|\epsilon_a| = 0.04\%$, how many significant digits are guaranteed correct? [1]

- (a) 2 (b) 3 (c) 4 (d) 5

Q2. The true value is 2.71828 and the approximation is 2.71. The absolute relative true error is closest to: [1]

- (a) 0.03% (b) 0.30% (c) 3.05% (d) 0.83%

Q3. A forward-difference derivative estimate is computed with smaller and smaller h . As $h \rightarrow 0$ on a real machine, the total error: [1]

- (a) decreases monotonically to zero (b) stays constant
(c) decreases, reaches a minimum, then increases (d) increases monotonically

Q4. Bisection starts on $[1, 2]$. The least number of iterations guaranteeing an interval width below 10^{-3} is: [1]

- (a) 8 (b) 9 (c) 10 (d) 11

Q5. Bisection begins on $[0, 0.11]$. The interval width after 5 iterations is: [1]

- (a) 0.0034375 (b) 0.006875 (c) 0.0011 (d) 0.022

Q6. Newton–Raphson is applied to $f(x) = x^2 - 2$ with $x_0 = 1$. Then x_2 equals: [1]

- (a) 1.5 (b) 1.416667 (c) 1.414216 (d) 1.333333

Q7. Which statement about the false-position method is **FALSE**? [1]

- (a) It is a bracketing method (b) It always converges faster than bisection
(c) One endpoint can stay frozen for many iterations (d) It uses a secant line through the bracket endpoints

Q8. Newton–Raphson applied to a root of multiplicity 2 converges: [1]

- (a) quadratically (b) linearly with rate $1/2$ (c) cubically (d) not at all

Q9. In naive Gauss elimination on an $n \times n$ system, the total number of multipliers m_{ik} computed is: [1]

- (a) n^2 (b) $n(n-1)/2$ (c) $n(n+1)/2$ (d) $n-1$

Q10. For $n = 4$, that number is: [1]

- (a) 4 (b) 6 (c) 10 (d) 16

Q11. After forward elimination with **two** row swaps, the diagonal of U multiplies to -1 . Then $\det A$ is: [1]

(a) -1 (b) $+1$ (c) -2 (d) cannot be determined

Q12. Compared with Gauss elimination, LU decomposition for a **single** right-hand side costs: [1]
 (a) about half as much (b) exactly the same (c) about twice as much (d) $n/4$ times less

Q13. A 2×2 matrix has $\text{tr } A = 7$ and $\det A = 12$. Its eigenvalues are: [1]
 (a) 2, 5 (b) 3, 4 (c) 1, 6 (d) $-3, -4$

Q14. If $\lambda = 2$ is an eigenvalue of A , then an eigenvalue of $A^3 - 2I$ is: [1]
 (a) 4 (b) 6 (c) 8 (d) 2

Q15. A matrix has eigenvalues 6 and 2. The power method's error decays per iteration by a factor of about: [1]
 (a) $1/2$ (b) $1/3$ (c) $1/6$ (d) 3

Q16. The power method will **fail to converge** to a single dominant direction when: [1]
 (a) the matrix is symmetric (b) $\lambda_1 = 5$ and $\lambda_2 = -5$
 (c) $\lambda_1 = 5$ and $\lambda_2 = 2$ (d) the starting vector is $[1, 1]^T$

Q17. Golden Section Search starts on $[0, 4]$. The interval width after **3** iterations is closest to: [1]
 (a) 0.5 (b) 0.944 (c) 1.528 (d) 2.472

Q18. Starting from an interval of width 1, the number of GSS iterations needed to shrink it below 0.01 is: [1]
 (a) 7 (b) 8 (c) 10 (d) 14

Q19. After 5 GSS iterations, the total number of *interior* function evaluations performed is: [1]
 (a) 5 (b) 6 (c) 10 (d) 11

Q20. The golden ratio R satisfies which identity? [1]
 (a) $R^2 = R + 1$ (b) $R^2 = 1 - R$ (c) $R^2 = 2R$ (d) $R^2 = 1 + R$

Q21. Gradient descent is run on $J(x) = 3(x - 2)^2$. It converges for exactly which learning rates? [1]
 (a) $0 < \alpha < 1/6$ (b) $0 < \alpha < 1/3$ (c) $0 < \alpha < 3$ (d) all $\alpha > 0$

Q22. Newton's method for optimization applied to a **quadratic** objective, from any starting point, converges in: [1]
 (a) exactly one step (b) two steps (c) infinitely many steps (d) it depends on the start

Q23. At a stationary point of $f(x, y)$ the Hessian has $\det H < 0$. The point is a: [1]
 (a) minimum (b) maximum (c) saddle point (d) cannot be classified

Q24. Newton's method for optimization is **dangerous** mainly because: [1]
 (a) it needs a learning rate (b) it converges only linearly
 (c) it can converge to a maximum or a saddle point (d) it needs a bracketing interval

Q25. A least-squares regression line with an intercept **always** passes through: [1]
 (a) the origin (b) the first data point (c) $(\bar{x}, \bar{y})$ (d) the median point

Q26. If $r = 0.8$, the percentage of variance in y explained by the model is: [1]
 (a) 80% (b) 64% (c) 89% (d) 40%

Q27. A scenario states that the regression errors follow a Poisson distribution. Which MLR assumption is violated? [1]
 (a) Linearity (b) Homoscedasticity (c) Normality (d) No multicollinearity

Q28. Given 5 data points, the interpolating polynomial has degree: [1]
 (a) exactly 5 (b) at most 4 (c) exactly 4 (d) at most 5

Q29. The data come from an exact cubic. The **4th** divided difference of the table is: [1]
 (a) the leading coefficient (b) 0 (c) 6 (d) undefined

Q30. A cubic spline is built through 6 data points. The number of unknown coefficients is: [1]
 (a) 18 (b) 20 (c) 24 (d) 16

Q31. Lagrange and Newton divided-difference interpolation on the same data set: [1]
 (a) give different polynomials (b) give the same polynomial in different form
 (c) give the same form (d) differ only at the nodes

Q32. A queue holds 0 customers for 9 hours and 10 customers for 1 hour. The time-average queue length is: [1]

- (a) 5 (b) 1.0 (c) 10 (d) 0.9

Q33. Customers arrive at 8/hour and one server serves 10/hour. The utilization ρ is: [1]

- (a) 0.8 (b) 1.25 (c) 0.2 (d) 80

Q34. An LCG has $X_0 = 5$, $a = 3$, $c = 7$, $m = 16$. Then X_3 is: [1]

- (a) 6 (b) 9 (c) 2 (d) 13

Q35. To reduce the error of a Monte Carlo estimate by a factor of 5, the number of samples must be multiplied by: [1]

- (a) 5 (b) 10 (c) 25 (d) 125

15.2 Part B — Short Answer (20 marks)

S1. State the unimodality assumption of Golden Section Search and explain exactly what goes wrong without it. [2]

S2. Bisection and false position are both bracketing methods, yet false position was measured as *slower* on $f(x) = \ln x$ over $[10^{-4}, 10^4]$. Explain the mechanism. [2]

S3. A model passes every verification check but still gives wrong predictions. Which of verification/validation failed, and why is that possible? [2]

S4. Why does the area accumulator in a discrete-event simulation use the state value from *before* the current event? [2]

S5. A generator produces 0.01, 0.02, 0.03, ..., 0.99 in order. Which of the two “sacred properties” does it satisfy, which does it violate, and which statistical test catches it? [2]

S6. Explain why LU decomposition costs the same as Gauss elimination for one right-hand side but wins dramatically when computing A^{-1} . [2]

S7. State what the Lagrange multiplier λ measures beyond simply locating the optimum. [2]

S8. Why does adding more equally spaced interpolation nodes sometimes make the fit *worse*? Name the phenomenon and where the error concentrates. [2]

S9. An RNG is subjected to 10 independent tests at $\alpha = 0.05$ and fails one. Should it be rejected? Justify quantitatively. [2]

S10. Distinguish a discrete-time statistic from a continuous-time statistic in simulation output analysis, giving one example of each. [2]

15.3 Part C — Problems (40 marks)

Practice Problems (with full solutions)

P1. Maximize $f(x) = -x^2 + 6x - 5$ on $[0, 5]$ using Golden Section Search. Perform **2** iterations showing x_1, x_2, f_1, f_2 and the surviving bracket. Then, *without iterating further*, state the bracket width after 6 iterations. [5]

P2. Solve using Gauss elimination **with partial pivoting**, showing every swap and multiplier:

$$\begin{bmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 8 \\ -11 \\ -3 \end{bmatrix}$$

Also report $\det A$ from your elimination. [5]

P3. For $A = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}$ with $\mathbf{x}_0 = [1, 0]^\top$, perform **4** power-method iterations (normalize by the largest-magnitude entry). Report each λ estimate and $|\epsilon_a|$ at iteration 4. Then confirm the exact eigenvalues using trace and determinant only, and state the theoretical convergence ratio. [5]

P4. Fit a least-squares line to $(1, 3), (2, 4), (3, 8), (4, 9), (5, 11)$. Report m , b , r and r^2 . Then **verify** two structural properties: that the line passes through the centroid, and that the residuals sum to zero. [5]

P5. The function $y = 2x^3 - x + 1$ is sampled at $x = 0, 1, 2, 3$. Build the complete Newton divided-difference table, write $P_3(x)$, and explain what the 3rd divided difference equals and what the 4th would be. [5]

P6. A single-server queue receives 3 customers with interarrival times 0.5, 1.0, 0.8 and service times 1.5, 0.9, 1.2. Build the event trace, then compute $\hat{d}(3)$, $\hat{q}(3)$ and $\hat{u}(3)$ over the run ending when customer 3 departs. Finally check Little's Law $L_q = \lambda W_q$. [5]

P7. An LCG is defined by $X_{i+1} = (3X_i + 7) \bmod 16$ with $X_0 = 5$. Generate 4 values and their R_i . Then determine whether this generator can achieve the maximum period $m = 16$, justifying with the max-period conditions. [5]

P8. A Monte Carlo estimator has $\sigma = 0.5$. How many samples are needed so that the 95% confidence interval half-width is at most 0.01? If a colleague wants half-width 0.002 instead, by what factor does n grow? [5]

15.4 Answer Key

MCQ / Rapid-Fire Bank

Part A.

- 1 (b) $m = \lfloor 2 - \log_{10}(2 \times 0.04) \rfloor = \lfloor 3.097 \rfloor = 3$.
- 2 (b) $|0.00828/2.71828| = 0.3046\%$.
- 3 (c) Truncation error falls with h but round-off rises as h shrinks: the total is U-shaped.
- 4 (c) $n \geq \log_2(1/10^{-3}) = 9.97 \Rightarrow 10$.
- 5 (a) $0.11/2^5 = 0.0034375$.
- 6 (b) $x_1 = 1.5$, $x_2 = 1.416667$ (note $x_3 = 1.414216$ — read the index carefully).
- 7 (b) On $\ln x$ over $[10^{-4}, 10^4]$ bisection took 34 iterations vs false position's 126.
- 8 (b) Multiplicity k gives linear convergence at rate $1 - 1/k = 1/2$.
- 9 (b) $(n-1) + (n-2) + \dots + 1 = n(n-1)/2$.
- 10 (b) $4 \cdot 3/2 = 6$.
- 11 (a) $\det A = (-1)^2 \times (-1) = -1$; an *even* number of swaps leaves the sign alone.
- 12 (b) The cost analysis gives $C_{T|LU} = C_{T|GE}$ exactly for one RHS.
- 13 (b) $\lambda^2 - 7\lambda + 12 = 0 \Rightarrow 3, 4$.
- 14 (b) $\lambda^3 - 2 = 8 - 2 = 6$.
- 15 (b) $\text{decay} = |\lambda_2/\lambda_1| = 2/6 = 1/3$.
- 16 (b) Equal magnitudes mean no strictly dominant eigenvalue; the iterate oscillates.
- 17 (b) $L_3 = 4R^3 = 4(0.236068) = 0.944$. *This is the past-exam question.*
- 18 (c) $n \geq \ln(0.01)/\ln(0.618034) = 9.57 \Rightarrow 10$.
- 19 (b) 2 interior evaluations on iteration 1, then 1 each: $2 + 4 = 6$.
- 20 (b) $R^2 + R - 1 = 0 \Rightarrow R^2 = 1 - R = 0.381966$.
- 21 (b) $J = a(x-2)^2$ with $a = 3$; converges iff $0 < \alpha < 1/a = 1/3$.
- 22 (a) The second-order Taylor model is exact for a quadratic, so the first step lands on the vertex.
- 23 (c) $\det H < 0$ means eigenvalues of opposite sign: a saddle.
- 24 (c) It solves $f' = 0$, which is satisfied by minima, maxima and saddles alike.
- 25 (c) Follows directly from $b = \bar{y} - m\bar{x}$.
- 26 (b) $r^2 = 0.64$.
- 27 (c) Poisson errors are not normal.
- 28 (b) $(n+1)$ points give *at most* degree n ; collinear data collapse to lower degree.
- 29 (b) The 3rd divided difference of a cubic is its leading coefficient (constant); the 4th is 0.
- 30 (b) 6 points $\Rightarrow n = 5$ intervals $\Rightarrow 4n = 20$.
- 31 (b) The interpolating polynomial is unique; only the representation differs.
- 32 (b) $(0 \cdot 9 + 10 \cdot 1)/10 = 1.0$. Averaging 0 and 10 to get 5 is the trap.
- 33 (a) $\rho = \lambda/\mu = 8/10 = 0.8$.
- 34 (c) $X_1 = 6$, $X_2 = 9$, $X_3 = 2$.
- 35 (c) Error $\propto 1/\sqrt{n}$, so n scales by $5^2 = 25$.

Part B (model answers, condensed).

- S1.** Exactly one extremum in $[x_L, x_u]$. Without it, the “keep the better side” comparison can discard the sub-interval that actually contains the global optimum, and the method converges confidently to the wrong point.
- S2.** False position replaces one endpoint each iteration, but if the function is strongly curved, one endpoint can remain *frozen* indefinitely while the other creeps; the bracket then shrinks far slower than bisection's guaranteed halving (34 vs 126 iterations in the measured test).
- S3.** Validation failed. Verification only asks “does the code implement the conceptual model correctly?”; the model's assumptions themselves can be wrong, so a perfectly verified program can faithfully implement a model that does not match reality.
- S4.** Because the rectangle's height must be the state that actually held *during* the elapsed interval [time of last event, clock]. The current event changes the state at its right endpoint, so using the

post-update value would attribute the new state to time it did not occupy.

S5. It satisfies **uniformity** (the values are evenly spread over $(0, 1)$) but violates **independence** (each value perfectly predicts the next). Uniformity tests (K-S, chi-square) ignore ordering and would pass it; the **autocorrelation test** catches it.

S6. For one RHS the total work is identical — LU just relabels the same elimination arithmetic. The payoff is that L and U do not depend on the RHS: computing A^{-1} means n solves, and LU pays the $O(n^3)$ decomposition *once* and then n cheap $O(n^2)$ substitutions, while Gauss redoes the full $O(n^3)$ elimination n times — ratio $\approx n/4$.

S7. λ is the sensitivity or shadow price: it measures how much the optimal objective value changes per unit relaxation of the constraint, and it carries scale information about how hard the constraint is binding.

S8. Runge's phenomenon: more equally spaced nodes force a higher-degree global polynomial, which oscillates increasingly violently. The error concentrates at the **edges** of the interval. The fix is local low-order pieces — splines.

S9. No. Under a correct generator, $P(\text{at least one false rejection}) = 1 - (0.95)^{10} \approx 0.40$, so a single failure in 10 tests is the *expected* outcome about 40% of the time. Concern is warranted only for repeated failures of the same test or failures across several different tests.

S10. A discrete-time statistic averages over a discrete index, e.g. $\hat{d}(n) = \frac{1}{n} \sum D_i$ (average delay). A continuous-time statistic is a time-weighted average of a function of time, e.g. $\hat{q}(n) = \int_0^T Q(t) dt / T$ (time-average queue length) or $\hat{u}(n)$.

Practice Problems (with full solutions)

Part C solutions.

P1. $R = 0.618034$, $L_0 = 5$.

Iteration 1: $d = R(5 - 0) = 3.09017$; $x_1 = 5 - 3.09017 = 1.90983$, $x_2 = 0 + 3.09017 = 3.09017$. $f(x_1) = 2.81153$, $f(x_2) = 3.99187$. Since $f_1 < f_2$ the maximum lies to the right: $x_L = x_1 = 1.90983$. New bracket $[1.90983, 5]$, width $3.09017 = 5R$. ✓

Iteration 2: $d = R(5 - 1.90983) = 1.90983$; $x_1 = 3.09017$ (reused), $x_2 = 3.81966$. $f(x_1) = 3.99187$, $f(x_2) = 3.32816$. Now $f_1 > f_2$, so the maximum is to the left: $x_u = x_2 = 3.81966$. Bracket $[1.90983, 3.81966]$, width $1.90983 = 5R^2$. ✓

Shortcut: width after 6 iterations $= L_0 R^6 = 5(0.055728) = \boxed{0.27864}$ — no iteration required. (True optimum $x^* = 3$ lies in every bracket.)

P2. Column 1 candidates $|2|, |-3|, |-2|$: pivot is -3 , **swap** $R_1 \leftrightarrow R_2$. Multipliers $m_{21} = 2/(-3) = -0.666667$, $m_{31} = -2/(-3) = 0.666667$:

$$\begin{bmatrix} -3 & -1 & 2 & -11 \\ 0 & 0.333333 & 0.333333 & 0.666667 \\ 0 & 1.666667 & 0.666667 & 4.333333 \end{bmatrix}$$

Column 2 candidates $|0.333333|, |1.666667|$: **swap** $R_2 \leftrightarrow R_3$. Then $m_{32} = 0.333333/1.666667 = 0.2$:

$$\begin{bmatrix} -3 & -1 & 2 & -11 \\ 0 & 1.666667 & 0.666667 & 4.333333 \\ 0 & 0 & 0.2 & -0.2 \end{bmatrix}$$

Back substitution: $x_3 = -1$; $1.666667x_2 + 0.666667(-1) = 4.333333 \Rightarrow x_2 = 3$; $-3x_1 - 3 - 2 = -11 \Rightarrow x_1 = 2$.

$$(x_1, x_2, x_3) = (2, 3, -1)$$

Determinant: 2 swaps (even), so $\det A = (-1)^2(-3)(1.666667)(0.2) = -1$.

P3. $\mathbf{x}_0 = [1, 0]^\top$:

Iter	$A\mathbf{x}_k$	λ estimate	$\mathbf{x}_{k+1}$
1	$[4, 2]^\top$	4.00000	$[1, 0.5]^\top$
2	$[4.5, 3.5]^\top$	4.50000	$[1, 0.77778]^\top$
3	$[4.77778, 4.33333]^\top$	4.77778	$[1, 0.90698]^\top$
4	$[4.90698, 4.72093]^\top$	4.90698	$[1, 0.96209]^\top$

$|\epsilon_a|$ at iteration 4 = $|(4.90698 - 4.77778)/4.90698| \times 100\% = 2.633\%$.

Exact check without the characteristic polynomial: $\text{tr } A = 7$ and $\det A = 10$, so $\lambda^2 - 7\lambda + 10 = 0 \Rightarrow \lambda = 5, 2$. The iterates are climbing toward 5. Convergence ratio = $|\lambda_2/\lambda_1| = 2/5 = 0.4$ per iteration.

P4. $n = 5$: $\sum x = 15$, $\sum y = 35$, $\sum xy = 126$, $\sum x^2 = 55$, $\sum y^2 = 291$.

$$m = \frac{5(126) - 15(35)}{5(55) - 15^2} = \frac{630 - 525}{275 - 225} = \frac{105}{50} = 2.1$$

$$b = \frac{35 - 2.1(15)}{5} = \frac{35 - 31.5}{5} = 0.7 \quad \Rightarrow \quad \hat{y} = 2.1x + 0.7$$

$$r = \frac{105}{\sqrt{(50)(5(291) - 35^2)}} = \frac{105}{\sqrt{50 \times 230}} = 0.979130, \quad r^2 = 0.958696$$

Centroid: $\bar{x} = 3$, $\bar{y} = 7$; $\hat{y}(3) = 2.1(3) + 0.7 = 7 = \bar{y}$. ✓ *Residuals:* $y - \hat{y} = (0.2, -0.9, 1.0, -0.1, -0.2)$, summing to 0. ✓

P5. $y(0) = 1$, $y(1) = 2$, $y(2) = 15$, $y(3) = 52$.

x_i	y_i	1st d.d.	2nd d.d.	3rd d.d.
0	1			
		1		
1	2		6	
		13		2
2	15		12	
		37		
3	52			

$$P_3(x) = 1 + 1(x - 0) + 6(x - 0)(x - 1) + 2(x - 0)(x - 1)(x - 2)$$

(Expanding confirms $P_3(x) = 2x^3 - x + 1$ exactly.) The 3rd divided difference equals 2, which is precisely the **leading coefficient** of the cubic — consistent with $f[x_0..x_n] = f^{(n)}(\xi)/n!$ and $f''' = 12$, giving $12/3! = 2$. The 4th divided difference would be **0**, because the 4th derivative of a cubic vanishes.

P6. Arrival times (cumulative): 0.5, 1.5, 2.3.

Cust.	Arrives	Service starts	Departs	Delay D_i
1	0.5	0.5	2.0	0
2	1.5	2.0	2.9	0.5
3	2.3	2.9	4.1	0.6

Run ends at $T = 4.1$.

$$\hat{d}(3) = \frac{0 + 0.5 + 0.6}{3} = 0.366667$$

$Q(t) = 1$ on $[1.5, 2.0]$ (customer 2 waiting) and on $[2.3, 2.9]$ (customer 3 waiting):

$$\int_0^{4.1} Q(t) dt = 1(0.5) + 1(0.6) = 1.1 \quad \hat{q}(3) = \frac{1.1}{4.1} = 0.268293$$

The server is busy from 0.5 to 4.1:

$$\int_0^{4.1} B(t) dt = 3.6 \quad \hat{u}(3) = \frac{3.6}{4.1} = 0.878049$$

Little's Law: $\lambda = 3/4.1 = 0.731707$, $W_q = \hat{d}(3) = 0.366667$, so

$$\lambda W_q = 0.731707 \times 0.366667 = 0.268293 = \hat{q}(3). \quad \checkmark$$

P7. $X_1 = (15 + 7) \bmod 16 = 22 \bmod 16 = 6 \Rightarrow R_1 = 0.375$. $X_2 = (18 + 7) \bmod 16 = 25 \bmod 16 = 9 \Rightarrow R_2 = 0.5625$. $X_3 = (27 + 7) \bmod 16 = 34 \bmod 16 = 2 \Rightarrow R_3 = 0.125$. $X_4 = (6 + 7) \bmod 16 = 13 \Rightarrow R_4 = 0.8125$.

Max-period test: this is a *mixed* generator ($c = 7 \neq 0$) with $m = 16 = 2^4$. The conditions for $P = m$ are (i) $\gcd(c, m) = 1$ and (ii) $a = 1 + 4k$. Here $\gcd(7, 16) = 1$ ✓, but $a = 3$ and $3 = 1 + 4k$ has no integer solution ($k = 0.5$) ✗. **So the full period 16 is not achieved.**

P8. The 95% half-width is $1.96\sigma/\sqrt{n}$. Setting $1.96(0.5)/\sqrt{n} \leq 0.01$:

$$\sqrt{n} \geq \frac{1.96 \times 0.5}{0.01} = 98 \quad \Rightarrow \quad n \geq 9604$$

For half-width 0.002 the target is $5\times$ tighter, and since $\text{error} \propto 1/\sqrt{n}$, n grows by $5^2 = \mathbf{25}$, i.e. $n \geq 240100$.